

```

# Codebase

## `.gitignore`

```text
See https://help.github.com/articles/ignoring-files/ for more about
ignoring files.

dependencies
/node_modules
/.pnp
.pnp.*
.yarn/*
!.yarn/patches
!.yarn/plugins
!.yarn/releases
!.yarn/versions

testing
/coverage

next.js
/.next/
/out/

production
/build

misc
.DS_Store
*.pem

debug
npm-debug.log*
yarn-debug.log*
yarn-error.log*
.pnpm-debug.log*

env files (can opt-in for committing if needed)
.env*

vercel
.vercel

typescript
*.tsbuildinfo
next-env.d.ts
```

## `AGENTS.md`

```md
<!-- BEGIN:nextjs-agent-rules -->

This is NOT the Next.js you know

This version has breaking changes – APIs, conventions, and file structure may
all differ from your training data. Read the relevant guide in `node_modules/
next/dist/docs/` (resolved from this file's directory; in monorepos the
`next` package may not be visible from the repo root) before writing any

```

code. Heed deprecation notices.

This block is written and re-added by `next dev` – verify at `node\_modules/next/dist/server/lib/generate-agent-files.js`. Removing it from a diff only re-creates the uncommitted change; committing it with your work keeps the tree clean.

```
<!-- END:nextjs-agent-rules -->
```

```
...
```

```
`check.js`
```

```
```js
const { createClient } = require('@supabase/supabase-js');

const supabase = createClient(process.env.NEXT_PUBLIC_SUPABASE_URL,
  process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY);

async function check() {
  const { data, error } = await supabase
    .from('transactions')
    .select('*')
    .eq('project_id', '10000000-0000-0000-0000-000000000000');

  console.log(data);
}

check();
```

```
...
```

```
## `CLAUDE.md`
```

```
```md
@AGENTS.md
```

```
...
```

```
`Construction Site Management – Detailed Functional Requirements.md`
```

```
```md
# Construction Site Management – Detailed Functional Requirements
```

```
## 1. Purpose
```

Build a **mobile-first construction site management web application** for quickly recording and tracking labour, food, machinery, salaries, transport, materials, and cash transactions.

This document is **standalone** and specifically defines the detailed user workflows and data requirements for these modules.

The application should be designed for real-world use on a mobile phone by someone working at a construction site.

The user should rarely need to type information manually.

The application should rely heavily on:

- Dropdowns

- Searchable dropdowns
- Existing-item selection
- Quick-select buttons
- Date pickers
- Time pickers
- Auto-filled fields
- "Add New" options
- Optional comments
- Reusable people, buildings, materials, shops and machinery

2. Core UX principle

The application should work like this:

> Select !' Search !' Choose !' Enter amount/details !' Save.

Avoid long forms.

For example, if the user wants to record labour:

```
```text
LABOUR
!"
Building: [North 01 %4]
Worker: [Search worker...]
Work: [Soil Dressing %4]
Date: [30 Aug 2026]
Time: [10:30 AM]
Amount: 1 _____
Comments: _____
SAVE
```
```

The user should not have to manually type "North Building 01" every time.

3. Reusable master data

The application should maintain reusable records for:

People

Each person should have:

- Name
- Worker type
- Phone number – optional
- Notes – optional
- Active/inactive status

Worker types should include:

- Labour
- Mason
- Helper
- Electrician
- Plumber
- Carpenter

- Watchman
- Security
- Office Boy
- House Help
- Driver
- Operator
- Contractor
- Other

The user must be able to add new worker types.

4. Searchable selection

Whenever an existing person, material, shop, building, machinery item, etc. is required, use a searchable selector.

Example:

```
```text
Worker

[Ø=Ý Search worker]

Balakrishna
Bhadraiah
Ramesh
Suresh
```
```

Typing:

```
```text
bal
```
```

should immediately filter:

```
```text
B. Balakrishna
```
```

There should always be:

```
```text
+ Add New
```
```

if the required item doesn't exist.

5. Buildings / locations

The construction project consists of multiple buildings/locations.

The system should support building identifiers such as:

```
```text
North 01
```
```

North 02
North 03

West 01
West 02
West 03

East 01
East 02
East 03
```

The user should be able to create additional:

- Directions
- Building numbers
- Blocks
- Areas
- Other site locations

Structure:

```
```text
Building
    Direction
    Number
    Name (optional)
```
```

Example:

```
```text
Direction: North
Number: 05
Name: Villa 05
```
```

Display:

```
> **North 05**
```

rather than requiring the user to type the complete name.

---

## # 6. Work types

Labour must have predefined work types.

Initial work types:

- Soil Dressing
- Soil Leveling
- Shifting
- Cleaning
- Steel Unloading

The user must be able to add additional work types.

Examples of future additions:

- Brick Work
- Masonry
- Plastering
- Painting
- Electrical Work
- Plumbing Work
- Excavation
- Loading
- Unloading
- Transportation

Work types should be reusable and searchable.

---

## # 7. LABOUR MODULE

### ## 7.1 Purpose

Record labour work and labour-related payments.

A labour record should capture:

- Building/location
- Worker/labour name
- Worker type
- Work type
- Date
- Time
- Amount
- Payment source
- Person who provided cash, if applicable
- Optional comments

---

### ## 7.2 Labour entry

Example:

```
```text
```

```
ADD LABOUR
```

```
Building
[ North 05 %4 ]
```

```
Worker
[ Ø=Ý Search worker ]
```

```
Worker Type
[ Labour %4 ]
```

```
Work Type
[ Soil Dressing %4 ]
```

```
Date
[ 30 Aug 2026 ]
```

```
Time
[ 10:30 AM ]
```

Amount
[^ 1,200]
Payment
[Cash %4]
Cash Given By
[0=Y Search person]
Comments
[Optional]
[_SAVE]
``,`

8. Cash input for labour

The system needs to distinguish between:

Who performed the work

and

Who provided the money.

Example:

```text  
Worker:  
B. Balakrishna

Cash given by:  
Varun Sir

Amount:  
^13,200  
``,`

Therefore:

```text  
worker_id
cash_provider_id
``,`

must be separate fields.

Do not assume the person receiving money is the person who supplied the money.

9. Labour payment vs labour record

The system should allow:

Work record

```text  
Balakrishna

Soil Dressing  
North 05  
30 Aug  
```

and separately:

Payment

```text  
Balakrishna  
13,200  
Cash  
```

But the UI should make it easy to combine them when appropriate.

The user should not be forced to create duplicate records.

10. Labour comments

Comments are optional.

Examples:

```text  
Worked half day  
Extra work  
Shifted soil from North 05  
Worked with JCB  
```

Never make comments mandatory.

11. FOOD MODULE

The food module is for food, groceries and worker/site provisions.

A food record should contain:

- Person/worker
- Worker type
- Food category
- Date
- Start date / end date
- Amount – if applicable
- Shop/vendor – optional
- Comments

12. Food period

The user should be able to select:

Single day


```
```text
30 Aug 2026
```
```

or:

Date range

```
```text
From:
25 Aug 2026
```

```
To:
30 Aug 2026
```
```

The UI should make this very easy.

Provide:

```
```text
(•) Single Day
() Date Range
```
```

13. Food recipient

Example:

```
```text
Worker
[Ø=Ý Search]
```

```
Worker Type
[Watchman %¼]
```

```
Food
[Grocery %¼]
```

```
Date
[30 Aug 2026]
```

```
Comments
[Optional]
```
```

The worker type should normally auto-fill from the person's profile.

The user should still be able to change it if necessary.

14. Food categories

Initial categories:

- Food
- Groceries
- Vegetables

- Milk
- Drinking Water
- Tea/Snacks
- Kitchen Supplies
- Other

The user can add additional categories.

15. MACHINERY MODULE

Machinery should support:

- Crane
- Tractor
- JCB
- Compressor
- Excavator
- Other machinery

The user must be able to add additional machinery types.

16. Machinery record

Example:

```
```text
```

```
MACHINERY
```

```
Machine
```

```
[JCB %4]
```

```
Operator
```

```
[Ø=Ý Search operator]
```

```
Building
```

```
[North 05 %4]
```

```
Date
```

```
[30 Aug 2026]
```

```
Start Time
```

```
[09:00 AM]
```

```
End Time
```

```
[02:00 PM]
```

```
Hours
```

```
[5]
```

```
Amount
```

```
[^1 _____]
```

```
Comments
```

```
[Optional]
```

```
```
```

The system should be able to calculate:

```
```text
End Time - Start Time = Hours
```
```

when times are provided.

Alternatively, the user can directly enter hours.

17. Machinery flexibility

Not every machine will be charged by the hour.

Therefore support:

```
```text
Charging Type

Hourly
Daily
Fixed Amount
Load
Other
```
```

Example:

```
```text
JCB
5 hours
16,000
```
```

or:

```
```text
Crane
1 day
18,000
```
```

18. SALARY MODULE

Salary is different from ordinary labour payments.

Salary records should contain:

- Employee/worker
- Worker type
- From date
- To date
- Salary amount
- Payment date
- Payment method
- Person who provided cash, if applicable
- Comments

19. Salary worker types

Initial options:

- Watchman
- Security
- Office Boy
- House Help

Allow additional types.

20. Salary entry

Example:

```text  
SALARY

Employee  
[ Ø=Ý Search ]

Worker Type  
[ Watchman %¼ ]

Salary Period

From  
[ 01 Aug 2026 ]

To  
[ 31 Aug 2026 ]

Salary  
[ ¹ \_\_\_\_\_ ]

Payment Date  
[ 31 Aug 2026 ]

Payment Method  
[ Cash %¼ ]

Cash Given By  
[ Ø=Ý Search ]

Comments  
[ Optional ]

[ SAVE SALARY ]  
```\n

21. TRANSPORT MODULE

Transport must be linked to materials whenever possible.

This is important.

A transport record should not simply be:

```
```text
Transport 12,000
```
```

It should be possible to say:

```
```text
Material:
Sand

Supplier/Source:
XYZ
```

```
Transport:
Auto
```

```
From:
Location A
```

```
To:
North Site
```

```
Cost:
 12,000
```
```

22. Transport types

Initial options:

- Auto
- Tractor
- Truck
- Lorry
- Mini Truck
- Tempo
- Other

Allow additional transport types.

23. Transport-material relationship

Transport should support:

```
```text
Transport
 !"
Material Delivery
 !"
Material
```
```

Example:

```
```text
Material:
Sand

Load:
1

Weight:
10 tonnes

Transport:
Truck

Transport Cost:
 14,000
```
```

The transport record should be linked to that specific material delivery.

24. Material delivery

A delivery should have:

- Material
- Supplier/source
- Date
- Time – optional
- Quantity
- Unit
- Weight – optional
- Vehicle/transport type
- Transport record
- Destination/building/site
- Comments

25. MATERIAL MODULE

Materials should be divided into:

Large / bulk materials

Examples:

- Sand
- Steel
- Bricks
- Gravel
- RMC
- Stone
- Pipes
- Other bulk materials

Small purchased materials

Examples:

- Screws

- Electrical items
- Plumbing items
- Hardware
- Tools
- Fittings
- Small shop purchases

26. Bulk material entry

Large material entries do not necessarily require a cost.

The system should allow:

```text

MATERIAL DELIVERY

Material

[ Sand %4 ]

Date

[ 30 Aug 2026 ]

Quantity

[ 1 ]

Unit

[ Load %4 ]

Weight

[ 10 ]

Weight Unit

[ Tonnes %4 ]

Supplier/Source

[ Ø=Y Search ]

Destination

[ North 05 %4 ]

Transport

[ Link Transport ]

Comments

[ Optional ]

```

Cost should be optional.

27. Steel

Steel needs size/diameter information.

Example:

```text

Steel

```
Size
[8 mm %4]

Quantity
[5]

Unit
[Tonnes %4]

Weight
[5.2]

Date
[30 Aug 2026]
```
```

Steel sizes should be configurable.

Initial examples:

- 6 mm
- 8 mm
- 10 mm
- 12 mm
- 16 mm
- 20 mm
- 25 mm

Allow custom sizes.

28. Bricks

Bricks also need size/type.

Example:

```
```text
Bricks

Type/Size
[9 x 4 x 3 %4]

Quantity
[5,000]

Unit
[Pieces %4]

Weight
[Optional]

Date
[30 Aug 2026]
```
```

The user must be able to add custom brick sizes.

29. Pipes

Pipes should support:

- Pipe type
- Diameter/size
- Length
- Quantity
- Unit
- Supplier
- Date
- Destination

Example:

```
```text
PVC Pipe
4 inch
20 pieces
North 05
```
```

30. Small shop materials

Small items purchased from shops should support much more specific information.

Example:

```
```text
SHOP MATERIAL
```

```
Shop
[Ø=Ý Search shop]
```

```
Material
[Screw %¼]
```

```
Size
[2 inch %¼]
```

```
Quantity
[5]
```

```
Unit
[Boxes %¼]
```

```
Cost
[¹1,250]
```

```
Date
[30 Aug 2026]
```

```
Comments
[Optional]
```
```

31. Screws

Screws should specifically support:

- Type
- Size
- Number of boxes
- Units per box – optional
- Total quantity – optional
- Shop
- Cost
- Date

Example:

```
```text
Screw
Size: 2 inch
Boxes: 5
Shop: ABC Hardware
Cost: 11,250
```
```

32. Other small materials

The same flexible model should support:

```
```text
Electrical wire
Switches
Sockets
PVC fittings
Nails
Bolts
Nuts
Washers
Tape
Adhesive
Plumbing fittings
Hardware
```
```

Each item can optionally have:

- Size
- Brand
- Quantity
- Unit
- Boxes
- Cost
- Shop
- Date
- Comments

33. Shop master

Create reusable shops.

Each shop should contain:

```
```text
Shop Name
Location
Phone – optional
Notes – optional
```
```

Example:

```
```text
ABC Hardware
XYZ Electricals
Sandeep Kirana
```
```

The user can search:

```
```text
[0=Y Search shop]
```
```

and select an existing shop.

34. Cost rules for materials

The system must distinguish between:

Cost known

Example:

```
```text
Screws
5 boxes
11,250
```
```

and:

Cost not recorded

Example:

```
```text
Sand
10 tonnes
Cost not entered
```
```

Do not force a cost for bulk materials.

The system should allow:

```
```text
Cost
```
```

[Skip / Not Available]

35. Material + transport combined entry

The preferred workflow should allow:

```text

ADD MATERIAL

Material

[ Sand ]

Weight

[ 10 Tonnes ]

Supplier

[ XYZ ]

Destination

[ North 05 ]

Transport

[ + Add Transport ]

---

Then:

```text

TRANSPORT

Type

[Truck]

From

[Quarry]

Cost

[14,000]

Saving the material should automatically link the transport record.

36. Linking

Records should have relationships.

Example:

```text

Material Delivery #102

%

% % % Sand

% % % 10 tonnes

% % % XYZ Supplier

% % % North 05

%

```

 % % % Transport #55
 % % % Truck
 % % % 14,000
...

```

When viewing the material delivery, the user should be able to see:

```
> Transport cost: 14,000
```

When viewing the transport record, the user should be able to see:

```
> Material: Sand – 10 tonnes
```

---

### # 37. Date and time

Every major transaction should support a date.

Where useful, also support time.

Required date fields:

- Labour
- Food
- Machinery
- Salary payment
- Transport
- Materials

Time should be optional unless the workflow requires it.

The application should default to:

```
```text
Today
Current time
```

```

but allow modification.

---

### # 38. Comments

Comments should be available throughout the application.

They should always be optional.

Use a small expandable field:

```
```text
+ Add comment
```

```

rather than taking up space in the main form.

---

### # 39. Search everywhere

Search should be available for:

- Workers
- Employees
- Suppliers
- Shops
- Materials
- Buildings
- Work types
- Machinery
- Transport
- Transactions

Search should support partial matches.

Example:

```
```text
Search: bal
```
```

returns:

```
```text
B. Balakrishna
Balaji Electricals
```
```

---

# 40. Recent selections

The application should learn from usage.

If the user frequently selects:

```
```text
North 05
Labour
Balakrishna
Soil Dressing
Cash
```
```

these should appear under:

### Recent

This dramatically reduces data-entry time.

---

# 41. Add-new behavior

Every dropdown/search selector should have:

```
```text
+ Add New
```
```

Example:

```
```text
Worker
-----
Search...

Ramesh
Balakrishna
Suresh

+ Add New Worker
```
```

The user should not need to leave the current form.

---

#### # 42. Form persistence

If the user accidentally navigates away from a form, the application should warn them or preserve the entered values.

Do not silently discard partially entered financial information.

---

#### # 43. Validation

Required fields should be clearly indicated.

For labour:

Required:

- Worker
- Work type
- Date
- Amount, if recording a payment

Optional:

- Building
- Time
- Comments
- Cash provider

For material delivery:

Required:

- Material
- Date
- Quantity or weight

Optional:

- Cost
- Supplier
- Transport
- Destination
- Comments

---

#### # 44. Dashboard summaries

The dashboard should eventually provide:

```text

TODAY

Labour

¹_____

Food

¹_____

Machinery

¹_____

Materials

¹_____

Transport

¹_____

Salary

¹_____

```

And:

```text

THIS MONTH

Total expenditure

¹_____

```

---

#### # 45. Activity history

The application should show recent activity:

```text

Today

^{13,200}

Labour – Balakrishna

^{11,200}

Food – Sandeep Kirana

10 tonnes

Sand received

^{14,000}

Transport – Truck

Linked to Sand delivery

```



---

#### # 46. Filtering

Transactions should be filterable by:

- Date
- Date range
- Building
- Worker
- Worker type
- Work type
- Material
- Material size
- Shop
- Supplier
- Machinery
- Category
- Payment method
- Amount

---

#### # 47. Building-level view

A particularly useful feature should be:

```
```text
BUILDINGS
```

```
North 01
North 02
North 03
```

```
...
West 01
```

```
...
East 01
```
```

Selecting a building should show:

```
```text
North 05
```

```
Labour
  1 _____
```

```
Materials
  1 _____
```

```
Machinery
  1 _____
```

```
Transport
  1 _____
```

```
Total recorded
  1 _____
```
```

and the associated transactions.

---

#### # 48. Worker-level view

Selecting a worker should show:

```
```text
Balakrishna

Worker Type:
Labour

Labour Records:
...

Payments:
...

Total Paid:
  1 _____

Recent Work:
Soil Dressing
Shifting
Cleaning
```
```

---

#### # 49. Material-level view

Selecting:

```
```text
Sand
```
```

should show:

```
```text
Total loads
Total weight
Suppliers
Deliveries
Transport costs
```
```

The user should be able to see where the material went.

---

#### # 50. Important distinction

The application must keep these concepts separate:

### Work

What someone did.

### Payment

```
```text
+ ADD
```
```

and choose:

```
```text
Labour
Food
Machinery
Salary
Transport
Material
Money In
Payment
```
```

---

# 53. Do not overload the first screen

Advanced fields should be hidden under:

```
```text
+ More details
```
```

For example:

Labour initially:

```
```text
Worker
Work
Building
Amount
Date
```
```

Advanced:

```
```text
Time
Cash provider
Worker type
Comments
```
```

---

# 54. Database design principles

The implementation should avoid separate databases/tables for:

```
```text
North labour
West labour
Cement
Sand
Steel
Bricks
```
```

Instead, use normalized master data.

Core entities:

```
```text
Person
Building
WorkType
Material
MaterialVariant
Shop
Supplier
Machinery
TransportType
Account
Transaction
MaterialDelivery
TransportRecord
LabourRecord
FoodRecord
SalaryRecord
```
```

Relationships should be used rather than duplicated text.

---

#### # 55. Extensibility

The user must be able to add:

- New buildings
- New workers
- New worker types
- New work types
- New machinery
- New material types
- New material sizes
- New shops
- New suppliers
- New transport types
- New food categories

without changing the application code.

These should be database-driven master records.

---

#### # 56. Data entry philosophy

The application should prioritize:

### Speed

Few taps.

### Accuracy

Select existing records rather than repeatedly typing names.

### Flexibility

Optional fields where information isn't available.

#### ### Traceability

Related records should be linked.

#### ### Simplicity

No accounting terminology in the normal user interface.

---

### # 57. Critical implementation requirement

Before building the UI, the coding agent should create the data model and relationships for:

```
```text
People
Buildings
Work Types
Materials
Material Variants/Sizes
Shops
Suppliers
Machinery
Transport Types
Transactions
Labour
Food
Salary
Material Deliveries
Transport
```
```

Then build the UI around those relationships.

Do not build independent forms first and attempt to connect them later.

---

### # 58. Priority

#### ### P0 – Must have

- Mobile-first interface
- Searchable dropdowns
- Add/select existing people
- Buildings
- Labour
- Work types
- Cash provider
- Food
- Machinery
- Salary
- Transport
- Materials
- Material sizes
- Shop materials
- Date/date range
- Optional comments

- Linking transport to material
- Add-new functionality

### ### P1 – Important

- Building-level history
- Worker history
- Material history
- Supplier history
- Recent selections
- Filters
- Reports
- Cash balance

### ### P2 – Later

- OCR
- AI categorization
- Voice entry
- Offline mode
- Advanced analytics
- Tally export

---

### # 59. Final product goal

The finished application should feel less like accounting software and more like a **simple digital site register**.

A site manager should be able to pull out a phone and do this:

```
> Select **Labour** !' select **Balakrishna** !' select **Soil Dressing** !' select
North 05 !' enter 13,200 !' select **Cash** !' select **Varun Sir** !' Save.
```

Or:

```
> Select **Material** !' select **Sand** !' enter **10 tonnes** !' select **North
05** !' add **Truck 14,000** !' Save.
```

Or:

```
> Select **Food** !' select **Ramesh** !' select **Groceries** !' select date range
!' Save.
```

Or:

```
> Select **Salary** !' select **Watchman** !' select employee !' select August
period !' enter salary !' Save.
```

The application should make these common operations **faster than maintaining the current Excel workbook**, while retaining enough structured information to produce useful reports later.

## `Construction Site Management – Expanded Materials, Fuel, Weighbridge & Consumables Requirements.md`

```md

Construction Site Management – Expanded Materials, Fuel, Weighbridge & Consumables Requirements

1. New top-level modules

The application should organize site operations into these modules:

1. Labour
2. Food & Groceries
3. General Store
4. Machinery
5. Fuel
6. Salary
7. Transport
8. Materials
9. Weighbridge
10. Shops & Suppliers
11. Buildings / Locations
12. Cash & Payments

The modules must be connected rather than functioning as independent expense forms.

2. Important relationship model

The application should understand the following relationships:

```
```text
MATERIAL
%
% % % Supplier / Source
%
% % % Quantity / Weight
%
% % % Destination / Building
%
% % % Transport
%
% % % Weighbridge
%
% % % Material Cost
```
```

And:

```
```text
MACHINERY
%
% % % Operator
% % % Building / Job
% % % Work
% % % Hours
% % % FUEL
%
% % % Quantity
% % % Rate
% % % Cost
% % % Date
```
```

This is important because the application should eventually answer questions

such as:

> How much did the sand actually cost after transport and weighbridge?
and:

> How much diesel did the JCB consume this month?

3. FOOD & GROCERIES

Food should be its own module.

Do not put all food-related purchases into "Miscellaneous."

3.1 Food categories

Initial categories:

Milk & dairy

- Milk
- Curd
- Buttermilk
- Paneer
- Other dairy

Staples

- Rice
- Dal
- Wheat
- Flour
- Atta
- Oil
- Sugar
- Salt
- Spices
- Pulses

Vegetables

- Vegetables
- Leafy vegetables
- Onions
- Potatoes
- Tomatoes
- Other vegetables

Non-vegetarian

- Chicken
- Mutton
- Fish
- Eggs
- Other non-veg

Food

- Breakfast

- Lunch
- Dinner
- Meals
- Snacks
- Tea
- Coffee
- Biscuits
- Drinking water

Other

- Cooking gas
- Kitchen supplies
- Other food expenses

The user must be able to add categories.

4. Food recipient

Food should support linking the expense to the person or worker group.

Example:

```text  
Food

Person:  
[ Search worker ]

Worker Type:  
[ Labour %4 ]

Category:  
[ Milk %4 ]

Quantity:  
[ 10 ]

Unit:  
[ Litres %4 ]

Date:  
[ 30 Aug ]

Amount:  
[ 1600 ]

Comments:  
[ Optional ]  
```

Or:

```text  
Food

For:  
[ All Site Workers %4 ]

Category:  
[ Lunch %4 ]

Date:  
[ 30 Aug ]

Amount:  
[ 13,500 ]

The user should not have to enter each worker individually for common group food expenses.

---

#### # 5. FOOD DATE RANGE

Support:

```
```text
Single Day
Date Range
```
```

Example:

```
```text
From:
25 Aug 2026
```

```
To:
30 Aug 2026
```
```

This is particularly useful for food provided to a worker over several days.

---

#### # 6. GENERAL STORE MODULE

Create a separate module called:

```
> **General Store**
```

This is for everyday site consumables that are neither construction materials nor food.

Examples:

- Sponge
- Cleaning cloth
- Broom
- Mop
- Bucket
- Soap
- Detergent
- Floor cleaner
- Toilet cleaner
- Phenyl
- Brush
- Scrubber

- Garbage bags
- Dustbin
- Tissue
- Paper
- Tape
- Glue
- Chippo glue
- Fevicol
- Adhesives
- Rope
- Plastic sheets
- Batteries
- Torch
- Stationery
- Pens
- Markers
- Registers
- Files
- Printer supplies
- Drinking cups
- Disposable plates
- Other general consumables

The user can add new items.

---

#### # 7. General Store entry

Example:

```text

GENERAL STORE

Item

[Ø=Ý Search item]

Category

[Cleaning %¼]

Quantity

[5]

Unit

[Pieces %¼]

Shop

[Ø=Ý Search shop]

Cost

[^1350]

Date

[30 Aug]

Comments

[Optional]

SAVE

```

---

## # 8. General Store categories

Initial categories:

### ### Cleaning

- Sponge
- Scrubber
- Broom
- Mop
- Cleaning cloth
- Floor cleaner
- Phenyl
- Detergent
- Toilet cleaner
- Brush
- Garbage bags

### ### Adhesives

- Chippo glue
- Fevicol
- Silicone
- Adhesive
- Tape

### ### Office

- Paper
- Pens
- Markers
- Registers
- Files
- Stationery

### ### Safety

- Gloves
- Masks
- Safety tape
- Reflective items
- First-aid consumables

### ### Miscellaneous

User-defined items.

---

## # 9. MACHINERY

Machinery should have a master record.

Examples:

- JCB
- Tractor
- Crane
- Compressor

- Excavator
- Concrete mixer
- Generator
- Water tanker
- Truck
- Other

Each machine should have:

```
```text
Machine ID
Machine Type
Registration/Serial Number – optional
Operator
Owner – optional
Fuel Type
Meter Type
Notes
Active/Inactive
```
```

---

#### # 10. FUEL MUST BE LINKED TO MACHINERY

Fuel should **\*\*not\*\*** simply be recorded as:

```
```text
Fuel 15,000
```
```

The application should know:

```
```text
Fuel
  !"
Machine
  !"
Project / Building
```
```

Example:

```
```text
FUEL

Machine
[ JCB-01 %4 ]

Fuel
[ Diesel %4 ]

Quantity
[ 50 ]

Unit
[ Litres ]

Rate
[ 192 ]
```

Total
14,600

Date
[30 Aug]

Building / Job
[North 05 %4]

Given By
[Search person]

Comments
[Optional]

This allows fuel expenditure to be attributed to the machine and job.

Construction guidance specifically recommends recording fuel against equipment and checking equipment meter readings when fuel is issued.

11. Fuel types

Initial:

- Diesel
- Petrol
- Other

The system should allow additional fuel types.

12. Fuel quantity and cost

Fuel records should support:

```
```text
Quantity
Unit
Rate
Total Cost
```
```

If:

```
```text
50 litres x 192
```
```

the application should calculate:

```
```text
14,600
```
```

The user can override the calculated amount when necessary.

13. MACHINE METER READING

For machines with an hour meter, support:

```
```text
Previous Meter
Current Meter
Usage
```
```

Example:

```
```text
JCB-01
```

Previous:  
1,240 hours

Current:  
1,247 hours

Usage:  
7 hours  
```

This enables future reports such as:

> JCB used 42 hours and consumed 185 litres of diesel.

14. Fuel efficiency

The system should calculate, where enough data exists:

```
```text
Litres / machine hour
```
```

Example:

```
```text
JCB-01
```

Hours:  
42

Diesel:  
185 L

Average:  
4.40 L/hour  
```

Do not show efficiency if the necessary data is missing.

15. Fuel purchase vs fuel usage

Keep these separate.

Fuel purchase

```
```text
Bought 500 litres diesel
 146,000
```
```

Fuel issue/use

```
```text
50 litres
JCB-01
North 05
```
```

This distinction allows the system to track fuel stock later.

16. FUEL STOCK

The system should be designed to support a fuel tank/store.

Example:

```
```text
Fuel Stock
```

Opening:  
300 L

Purchases:  
500 L

Issued:  
350 L

Current:  
450 L  
```

This should be implemented if practical during the inventory phase.

Do not assume every fuel purchase was consumed immediately.

17. MATERIALS

Materials should be divided into:

Bulk materials

- Sand
- M-Sand
- Gravel
- Aggregate
- Stone
- Steel

- Bricks
- RMC
- Cement
- Pipes
- Other bulk materials

Hardware

- Screws
- Nails
- Bolts
- Nuts
- Washers
- Binding wire
- Anchors
- Fasteners

Electrical

- Wire
- Cable
- Switches
- Sockets
- MCB
- DB
- Conduit
- Electrical fittings
- LED lights
- Other electrical items

Plumbing

- PVC pipes
- CPVC pipes
- UPVC pipes
- Elbows
- Tees
- Couplers
- Valves
- Taps
- Fittings
- Other plumbing items

Finishing

- Paint
- Putty
- Primer
- Adhesives
- Tiles
- Tile adhesive
- Grout
- Other finishing materials

The exact material list must remain user-editable.

18. MATERIAL VARIANTS

Materials should support variants.

Example:

```
```text
Steel
% % % 6 mm
% % % 8 mm
% % % 10 mm
% % % 12 mm
% % % 16 mm
% % % 20 mm
% % % 25 mm
```
```

Bricks:

```
```text
Bricks
% % % 9 x 4 x 3
% % % 9 x 4 x 4
% % % Custom
```
```

Pipes:

```
```text
PVC Pipe
% % % 1 inch
% % % 2 inch
% % % 3 inch
% % % 4 inch
% % % Custom
```
```

19. MATERIAL UNITS

Each material should have a suitable unit.

Examples:

```
```text
Pieces
Boxes
Bags
Kg
Tonnes
Loads
Litres
Metres
Feet
Bundles
Rolls
Sets
```
```

Do not allow the user to mix units accidentally within the same material record.

For example, steel quantities should not sometimes mean "bags."

Material tracking systems benefit from maintaining a consistent item name and unit for every material.

20. BULK MATERIALS

Bulk materials should support:

```
```text
Material
Load Number
Quantity
Weight
Unit
Supplier
Source
Destination
Date
Transport
Weighbridge
Material Cost
Transport Cost
Weighbridge Cost
```
```

Cost can remain optional where the user does not know it.

21. WEIGHBRIDGE

Create a dedicated ****Weighbridge record****, but it must be linked to a material delivery.

Example:

```
```text
WEIGHBRIDGE

Material
[Sand]

Vehicle
[AP XX XXXX]

Supplier
[XYZ]

Date
[30 Aug]

Gross Weight
[24.80 tonnes]

Tare Weight
[14.20 tonnes]

Net Weight
[10.60 tonnes]
```
```

Weighbridge Fee
[1200]

Destination
[North Site]

[SAVE]

The application should calculate:

```
```text
Net Weight =
Gross Weight - Tare Weight
```
```

22. WEIGHBRIDGE LINK

The relationship should be:

```
```text
Material Delivery
 %
 % % % Vehicle
 % % % Supplier
 % % % Gross Weight
 % % % Tare Weight
 % % % Net Weight
 % % % Weighbridge Fee
```
```

The weighbridge record should never exist as an unrelated expense when it is associated with a material delivery.

23. Material cost calculation

For a bulk material, the system should be able to calculate:

```
```text
Material Cost
+
Transport Cost
+
Weighbridge Cost
=
Landed Cost
```
```

Example:

```
```text
Sand
```

Material:  
120,000

Transport:

14,000

Weighbridge:

1200

-----  
Landed Cost:

124,200  
```

This is one of the most important calculations in the application.

24. Cost per unit

Where sufficient information exists, calculate:

```text  
Landed Cost / Net Weight  
```

Example:

```text  
Net Weight:  
10.60 tonnes

Landed Cost:  
124,200

Cost:  
12,283 / tonne  
```

This should be shown as an analytical figure and not overwrite the original transaction values.

25. Material delivery workflow

The preferred workflow:

```text  
ADD MATERIAL DELIVERY  
!"  
Select Material  
!"  
Select Supplier  
!"  
Enter Weight / Quantity  
!"  
Select Destination  
!"  
Add Transport  
!"  
Add Weighbridge  
!"  
Enter Material Cost

!"

The user should be able to skip transport or weighbridge if not applicable.

— — —

## # 26. Combined delivery screen

For a bulk material:

[illegible]

— — —

## # 27. Vehicle master

## Create reusable vehicles.

Fields:

```
```text
Vehicle Number
Vehicle Type
Owner
Driver
Capacity – optional
Notes
```
```

Examples:

```
```text
AP XX 1234
Truck

TS XX 5678
Tractor
```
```

The user can search and select existing vehicles.

---

#### # 28. Weighbridge master

If the same weighbridge is used repeatedly, maintain:

```
```text
Weighbridge Name
Location
Rate/Fee – optional
Phone – optional
Notes
```
```

Example:

```
```text
XYZ Weighbridge
Warangal Road
```
```

The user can select it instead of typing the name every time.

---

#### # 29. Weighbridge fee

The fee should be independent from the material weight.

Example:

```
```text
Net weight:
10.6 tonnes

Weighbridge fee:
1200
```
```

The fee should contribute to the landed cost of the associated material delivery.



---

### # 30. Transport + Weighbridge + Material relationship

The application should be capable of displaying:

```
```text
SAND DELIVERY #104
```

```
Material
Sand
```

```
Net Weight
10.60 tonnes
```

```
Material Cost
 120,000
```

```
Transport
Truck
 14,000
```

```
Weighbridge
XYZ Weighbridge
 1200
```

```
-----
LANDED COST
 124,200
```
```

This should be visible from the material delivery details page.

---

### # 31. Material cost without known price

Sometimes the material arrives before the invoice/price is known.

Allow:

```
```text
Material Cost:
[ Not Known ]
```
```

The delivery can still be saved.

Later:

```
```text
Edit Delivery
!' Add Material Cost
```
```

The system should recalculate landed cost automatically.

---

### # 32. Transport cost without material cost

Also support the reverse situation.

Example:

```
```text
Sand
10 tonnes
Material Cost: Unknown
```

```
Transport:
  14,000
```

```
Weighbridge:
  1200
```
```

The system should show:

```
```text
Known cost:
  14,200
```

```
Material price:
Pending
```
```

Do not invent the missing material cost.

---

### # 33. Material source

Materials should have:

```
```text
Supplier
or
Source
```
```

because bulk materials may come from:

- Supplier
- Quarry
- Sand yard
- Factory
- Local shop
- Other site
- Own stock

---

### # 34. Material movement

The application should eventually support:

```
```text
Received
Issued
Transferred
```

Returned
Adjusted
\\

For example:

```text  
Sand  
Received: 100 tonnes  
Issued: 60 tonnes  
Remaining: 40 tonnes  
```

This should be designed into the data model even if full inventory management is implemented later.

35. Building allocation

Material deliveries can be allocated to:

```text  
North 01  
North 02  
West 01  
East 03  
Common Site  
Storage  
```

If material is delivered to general site storage, use:

> ****Site Store****

rather than forcing a building.

36. Material cost allocation

If a material is delivered for multiple buildings, support:

```text  
Sand delivery:  
10 tonnes  
  
Allocation:  
North 05 !' 6 tonnes  
West 02 !' 4 tonnes  
```

This should be an advanced feature.

For MVP, allow one destination.

37. General site expenses

The system should also have a controlled miscellaneous/general category.

Potential items:

- Electricity
- Water
- Internet
- Phone
- Repairs
- Maintenance
- Waste disposal
- Security
- Cleaning
- Site office expenses
- Printing
- Stationery
- Permits/fees
- Testing
- Surveying
- Professional services
- Emergency expenses

Construction cost classifications commonly separate direct labour, materials, equipment/fuel, transportation, site expenses, repairs/maintenance, utilities and consumables rather than placing everything into miscellaneous.

38. Recommended overall category hierarchy

The final application should use:

```text

SITE MANAGEMENT

%

% % % LABOUR

% % % % Work Records

% % % % Labour Payments

%

% % % FOOD & GROCERIES

% % % % Milk

% % % % Curd

% % % % Rice

% % % % Vegetables

% % % % Non-Veg

% % % % Meals

% % % % Other

%

% % % GENERAL STORE

% % % % Cleaning

% % % % Adhesives

% % % % Office

% % % % Safety

% % % % Other

%

% % % MACHINERY

% % % % JCB

% % % % Crane

% % % % Tractor

% % % % Compressor

% % % % Other

%

```

% % % FUEL
% % % % Diesel
% % % % Petrol
%
% % % SALARY
% % % % Watchman
% % % % Security
% % % % Office Boy
% % % % House Help
%
% % % TRANSPORT
% % % % Auto
% % % % Tractor
% % % % Truck
% % % % Lorry
% % % % Other
%
% % % MATERIALS
% % % % Sand
% % % % Steel
% % % % Bricks
% % % % Cement
% % % % Gravel
% % % % RMC
% % % % Pipes
% % % % Electrical
% % % % Plumbing
% % % % Hardware
% % % % Finishing
%
% % % WEIGHBRIDGE
%
% % % GENERAL SITE EXPENSES
` ``

```

---

# 39. Important: don't make categories rigid

These are **\*\*initial defaults\*\***, not hard-coded categories.

The user must be able to:

```

` ``text
+ Add Category
+ Add Item
+ Add Size
+ Add Worker Type
+ Add Work Type
+ Add Machine
+ Add Shop
+ Add Supplier
+ Add Vehicle
+ Add Building
` ``

```

---

# 40. Search UX

Every selector should behave like:

```
```text
[ Ø=Ý Search material ]
```

```
Sand
M-Sand
Steel
Bricks
Cement
Pipes
```

```
+ Add New Material
```
```

After selecting:

```
```text
Sand '
```
```

the selector closes.

Do not use giant dropdown lists that require scrolling through hundreds of items.

---

#### # 41. Smart defaults

The application should remember frequently used combinations.

For example:

```
```text
Sand
!' Truck
!' XYZ Supplier
!' North Site
!' XYZ Weighbridge
```
```

If the user selects Sand again, suggest:

> Use previous delivery settings?

This should be optional.

---

#### # 42. Cost summary

For every material delivery, show:

```
```text
Material Cost
Transport Cost
Weighbridge Cost
Other Delivery Costs
-----
Landed Cost
```
```

For every machine:

```
```text
Machine Cost
Fuel Cost
Maintenance Cost
-----
Operating Cost
```
```

This creates useful real-world cost information without requiring traditional accounting concepts.

---

#### # 43. Machine operating-cost view

For example:

```
```text
JCB-01

This Month

Usage:
84 hours

Fuel:
320 L

Fuel Cost:
 129,440

Machine Hire:
 150,000

Maintenance:
 14,500

Total:
 183,940
```
```

The application should be designed so this calculation is possible.

---

#### # 44. Material comparison

The system should eventually show:

```
```text
SAND

Delivery #1
10.6 tonnes
 12,283 / tonne landed

Delivery #2
11.2 tonnes
```

12,350 / tonne landed

Delivery #3

10.8 tonnes

12,210 / tonne landed

\\

This helps identify changes in material/transport costs.

45. Weighbridge reconciliation

For each bulk delivery:

```text

Supplier Quantity

vs

Weighbridge Quantity

\\

If a supplier says:

```text

11 tonnes

\\

but weighbridge says:

```text

10.6 tonnes

\\

the system should show:

```text

Difference:

0.4 tonnes

\\

Do not automatically assume which figure is correct.

46. Dashboard additions

The dashboard should eventually show:

```text

TODAY

Labour 1 \_\_\_\_\_

Food 1 \_\_\_\_\_

Materials 1 \_\_\_\_\_

Fuel 1 \_\_\_\_\_

Machinery 1 \_\_\_\_\_

Transport 1 \_\_\_\_\_

Salary 1 \_\_\_\_\_

General 1 \_\_\_\_\_

\\



And:

```
```text
MATERIALS THIS MONTH
```

```
Sand          120 tonnes
Steel         42 tonnes
Bricks        18,000 pcs
Cement        850 bags
```
```

And:

```
```text
MACHINERY
```

```
JCB
42 hours
185 L diesel
```

```
Tractor
28 hours
90 L diesel
```
```

---

# 47. Most important design rule

Do not model:

```
```text
Fuel = Expense
Transport = Expense
Weighbridge = Expense
Material = Expense
```
```

as four unrelated records.

Instead model the operational relationships:

```
```text
MATERIAL DELIVERY
%
% % % Material
% % % Supplier
% % % Quantity
% % % Weight
% % % Cost
%
% % % TRANSPORT
%   % % % Vehicle
%   % % % Type
%   % % % Cost
%
% % % WEIGHBRIDGE
% % % Gross
% % % Tare
% % % Net
% % % Fee
```

...

And:

```
```text
MACHINE
%
% % % Work
% % % Hours
% % % Building
% % % FUEL
 % % % Quantity
 % % % Rate
 % % % Cost
```
```

That data model is what will make the application genuinely useful instead of becoming another prettier Excel sheet.

48. Priority for implementation

P0

- Food categories
- General Store
- Machinery
- Fuel linked to machinery
- Material master
- Material sizes/variants
- Material deliveries
- Transport linked to material
- Weighbridge linked to material
- Material cost
- Transport cost
- Weighbridge cost
- Landed-cost calculation
- Searchable selectors
- Add-new functionality

P1

- Machine hour meter
- Fuel consumption
- Fuel stock
- Material stock
- Material allocation by building
- Weight reconciliation
- Supplier history
- Shop history
- Vehicle master

P2

- Cost comparison
- Machine efficiency analytics
- Material price trends
- Automated anomaly detection
- OCR of weighbridge slips
- OCR of material invoices

- Voice entry

49. Example end-to-end workflow

A truck brings sand to the site.

The user opens:

****Materials !' Add Delivery****

Selects:

```text  
Material:  
Sand

Supplier:  
ABC Sand Supplier

Vehicle:  
TS XX 1234

Destination:  
North 05  
```

Enters:

```text  
Gross:  
24.8 tonnes

Tare:  
14.2 tonnes  
```

System calculates:

```text  
Net:  
10.6 tonnes  
```

User adds:

```text  
Material cost:  
120,000  
```

Then:

```text  
Transport:  
Truck  
14,000  
```

Then:

```
```text
Weighbridge:
XYZ Weighbridge
 1200
```
```

System displays:

```
```text
SAND DELIVERY
```

Net Weight: 10.6 tonnes

Material	120,000
Transport	14,000
Weighbridge	1200
-----	
Landed Cost	124,200

```
 12,283 / tonne
```
```

This single record should contain all those relationships.

50. Final product principle

The application should answer three questions for every important resource:

What happened?

Example:

> 10.6 tonnes of sand arrived.

Where did it go?

Example:

> North 05.

What did it really cost?

Example:

> Material 120,000 + transport 14,000 + weighbridge 1200 = 124,200.

Likewise for machinery:

What machine?

> JCB-01

What did it do?

> Soil leveling – North 05 – 7 hours

What did it consume/cost?

> 32 litres diesel + machinery charge.

This relationship-first design should be reflected in the database before implementing the UI.

```
## `eslint.config.mjs`
```

```
```:mjs
import { defineConfig, globalIgnores } from "eslint/config";
import nextVitals from "eslint-config-next/core-web-vitals";
import nextTs from "eslint-config-next/typescript";
```

```
const eslintConfig = defineConfig([
 ...nextVitals,
 ...nextTs,
 // Override default ignores of eslint-config-next.
 globalIgnores([
 // Default ignores of eslint-config-next:
 ".next/**",
 "out/**",
 "build/**",
 "next-env.d.ts",
]),
]);
```

```
export default eslintConfig;
```

```
```
```

```
## `next-env.d.ts`
```

```
```:ts
/// <reference types="next" />
/// <reference types="next/image-types/global" />
import "../next/dev/types/routes.d.ts";
import "../next/dev/types/root-params.d.ts";

// NOTE: This file should not be edited
// see https://nextjs.org/docs/app/api-reference/config/typescript for more
information.
```

```
```
```

```
## `next.config.ts`
```

```
```:ts
import type { NextConfig } from "next";

const nextConfig: NextConfig = {
 /* config options here */
};

export default nextConfig;
```

```
```
```

```
## `package.json`
```

```
```:json
{
```

```

"name": "shambala",
"version": "0.1.0",
"private": true,
"scripts": {
 "dev": "next dev",
 "build": "next build",
 "start": "next start",
 "lint": "eslint"
},
"dependencies": {
 "@supabase/ssr": "^0.12.5",
 "@supabase/supabase-js": "^2.112.4",
 "date-fns": "^4.4.0",
 "next": "16.3.3",
 "react": "19.2.8",
 "react-dom": "19.2.8",
 "xlsx": "^0.18.5"
},
"devDependencies": {
 "@tailwindcss/postcss": "^4",
 "@types/node": "^20",
 "@types/react": "^19",
 "@types/react-dom": "^19",
 "eslint": "^9",
 "eslint-config-next": "16.3.3",
 "tailwindcss": "^4",
 "typescript": "^5"
}
}
...

`postcss.config.mjs`

```mjs
const config = {
  plugins: {
    "@tailwindcss/postcss": {},
  },
};

export default config;
...

## `readExcel.js`

```js
const xlsx = require('xlsx');
const path = require('path');

const filePath = path.resolve('../EXPENDITURE 2025.xlsx');
try {
 const workbook = xlsx.readFile(filePath);
 console.log("Sheet names:", workbook.SheetNames);

 // Dump a bit of data from each sheet
 for (const sheetName of workbook.SheetNames) {
 console.log(`\n--- Sheet: ${sheetName} ---`);
 const sheet = workbook.Sheets[sheetName];
 const data = xlsx.utils.sheet_to_json(sheet, { header: 1 });
 }
}

```

```

 console.log(JSON.stringify(data.slice(0, 5), null, 2));
 }
} catch (err) {
 console.error("Error reading file:", err);
}
}
```

## `README.md`

```md
Shambala – Construction Site Expense & Cash Manager

A mobile-first web application for recording and managing construction site expenses, labour payments, material purchases, cash flow, and supplier balances.

```

## ## Features

- **10-second expense entry** – Large amount input, quick category grid, searchable people picker
- **Cash balance tracking** – Real-time cash and bank balance calculation
- **Transaction history** – Card-based list with search and filters
- **Money management** – Record cash received, bank transfers
- **Supplier tracking** – Track purchases and payments, view outstanding balances
- **Monthly reports** – Category breakdown with progress bars
- **Excel import** – Parse and import from existing EXPENDITURE 2025.xlsx with review system
- **Excel export** – Download transactions as XLSX
- **Mobile-first UI** – Designed for 360-430px Android screens
- **PWA-ready** – Can be installed via "Add to Home Screen"

## ## Tech Stack

Layer	Technology
Framework	Next.js 15 (App Router)
Language	TypeScript
Styling	Tailwind CSS v4
Database	Supabase PostgreSQL
Auth	Supabase Auth
File Storage	Supabase Storage
Excel	SheetJS (xlsx)
Hosting	Vercel

## ## Setup

### ### 1. Create a Supabase Project

1. Go to [supabase.com](https://supabase.com) and create a free account
2. Click **New Project** and give it a name (e.g., "shambala")
3. Choose a region close to you and set a database password
4. Wait for the project to be created

### ### 2. Set Up the Database

1. In your Supabase dashboard, go to **SQL Editor**
2. Open and run `supabase/migrations/001\_schema.sql` – this creates all tables
3. Open and run `supabase/migrations/002\_seed.sql` – this adds default categories, accounts, and project

### ### 3. Get Your Supabase Credentials

1. In your Supabase dashboard, go to **\*\*Settings !' API\*\***
2. Copy the **\*\*Project URL\*\*** and **\*\*anon public key\*\***

### ### 4. Configure Environment Variables

```
```bash
cp .env.local.example .env.local
```
```

Edit ``.env.local``:

```
```
NEXT_PUBLIC_SUPABASE_URL=https://your-project.supabase.co
NEXT_PUBLIC_SUPABASE_ANON_KEY=your-anon-key
```
```

### ### 5. Install & Run

```
```bash
npm install
npm run dev
```
```

Open `[http://localhost:3000]`(`http://localhost:3000`) on your phone (or use Chrome DevTools mobile view).

### ### 6. Deploy to Vercel

1. Push your code to a GitHub repository
2. Go to `[vercel.com]`(`https://vercel.com`) and import the repo
3. Add the environment variables (``NEXT_PUBLIC_SUPABASE_URL``, ``NEXT_PUBLIC_SUPABASE_ANON_KEY``)
4. Deploy

Or use the Vercel CLI:

```
```bash
npx vercel
```
```

### ## Project Structure

```
```
src/
% % % app/
%   % % % page.tsx           # Dashboard / Home
%   % % % layout.tsx        # Root layout with bottom nav
%   % % % globals.css       # Design system
%   % % % expenses/
%     % % % page.tsx        # Transaction history
%     % % % add/page.tsx    # Add expense form
%   % % % money/
%     % % % page.tsx        # Account balances
%     % % % add/page.tsx    # Money In
%     % % % transfer/page.tsx # Transfer between accounts
%     % % % reports/page.tsx # Monthly + supplier reports
%     % % % import/page.tsx # Excel import + review
%   % % % more/
%     % % % page.tsx        # Settings menu
%     % % % categories/page.tsx # Manage categories
```
```



```

% % % % % people/page.tsx # Manage people/suppliers
% % % % % accounts/page.tsx # Manage accounts
% % % % % export/page.tsx # Export data
% % % % % api/
% % % % % import/route.ts # Import API
% % % % % export/route.ts # Export API
% % % actions/ # Server actions
% % % % transactions.ts
% % % % parties.ts
% % % % categories.ts
% % % % accounts.ts
% % % % reports.ts
% % % components/
% % % % BottomNav.tsx
% % % % TransactionCard.tsx
% % % % AddExpenseForm.tsx
% % % lib/
% % % % types.ts # TypeScript types
% % % % constants.ts # App constants
% % % % utils.ts # Formatting utilities
% % % % excel-parser.ts # Excel import parser
% % % % supabase/
% % % % client.ts # Browser Supabase client
% % % % server.ts # Server Supabase client
...

```

## ## Database Schema

See `supabase/migrations/001\_schema.sql` for the full schema. Key tables:

- **projects** – Site/project (multi-project ready)
- **accounts** – Cash, Bank, UPI accounts
- **categories** – Hierarchical expense categories
- **parties** – Workers, suppliers, contractors
- **transactions** – All financial transactions (expense, income, transfer, salary, etc.)
- **transaction\_items** – Material quantity/rate details
- **imports** / **import\_rows** – Excel import tracking

## ## Importing Historical Data

1. Go to **More !** Import Excel
2. Upload `EXPENDITURE 2025.xlsx`
3. The parser will analyze all sheets and classify transactions
4. Review the results: auto-classified, needs review, rejected
5. Approve/reject individual rows
6. Click "Import" to save to database

The parser handles:

- Daily expenditure logs (SNO/EXPENDITURE/CREDIT/DEBIT format)
- Multi-line descriptions (merges continuation rows)
- Cash input records (VARUN SIR CASH, K.RAJESH SIR CASH)
- Salary records (name, designation, amount, bank/cash)
- Payment records (per-person payment history)
- Material delivery registers (date, quantity, unit)
- Carry-forward detection (excluded from import)
- Cross-sheet duplicate detection

...

## ## `replace\_dates.js`



|    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| Pu | •d | đg | Pn | Pr | t  | P_ | 4  | €) | À\ | à  | p  | o  | e  | 0t | đi | @  | PU | •D | N  | đT | N  | PL | À  | E  | ˆE | E  | àC | PS | p  |    |    |
| @E | ÀE | @E | C  | S  | 0A | @E | À\ | à  | d  | •r | Pc | @i | đn | T  | PX | @, | Àn | •D | àu | đb | Pr | T  | PX | @, | Àn | E  | àa | Đe | T  |    |    |
| P. | p. | \  | V  | •l | Àa | 0  | P\ | \  | à  | d  | •s | l  | y  | đn | m  | P  | @E | €T | G  | PN | PR | T  | PD | A  | ÀW | Y  | 0  | S  |    |    |    |
| €C | đA | ÀE | 0C | P( | @i | e  | 0t | •o | à  | À  | '  | '  | À  | àu | đb | Pr | À  | àa | Đe | •) | S  | @0 | E  | @, | Àn | e  | 0r | Pa |    |    |    |
| @I | ĐE | 0T | M  | T  | đA | @  | @  | àu | ÀL | D  | PF | U  | ÀT | N  | đW | €) | Àn | •; | Àn | Àn | Đ- | G  | Pn | Pr | l  | S  | @o | e  | C  |    |    |
| Đa | 0o | à, | w  | t  | 0h | đA | @  | Àn | 0R | PA | @E | T  | B  | ÀE | w  | đr | °e | @y | e  | 0  | €\ | à  | i  | @  | ˆE | E  | àC | PS | i  |    |    |
| @E | ÀE | @E | C  | S  | 0A | @E | À\ | à  | n  | m  | P  | @E | €T | N  | đT | N  | PL | À, | Àn | 0r | Pa | @e | @_ | t  | T  | •M | PS | @A |    |    |    |
| @E | ˆA | PL | @  | à0 | p( | •\ | à) | °\ | à\ | à- | Đ  | po | k  | T  | •p | Ps | (  | P. | p. | S  | đi | À  | @r | Ps | 0i | àg | À  | 0l | Pa |    |    |
| @A | L  | P  | po | k  | đt | •p | Ps | (  | Àn | •d | U  | PI | @  | R  | •M | R  | •  | °E | •  | @E | ˆA | PL | @  | Pu | •d | đg | Pn | Pr |    |    |    |
| r  | đj | Pc | @_ | •d | U  | PI | @  | à0 | @  | àU | ÀL | R  | PF | PR | PN | 0E | 0  | r  | đj | Pc | @s | €i | @) | 0  | à  | @E | ÀE | @E | C  |    |    |
| @E | €T | N  | đT | N  | PL | À, | Àn | S  | 0r | Pa | @e | @_ | t  | T  | •M | PS | @A | ĐP | @Z | N  | đT | N  | PL | À  | @E | ˆA | PL | @  | à0 |    |    |
| a  | t  | e  | 0  | €P | Po | l  | P, | S  | €o | s  | A  | 0u | pr | p) | °\ | à\ | àC | E  | T  | P  | @A | L  | P  | a  | t  | •e | 0l | s  | 0  |    |    |
| ˆE | •  | @E | ˆA | PL | @  | Pu | •d | đg | Pn | Pr | t  | P_ | 4  | €) | À\ | à  | p  | o  | e  | 0t | đi | @  | PU | •D | N  | đT | N  | PL | À  |    |    |
| r  | đj | Pc | @s | €i | @) | 0  | à  | @E | ÀE | @E | C  | S  | 0A | @E | À\ | à  | c  | Àa | 0s | p  | r  | @y | đc | Àa | 0s | N  | đT | N  | PL | À  |    |
| à0 | @  | àu | ÀL | À\ | à  | w  | đr | °e | _  | @y | e  | đi | @  | PU | •D | R  | PF | PR | PN | 0E | 0  | po | k  | Pr | đt | •p | Ps | €i | @) |    |    |
| àu | ÀL | D  | PF | U  | ÀT | T  | U  | P, | Àn | 0r | Pa | @e | @_ | t  | T  | •M | PS | @A | ĐP | @Z | N  | đT | N  | PL | À  | @E | ˆA | PL | @  |    |    |
| Đa | 0h | •n | Pr | •  | Đa | 0t | Pr | Àn | 0R | PA | @E | T  | B  | ÀE | m  | c  | €i | àe | y  | (  | Àn | •d | U  | PI | @  | R  | •M | R  | p  |    |    |
| Pu | •d | đg | Pn | Pr | t  | P_ | 4  | €) | À\ | à  | m  | c  | €i | àe | đi | @  | @E | €T | N  | đT | N  | PL | À, | -  | Đ  | P. | p. | \  | J  | 0B | Đ0 |
| @E | ÀE | @E | C  | S  | 0A | @E | À\ | à  | m  | c  | €i | àe | đi | @  | @E | €T | N  | đT | N  | PL | À, | -  | Đ  | P. | p. | \  | J  | 0B | Đ0 |    |    |
| Đa | 0h | •n | P_ | @y | e  | T  | PX | @  | à0 | @  | àu | ÀL | À\ | à  | r  | Pg | •s | @r | t  | •o | à_ | àu | đb | Pr | T  | PX | @, | Àn |    |    |    |
| E  | ˆE | E  | àC | PS | p  | r  | @i | Ps | €i | @) | 0  | à  | @E | ÀE | @E | S  | PT | N  | PL | À, | Àn | ˆu | PL | đt | •p | P  | @E | €T | N  |    |    |
| @E | €T | À\ | à  | i  | 0_ | c  | @i | ˆe | B  | đ0 | ÀE | N  | N  | đT | N  | PL | À  | @E | ˆA | PL | @  | @R | PE | À\ | à  | c  | e  | t  | Pd |    |    |
| à0 | @  | àu | ÀL | D  |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |

|    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |     |    |    |    |    |   |
|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|-----|----|----|----|----|---|
| @r | n  | 0a | 0t | •o | à_ | @y | e  | A  | 0  | PN | PM | (  | Àn | pm | ðn | Py | ði | à' | À\ | à  | '  | @r | n  | 0f | Pr | p,  | Àn |    |    |    |   |
| po | e  | a  | @i | ðñ | l  | ðe | €p | Pn | 0e | p, | Àn | (  | pg | Pn | Pr | l  | ðe | €p | Pn | 0e | p, | Àn | ps | Pp | l  | •e  | À_ | a  |    |    |   |
| po | e  | ài | àg | ðb | l  | n  | 0e | p\ | à) | \  | à\ | àc | E  | T  | P  | @Y | E  | p  | y  | ðe | àt | ðm | Pt | €o | @  | S   | E  | àU | ð  |    |   |
| pb | n  | _  | @r | n  | 0f | Pr | p, | '  | Pp | •' | À  | po | @h | Pr | p) | \  | à\ | àc | E  | T  | P  | @A | L  | P  | @r | n   | 0a | 0t | •o |    |   |
| R  | •M | R  | •  | °E | •  | @E | •A | PL | @  | Pu | •d | ðg | Pn | Pr | t  | P_ | `4 | €) | À\ | à  | p  | o  | e  | 0t | ði | @   | PU | •D | N  |    |   |
| E  | •E | E  | àC | PS | p  | o  | e  | 0t | 0( | •d | •  | ðN | D  | PL | PT | P  | 0A | 0C | D  | P, | Àn | @y | e  | t  | a  | às  | c  | @i |    |    |   |
| \  | d  | t  | P  | @A | @E | N  | ðT | N  | PL | À  | @E | •A | PL | @  | 0U | R  | PN | @_ | @A | @E | À\ | à  | a  | ðo | Pn | @   | àU | ðE |    |    |   |
| \  | à  | a  | 0c | ðu | àt | ði | @  | PU | •D | R  | PF | PR | PN | 0E | 0  | c  | 0o | Pn | @s | €i | @) | 0  | à  | @E | ÀE | @E  | S  | PT | N  | P  |   |
| @o | ða | 0c | ðu | àt | ði | @  | PU | •D | R  | PF | PR | PN | 0E | 0  | a  | t  | •e | 0( | •d | •  | ðN | D  | PL | PT | P  | 0E  | @  | àU | ÀL | À  | ð |
| @e | 0c | i  | t  | •o | à  | @E | €T | À\ | à  | p  | y  | ðe | àt | ðm | Pt | €o | @  | a  | •m | Pn | @_ | ðe | @h | ðd | N  | ðT  | N  | PL | À  | ð  |   |
| r  | Pf | Pr | Pn | 0e | ðt | b  | Àe | T  | PX | @, | -  | ð  | pl | b  | ðu | _  | e  | 0o | d  | 0' | À  | pf | ðo | @_ | e  | 0o  | d  | 0' | À  | ð  |   |
| e  | •e | e  | àc | P_ | •d | U  | PI | @, | ð- | l  | •n | °i | àg | b  | c  | °  | @o | o  | e  | a  | @i | ðñ | l  | r  | Pc | ðr  | @\ |    |    |    |   |
| @I | ðE | 0T | M  | T  | àO | @  | àU | ÀL | D  | PF | U  | ÀT | N  | ðW | €) | Àn | •; | Àn | Àn | ð- | ð  | ð  | ð  | ð  | ð  | ð   | ð  | ð  | ð  | ð  |   |
| ð= | ð= | ð= | ð= | ð= | ð= | ð= | ð= | ð= | ð= | ð= | ð= | ð= | ð= | ð= | ð= | ð= | ð= | ð= | ð= | ð= | ð= | ð= | ð= | ð= | ð= | ð=  | ð= | ð= | ð= | ð= |   |
| ðP | PR | T  | •O | àA | À  | E  | 0O | D  | 0\ | à- | ð  |    |    |    |    |    |    |    |    |    |    |    |    |    |    |     |    |    |    |    |   |
| ð= | ð= | ð= | ð= | ð= | ð= | ð= | ð= | ð= | ð= | ð= | ð= | ð= | ð= | ð= | ð= | ð= | ð= | ð= | ð= | ð= | ð= | ð= | ð= | ð= | ð= | ð=  | ð= | ð= | ð= | ð= |   |
| e  | 0o | d  | 0\ | àC | E  | T  | P  | @A | L  | P  | Àa | o  | Pr | ðr | Pc | ðr | @s | (  | Àn | •d | U  | PI | @  | R  | •M | R   | •  | °E |    |    |   |
| Pu | •d | ðg | Pn | Pr | t  | P_ | `4 | €) | À\ | à  | p  | o  | e  | 0t | ði | @  | PU | •D | N  | ðT | N  | PL | À  | E  | •E | E   | àC | PS | t  | •e |   |
| @E | ÀE | @E | C  | S  | 0A | @E | À\ | à  | w  | ðr | °E | •d | U  | PI | @  | àO | @  | àU | ÀL | R  | PF | PR | PN | 0E | 0  | a   | t  | •e |    |    |   |
| po | k  | Pr | ðt | •p | P_ | •d | U  | PI | @  | E  | •E | •c | ÀC | PS | w  | ðr | °e | _  | @y | e  | 0( | •d | •, | Àn | po | k   | ðt | •p | u  |    |   |
| E  | •E | E  | àC | PS | w  | ðr | _  | @y | e  | 0( | •d | •, | Àn | ÀC | PS | u  | •l | @i | àg | ði | @  | PU | •D | R  | PF | PR  | PN | 0E | 0  |    |   |
| @a | @e | D  | T  | P  | àO | @  | àU | ÀL | À\ | à  | t  | •m | P  | @I | ðE | À\ | à  | a  | ðo | Pn | @  | àU | ðE | I  | 0( | 5</ |    |    |    |    |   |

...

```
```js
```

```
const supabase = createClient(SUPABASE_URL, SUPABASE_KEY);
const PROJECT_ID = '10000000-0000-0000-0000-000000000000';
```

```
async function seed() {
  const filePath = path.resolve(__dirname, '../..\\EXPENDITURE 2025.xlsx');
  const workbook = xlsx.readFile(filePath);

  const names = new Set();
```

```

// Extract from PAY MENTS
if (workbook.Sheets['PAY MENTS']) {
  const sheet = workbook.Sheets['PAY MENTS'];
  const data = xlsx.utils.sheet_to_json(sheet, { header: 1 });
  if (data[3]) {
    data[3].forEach(cell => {
      if (cell && typeof cell === 'string') {
        let name = cell.replace(/PG NO\s*\d+/i, '').replace(/PG NP\s*\d+/i,
        '').trim();
        if (name && name !== 'TOTAL CASH') {
          names.add(name);
        }
      }
    });
  }
}

// Extract from IN PUT
if (workbook.Sheets['IN PUT']) {
  const sheet = workbook.Sheets['IN PUT'];
  const data = xlsx.utils.sheet_to_json(sheet, { header: 1 });
  if (data[2]) {
    data[2].forEach(cell => {
      if (cell && typeof cell === 'string') {
        let name = cell.replace(/CASH/i, '').trim();
        if (name && name !== 'SNO' && name !== 'DATE' && name !== 'CASH
AMOUNT') {
          names.add(name);
        }
      }
    });
  }
}

const parties = Array.from(names).map(name => ({
  project_id: PROJECT_ID,
  name: name,
  class: 'person', // default to person
  is_active: true
})));

console.log(`Found ${parties.length} unique parties to insert:`);
console.log(parties.map(p => p.name).join(', '));

// Insert into Supabase
const { data, error } = await supabase
  .from('parties')
  .insert(parties)
  .select();

if (error) {
  console.error('Error inserting parties:', error);
} else {
  console.log(`Successfully inserted ${data.length} parties.`);
}

seed();
...

```

```
```json
```

json	Đ-	S	€	a	Đb	l	:	I	ài	@i	l	M	s	@e	@a	@a	S	Pe	@	€E	P	M	đd	Pl	·\	à\	à-	Đ		
•n	@o	t	€e	d	Pf	u	Àt	p	o	e	ot	w	•t	€	Op	Pc	•f	•c	U	PI	@	Oo	i	@	Đa	@c	€e	0		
0o	às	@a	àt	0\	ài	às	PR	@	•N	@O	p	o	e	ot	0	€i	@,	n	m	P,	d	Ps	Or	•p	@i	đn	•\	àV	L	
€'	0	0	0	0	Đ0	0	-	0	0	Đ0	0	-	0	0	0	0	0	0	p,	'	àa	t	€	'i	Àl	s	p,	'	r	
0o	às	@r	Pc	@i	đn	p	o	e	ot	p)	Àn	đN	C	đN	\L	•C	@	€i	@)	D	đ	àO	@H	•N	p;	Àn	Àn	Đ-	I	
c	0o	Pn	@s	Àn	•N	0E	T	I	àT	đ	c	0o	Pn	@s	(	•d	À	r	đj	Pc	@_	•d	À	àa	Đe	À	@y	e	0	
•s	đđ	Pf	u	Àt	•\	àV	L	PE	0	Àn	€'	0	0	0	0	0	0	-	0	0	Đ0	0	-	0	0	0	0	0	0	
p1	0	0	0	-	0	0	Đ0	0	-	0	0	Đ0	0	0	0	0	0	'	À	pC	s	€	đn	H	n	@'	À	pc	s	
€'	0	0	0	0	Đ0	0	-	0	0	Đ0	0	-	0	0	0	0	0	0	1	p,										
p1	0	0	0	-	0	0	Đ0	0	-	0	0	Đ0	0	0	0	0	0	'	À	pI	@B	•	a	àk	p,	'	a	àk	p,	
00	àF	Ài	0T	(	•d	•	@O	N	đT	€I	àG	°\	à\	à-	Đ	u	•l	@i	àg	0	đ	Ào	0a	@i	đn	0\	ài	às	PR	
€p	o	e	0t	đi	@,	d	•r	Pc	@i	đn	À	àu	Đb	Pr	À	àa	Đe	•\	àV	L	PE	0	Àn							
€'	0	0	0	0	Đ0	0	-	0	0	Đ0	0	-	0	0	0	0	0	0	p,	'	ào	t	€'	À	p0	'	À	àu	ÀL	
€'	0	0	0	0	Đ0	0	-	0	0	Đ0	0	-	0	0	0	0	0	0	0	p,	'	ào	t	€'	À	p0	'	À	àu	ÀL
€'	0	0	0	0	Đ0	0	-	0	0	Đ0	0	-	0	0	0	0	0	0	0	p,	'	ào	t	€'	À	p0	0'	À	àu	ÀL
€'	0	0	0	0	Đ0	0	-	0	0	Đ0	0	-	0	0	0	0	0	0	0	p,	'	ào	t	€'	À	p0	P'	À	àu	ÀL
€'	0	0	0	0	Đ0	0	-	0	0	Đ0	0	-	0	0	0	0	0	0	0	p,	'	pe	0t	p,	'	1	p,	N	PL	À)
€'	0	0	0	0	Đ0	0	-	0	0	Đ0	0	-	0	0	0	0	0	0	0	p,	'	pe	0t	p,	'	2	p,	N	PL	À)
€'	0	0	0	0	Đ0	0	-	0	0	Đ0	0	-	0	0	0	0	0	0	0	p,	'	Pa	0t	p,	'	1	p,	N	PL	À)
€'	0	0	0	0	Đ0	0	-	0	0	Đ0	0	-	0	0	0	0	0	0	0	p,	'	Pa	0t	p,	'	2	p,	N	PL	À)
€'	0	0	0	0	Đ0	0	-	0	0	Đ0	0	-	0	0	0	0	0	0	0	p,	'	Pa	0t	p,	'	3	p,	N	PL	À)
€'	0	0	0	0	Đ0	0	-	0	0	Đ0	0	-	0	0	0	0	0	0	0	p,	'	Pa	0t	đn	'	0i	@e	p,	'	
\	à\	à-	Đ	po	k	Pr	T	•p	Ps	À																				

€' 0 0 0 0 Đ0 0 - 0 0 Đ0 0 - 0 0 0 0 0 0 p, ' l s @e i àg p) À\ à  
€' 0 0 0 0 Đ0 0 - 0 0 Đ0 0 - 0 0 0 0 0 0 0 p, ' a .n @i àg p) À\ à  
€' 0 0 0 0 Đ0 0 - 0 0 Đ0 0 - 0 0 0 0 0 0 0 p, ' Pl Pc @r .c l w òr °', An  
€' 0 0 0 0 Đ0 0 - 0 0 Đ0 0 - 0 0 0 0 0 0 0 p, ' l Pm i àg l w òr °', An  
€' 0 0 0 0 Đ0 0 - 0 0 Đ0 0 - 0 0 0 0 0 0 0 p, ' Px 0a `a @i ðn p) À\ à  
€' 0 0 0 0 Đ0 0 - 0 0 Đ0 0 - 0 0 0 0 0 0 0 p, ' Ào d .n p' ., An  
€' 0 0 0 0 Đ0 0 - 0 0 Đ0 0 - 0 0 0 0 0 0 0 p, ' Pn Ào d .n p' ., An  
€' 0 0 0 0 Đ0 0 - 0 0 Đ0 0 - 0 0 0 0 0 0 0 p, ' @r n 0p ðr @a @i ðn p) \  
0a @e po i Ps Àn .N 0E T I àT ð `o ðd ðc t Pg ðr .e 0 €i @, p o e 0t ði @, n m  
a e àt ðc t Pg ðr .) Àn `A ÀU PS Àn €' 00 0 0 0 Đ0 0 - 0 0 0 0 0 0 0 p, ' À pM .l °' À pM .l °' @a .r  
p1 0 0 0 - 0 0 Đ0 0 - 0 0 Đ0 0 0 0 0 0 0 0 2 p, ' À pM .l °' À pM .l °' @a .r  
p1 0 0 0 - 0 0 Đ0 0 - 0 0 Đ0 0 0 0 0 0 0 0 2 p, ' À pC Pr @' À pM .l °' @a .r  
€' 00 0 0 0 Đ0 0 - 0 0 Đ0 0 - 0 0 0 0 0 0 0 3 p, ' À pR .c P' À pS @a l Ps p) À\  
p1 0 0 0 - 0 0 Đ0 0 - 0 0 Đ0 0 0 0 0 0 0 0 4 p, ' À pR .c P' À pS @a l Ps p) À\  
€' 00 0 0 0 Đ0 0 - 0 0 Đ0 0 - 0 0 0 0 0 0 0 4 p, ' À pD l p, ' 0t p Àe 0' ., An  
p1 0 0 0 - 0 0 Đ0 0 - 0 0 Đ0 0 0 0 0 0 0 0 5 p, ' À pD l p, ' 0t p Àe 0' ., An  
€' 00 0 0 0 Đ0 0 - 0 0 Đ0 0 - 0 0 0 0 0 0 0 5 p, ' À pV Pg Pt b Àe 0' À pV Pg Pt  
p1 0 0 0 - 0 0 Đ0 0 - 0 0 Đ0 0 0 0 0 0 0 0 6 p, ' À pV Pg Pt b Àe 0' À pV Pg Pt  
€' 00 0 0 0 Đ0 0 - 0 0 Đ0 0 - 0 0 0 0 0 0 0 6 p, ' À pC €i 0k Pn p, ' ào à- `e pe  
p1 0 0 0 - 0 0 Đ0 0 - 0 0 Đ0 0 0 0 0 0 0 0 7 p, ' À pC €i 0k Pn p, ' ào à- `e pe  
€' 00 0 0 0 Đ0 0 - 0 0 Đ0 0 - 0 0 0 0 0 0 0 7 p, ' À pE pg 0' À pN ðn ðv Pg Pt r  
p1 0 0 0 - 0 0 Đ0 0 - 0 0 Đ0 0 0 0 0 0 0 0 8 p, ' À pE pg 0' À pN ðn ðv Pg Pt r  
€' 00 0 0 0 Đ0 0 - 0 0 Đ0 0 - 0 0 0 0 0 0 0 8 p, ' À pM Pa Às p, ' `o ðd p) À\ à  
p1 0 0 0 - 0 0 Đ0 0 - 0 0 Đ0 0 0 0 0 0 0 0 9 p, ' À pM Pa Às p, ' `o ðd p) À\ à  
€' 00 0 0 0 Đ0 0 - 0 0 Đ0 0 - 0 0 0 0 0 0 0 9 p, ' À pS àa 0k 0' À pF ðo @' ., An  
p1 0 0 0 - 0 0 Đ0 0 - 0 0 Đ0 0 0 0 0 0 0 0 0 p, ' À pS àa 0k 0' À pF ðo @' ., An  
€' 00 0 0 0 Đ0 0 - 0 0 Đ0 0 - 0 0 0 0 0 0 0 0 p, ' À pD i àk .n p pa @e ' À pF  
p1 0 0 0 - 0 0 Đ0 0 - 0 0 Đ0 0 0 0 0 0 0 0 1 p, ' À pD i àk .n p pa @e ' À pF  
€' 00 0 0 0 Đ0 0 - 0 0 Đ0 0 - 0 0 0 0 0 0 0 1 p, ' À pG o 0e i Ps p, ' ðt €e ' À pG  
p1 0 0 0 - 0 0 Đ0 0 - 0 0 Đ0 0 0 0 0 0 0 0 1 p, ' À pG o 0e i Ps p, ' ðt €e ' À pG  
Đa @e i l 0 Da 0t Pr ( u Àk & S Da Àl .\ à- Đ u Àk M t Pr .a Às An .N 0E T I  
€i @, p o e 0t ði @, c Àa 0s À àa De À @e `a Pl @\_ Pn .t .\ àV L PE 0\ à  
€' @0 0 0 0 Đ0 0 - 0 0 Đ0 0 - 0 0 0 0 0 0 0 1 p, ' À pb PL °' À pS n @' À pT ðn  
p1 0 0 0 - 0 0 Đ0 0 - 0 0 Đ0 0 0 0 0 0 0 0 2 p, ' À pb PL °' À pS n @' À pT ðn  
€' @0 0 0 0 Đ0 0 - 0 0 Đ0 0 - 0 0 0 0 0 0 0 2 p, ' À pb PL °' À pM ĐS n @' À pT  
p1 0 0 0 - 0 0 Đ0 0 - 0 0 Đ0 0 0 0 0 0 0 0 3 p, ' À pb PL °' À pM ĐS n @' À pT  
€' @0 0 0 0 Đ0 0 - 0 0 Đ0 0 - 0 0 0 0 0 0 0 3 p, ' À pb PL °' À pG a `e À' À pT  
p1 0 0 0 - 0 0 Đ0 0 - 0 0 Đ0 0 0 0 0 0 0 0 4 p, ' À pb PL °' À pG a `e À' À pT  
€' @0 0 0 0 Đ0 0 - 0 0 Đ0 0 - 0 0 0 0 0 0 0 4 p, ' À pb PL °' À pS @e Pl p, ' @o  
p1 0 0 0 - 0 0 Đ0 0 - 0 0 Đ0 0 0 0 0 0 0 0 5 p, ' À pb PL °' À pS @e Pl p, ' @o  
€' @0 0 0 0 Đ0 0 - 0 0 Đ0 0 - 0 0 0 0 0 0 0 5 p, ' À pb PL °' À pB i 0k 0' À pP  
p1 0 0 0 - 0 0 Đ0 0 - 0 0 Đ0 0 0 0 0 0 0 0 6 p, ' À pb PL °' À pB i 0k 0' À pP  
€' @0 0 0 0 Đ0 0 - 0 0 Đ0 0 - 0 0 0 0 0 0 0 6 p, ' À pb PL °' À pC Pm Pn @' À pB  
p1 0 0 0 - 0 0 Đ0 0 - 0 0 Đ0 0 0 0 0 0 0 0 7 p, ' À pb PL °' À pC Pm Pn @' À pB  
€' @0 0 0 0 Đ0 0 - 0 0 Đ0 0 - 0 0 0 0 0 0 0 7 p, ' À pb PL °' À pR ĐC p, ' 0u i  
p1 0 0 0 - 0 0 Đ0 0 - 0 0 Đ0 0 0 0 0 0 0 0 8 p, ' À pb PL °' À pR ĐC p, ' 0u i  
€' @0 0 0 0 Đ0 0 - 0 0 Đ0 0 - 0 0 0 0 0 0 0 8 p, ' À pp Àu Đb .n p' À pP `C P .p  
p1 0 0 0 - 0 0 Đ0 0 - 0 0 Đ0 0 0 0 0 0 0 0 9 p, ' À pp Àu Đb .n p' À pP `C P .p  
€' @0 0 0 0 Đ0 0 - 0 0 Đ0 0 - 0 0 0 0 0 0 0 9 p, ' À pe Àe 0t i 0a À' À pW .r P'  
p1 0 0 0 - 0 0 Đ0 0 - 0 0 Đ0 0 0 0 0 0 0 0 0 p, ' À pe Àe 0t i 0a À' À pW .r P'  
€' @0 0 0 0 Đ0 0 - 0 0 Đ0 0 - 0 0 0 0 0 0 0 0 p, ' À ph r @w r P' À pS 0r Pw 0' À  
Đa @e i l V r .a àt 0\ à- Đ 0t Pe À\ àI àS PR @ .N @0 ph m t Pr .a À\_ `a pS 0r Pw 0' À  
àa De . `A ÀU PS Àn €' @0 0 0 0 Đ0 0 - 0 0 0 0 0 0 0 0 4 p, ' À m t Pr .a À\_ `a pS 0r Pw 0' À  
€' @0 0 0 0 Đ0 0 - 0 0 Đ0 0 - 0 0 0 0 0 0 0 4 p, ' À € Đm p) À\ à  
€' @0 0 0 0 Đ0 0 - 0 0 Đ0 0 - 0 0 0 0 0 0 0 4 p, ' À 0 m Đ' ., An  
€' @0 0 0 0 Đ0 0 - 0 0 Đ0 0 - 0 0 0 0 0 0 0 4 p, ' À 2 m Đ' ., An  
€' @0 0 0 0 Đ0 0 - 0 0 Đ0 0 - 0 0 0 0 0 0 0 4 p, ' À 6 m Đ' ., An  
€' @0 0 0 0 Đ0 0 - 0 0 Đ0 0 - 0 0 0 0 0 0 0 4 p, ' À 0 m Đ' ., An  
€' @0 0 0 0 Đ0 0 - 0 0 Đ0 0 - 0 0 0 0 0 0 0 4 p, ' À 5 m Đ' .; An Àn Đ- B i





```

export async function getAccountBalances(): Promise<AccountBalance[]> {
 const supabase = await createClient();

 // Get all accounts
 const { data: accounts, error: accError } = await supabase
 .from('accounts')
 .select('*')
 .eq('project_id', DEFAULT_PROJECT_ID);

 if (accError) throw new Error(accError.message);

 // Get all transactions
 const { data: transactions, error: txnError } = await supabase
 .from('transactions')
 .select('type, amount, account_id, to_account_id')
 .eq('project_id', DEFAULT_PROJECT_ID);

 if (txnError) throw new Error(txnError.message);

 // Calculate balance for each account
 const balances: AccountBalance[] = (accounts || []).map((account: Account)
=> {
 let balance = account.initial_balance;

 for (const txn of transactions || []) {
 // Money coming IN to this account
 if (txn.account_id === account.id) {
 if (txn.type === 'money_in' || txn.type === 'opening_balance') {
 balance += txn.amount;
 } else if (
 txn.type === 'operational_expense' ||
 txn.type === 'general_expense' ||
 txn.type === 'supplier_payment'
) {
 balance -= txn.amount;
 }
 }

 // Transfers
 if (txn.type === 'transfer') {
 if (txn.account_id === account.id) {
 balance -= txn.amount; // transferring FROM this account
 }
 if (txn.to_account_id === account.id) {
 balance += txn.amount; // transferring TO this account
 }
 }
 }

 return {
 account_id: account.id,
 account_name: account.name,
 account_type: account.type,
 balance,
 };
 });

 return balances;
}

export async function getThisMonthSpent(): Promise<number> {

```

```

const supabase = await createClient();

const now = new Date();
const year = now.getFullYear();
const month = now.getMonth() + 1;

const firstDay = `${year}-${String(month).padStart(2, '0')}-01`;
const lastDate = new Date(year, month, 0).getDate();
const lastDay = `${year}-${String(month).padStart(2, '0')}-${
String(lastDate).padStart(2, '0')}`;

const { data, error } = await supabase
 .from('transactions')
 .select('amount')
 .eq('project_id', DEFAULT_PROJECT_ID)
 .in('type', ['operational_expense', 'general_expense',
'supplier_payment'])
 .gte('date', firstDay)
 .lte('date', lastDay);

if (error) throw new Error(error.message);
return (data || []).reduce((sum: number, t: { amount: number }) => sum +
t.amount, 0);
}
...

`src\actions\food.ts`

```ts
'use server';

import { createClient } from '@lib/supabase/server';
import { DEFAULT_PROJECT_ID } from '@lib/constants';
import type { FoodRecord } from '@lib/types';
import { revalidatePath } from 'next/cache';

export async function createFoodRecord(data: Partial<FoodRecord> &
{ account_id?: string }) {
  const supabase = await createClient();

  const { data: record, error } = await supabase.from('food_records').insert({
    project_id: DEFAULT_PROJECT_ID,
    worker_id: data.worker_id || null,
    worker_type_id: data.worker_type_id || null,
    food_category_id: data.food_category_id,
    start_date: data.start_date,
    end_date: data.end_date,
    quantity: data.quantity || null,
    unit: data.unit || null,
    amount: data.amount || null,
    shop_id: data.shop_id || null,
    comments: data.comments || null,
  }).select('id').single();

  if (error) throw new Error('Failed to save food record');

  // If there is an amount, we optionally create a transaction if it was paid
  immediately.
  if (data.amount && data.amount > 0) {
    await supabase.from('transactions').insert({

```

```

        project_id: DEFAULT_PROJECT_ID,
        type: 'operational_expense',
        date: data.start_date,
        amount: data.amount,
        party_id: data.shop_id || null,
        account_id: data.account_id || null,
        reference_table: 'food_records',
        reference_id: record.id,
        description: `Food payment for ${data.start_date}`,
    });
}

revalidatePath('/');
revalidatePath('/food');
}

export async function getFoodRecords(limit = 50) {
    const supabase = await createClient();

    const { data, error } = await supabase
        .from('food_records')
        .select(`
            *,
            worker:parties!food_records_worker_id_fkey(name),
            worker_type:worker_types(name),
            food_category:food_categories(name),
            shop:parties!food_records_shop_id_fkey(name)
        `)
        .eq('project_id', DEFAULT_PROJECT_ID)
        .order('start_date', { ascending: false })
        .order('created_at', { ascending: false })
        .limit(limit);

    if (error) throw new Error('Failed to load food records');
    return data;
}

...

## `src\actions\labour.ts`

```ts
'use server';

import { createClient } from '@lib/supabase/server';
import { DEFAULT_PROJECT_ID } from '@lib/constants';
import type { LabourRecord } from '@lib/types';
import { revalidatePath } from 'next/cache';

export async function createLabourRecord(data: Partial<LabourRecord> &
{ account_id?: string }) {
 const supabase = await createClient();

 const { data: record, error } = await
supabase.from('labour_records').insert({
 project_id: DEFAULT_PROJECT_ID,
 worker_id: data.worker_id,
 worker_type_id: data.worker_type_id,
 work_type_id: data.work_type_id,
 building_id: data.building_id,
 date: data.date,

```

```

 time: data.time || null,
 amount: data.amount || null,
 cash_provider_id: data.cash_provider_id || null,
 payment_method: data.payment_method || 'cash',
 comments: data.comments || null,
 }).select('id').single();

 if (error) {
 console.error('Failed to create labour record:', error);
 throw new Error('Failed to save labour record');
 }

 // Generate transaction if there is an amount paid
 if (data.amount && data.amount > 0) {
 await supabase.from('transactions').insert({
 project_id: DEFAULT_PROJECT_ID,
 type: 'operational_expense',
 date: data.date,
 amount: data.amount,
 party_id: data.worker_id,
 account_id: data.account_id || null,
 payment_method: data.payment_method || 'cash',
 reference_table: 'labour_records',
 reference_id: record.id,
 description: `Labour payment for ${data.date}`,
 });
 }

 revalidatePath('/');
 revalidatePath('/labour');
}

export async function getLabourRecords(limit = 50) {
 const supabase = await createClient();

 const { data, error } = await supabase
 .from('labour_records')
 .select(`
 *,
 worker:parties!labour_records_worker_id_fkey(name),
 worker_type:worker_types(name),
 work_type:work_types(name),
 building:buildings(display_name),
 cash_provider:parties!labour_records_cash_provider_id_fkey(name)
 `)
 .eq('project_id', DEFAULT_PROJECT_ID)
 .order('date', { ascending: false })
 .order('created_at', { ascending: false })
 .limit(limit);

 if (error) throw new Error('Failed to load labour records');
 return data;
}

...

`src\actions\machinery.ts`

```ts
'use server';

```

```

import { createClient } from '@lib/supabase/server';
import { DEFAULT_PROJECT_ID } from '@lib/constants';
import type { MachineryRecord, FuelRecord } from '@lib/types';
import { revalidatePath } from 'next/cache';

export async function createMachineryRecord(
  data: Partial<MachineryRecord> & { account_id?: string },
  fuelData?: Partial<FuelRecord> & { account_id?: string }
) {
  const supabase = await createClient();

  const { error } = await supabase.from('machinery_records').insert({
    project_id: DEFAULT_PROJECT_ID,
    machine_id: data.machine_id,
    operator_id: data.operator_id || null,
    building_id: data.building_id || null,
    date: data.date,
    start_time: data.start_time || null,
    end_time: data.end_time || null,
    hours: data.hours || null,
    amount: data.amount || null,
    charging_type: data.charging_type || null,
    comments: data.comments || null,
  });

  if (error) throw new Error('Failed to save machinery record');

  if (fuelData) {
    const { error: fuelError } = await supabase.from('fuel_records').insert({
      project_id: DEFAULT_PROJECT_ID,
      machine_id: data.machine_id,
      building_id: data.building_id || null,
      fuel_type: fuelData.fuel_type,
      quantity: fuelData.quantity,
      unit: fuelData.unit || 'Liters',
      rate: fuelData.rate || null,
      amount: fuelData.amount || null,
      previous_meter: fuelData.previous_meter || null,
      current_meter: fuelData.current_meter || null,
      date: data.date,
      provider_id: fuelData.provider_id || null,
      comments: fuelData.comments || null,
    });

    if (fuelError) console.error('Failed to save fuel record', fuelError);

    if (fuelData.amount && fuelData.amount > 0) {
      await supabase.from('transactions').insert({
        project_id: DEFAULT_PROJECT_ID,
        type: 'operational_expense',
        date: data.date,
        amount: fuelData.amount,
        party_id: fuelData.provider_id || null,
        account_id: fuelData.account_id || data.account_id || null,
        reference_table: 'fuel_records',
        description: `Fuel payment for ${data.date}`,
      });
    }
  }

  if (data.amount && data.amount > 0) {
    await supabase.from('transactions').insert({

```

```

        project_id: DEFAULT_PROJECT_ID,
        type: 'operational_expense',
        date: data.date,
        amount: data.amount,
        party_id: data.operator_id || null,
        account_id: data.account_id || null,
        reference_table: 'machinery_records',
        description: `Machinery payment for ${data.date}`,
    });
}

revalidatePath('/');
revalidatePath('/machinery');
}

export async function createFuelRecord(data: Partial<FuelRecord> &
{ account_id?: string }) {
    const supabase = await createClient();

    const { error } = await supabase.from('fuel_records').insert({
        project_id: DEFAULT_PROJECT_ID,
        machine_id: data.machine_id,
        building_id: data.building_id || null,
        fuel_type: data.fuel_type,
        quantity: data.quantity,
        unit: data.unit || 'Litres',
        rate: data.rate || null,
        amount: data.amount || null,
        previous_meter: data.previous_meter || null,
        current_meter: data.current_meter || null,
        date: data.date,
        provider_id: data.provider_id || null,
        comments: data.comments || null,
    });

    if (error) throw new Error('Failed to save fuel record');

    if (data.amount && data.amount > 0) {
        await supabase.from('transactions').insert({
            project_id: DEFAULT_PROJECT_ID,
            type: 'operational_expense',
            date: data.date,
            amount: data.amount,
            party_id: data.provider_id || null,
            account_id: data.account_id || null,
            reference_table: 'fuel_records',
            description: `Fuel payment for ${data.date}`,
        });
    }

    revalidatePath('/');
    revalidatePath('/machinery');
}

export async function getMachineryRecords(limit = 50) {
    const supabase = await createClient();

    const { data, error } = await supabase
        .from('machinery_records')
        .select(`
            *,

```

```

        machine:machinery(machine_id, machine_type),
        operator:parties!machinery_records_operator_id_fkey(name),
        building:buildings(display_name)
    `)
    .eq('project_id', DEFAULT_PROJECT_ID)
    .order('date', { ascending: false })
    .order('created_at', { ascending: false })
    .limit(limit);

    if (error) throw new Error('Failed to load machinery records');
    return data;
}

...

## `src\actions\master.ts`

```ts
'use server';

import { createClient } from '@lib/supabase/server';
import { DEFAULT_PROJECT_ID } from '@lib/constants';

export async function getBuildings() {
 const supabase = await createClient();
 const { data, error } = await supabase
 .from('buildings')
 .select('*')
 .eq('project_id', DEFAULT_PROJECT_ID)
 .order('direction')
 .order('number');

 if (error) throw new Error('Failed to load buildings');
 return data;
}

export async function getWorkerTypes() {
 const supabase = await createClient();
 const { data, error } = await supabase
 .from('worker_types')
 .select('*')
 .eq('project_id', DEFAULT_PROJECT_ID)
 .order('name');

 if (error) throw new Error('Failed to load worker types');
 return data;
}

export async function getWorkTypes() {
 const supabase = await createClient();
 const { data, error } = await supabase
 .from('work_types')
 .select('*')
 .eq('project_id', DEFAULT_PROJECT_ID)
 .order('name');

 if (error) throw new Error('Failed to load work types');
 return data;
}

export async function getTransportVehicles() {

```



```

 const supabase = await createClient();
 const { data, error } = await supabase
 .from('transport_vehicles')
 .select('*')
 .eq('project_id', DEFAULT_PROJECT_ID);

 if (error) throw new Error('Failed to load transport vehicles');
 return data;
 }

export async function getWeighbridges() {
 const supabase = await createClient();
 const { data, error } = await supabase
 .from('weighbridges')
 .select('*')
 .eq('project_id', DEFAULT_PROJECT_ID);

 if (error) throw new Error('Failed to load weighbridges');
 return data;
}

export async function getMachinery() {
 const supabase = await createClient();
 const { data, error } = await supabase
 .from('machinery')
 .select('*')
 .eq('project_id', DEFAULT_PROJECT_ID)
 .eq('is_active', true);

 if (error) throw new Error('Failed to load machinery');
 return data;
}

export async function getFuelTypes() {
 const supabase = await createClient();
 const { data, error } = await supabase
 .from('fuel_types')
 .select('*')
 .eq('project_id', DEFAULT_PROJECT_ID);

 if (error) throw new Error('Failed to load fuel types');
 return data;
}

export async function getFoodCategories() {
 const supabase = await createClient();
 const { data, error } = await supabase
 .from('food_categories')
 .select('*')
 .eq('project_id', DEFAULT_PROJECT_ID);

 if (error) throw new Error('Failed to load food categories');
 return data;
}

...

`src\actions\materials.ts`

```ts
'use server';

```

```

import { createClient } from '@lib/supabase/server';
import { DEFAULT_PROJECT_ID } from '@lib/constants';
import type { MaterialDelivery, TransportRecord, WeighbridgeRecord } from '@lib/types';
import { revalidatePath } from 'next/cache';

export async function createMaterialDelivery(
  delivery: Partial<MaterialDelivery>,
  transport?: Partial<TransportRecord> & { account_id?: string },
  weighbridge?: Partial<WeighbridgeRecord> & { account_id?: string },
  account_id?: string
) {
  const supabase = await createClient();

  // Insert delivery
  const { data: deliveryData, error: deliveryError } = await supabase
    .from('material_deliveries')
    .insert({
      project_id: DEFAULT_PROJECT_ID,
      material_id: delivery.material_id,
      variant_id: delivery.variant_id || null,
      supplier_id: delivery.supplier_id || null,
      building_id: delivery.building_id || null,
      date: delivery.date,
      time: delivery.time || null,
      quantity: delivery.quantity || null,
      unit: delivery.unit || null,
      material_cost: delivery.material_cost || null,
      comments: delivery.comments || null,
    })
    .select('id')
    .single();

  if (deliveryError) {
    console.error('Failed to create delivery:', deliveryError);
    throw new Error('Failed to save material delivery');
  }

  const deliveryId = deliveryData.id;

  // Insert transport if provided
  if (transport && (transport.transport_type || transport.amount)) {
    await supabase.from('transport_records').insert({
      delivery_id: deliveryId,
      vehicle_id: transport.vehicle_id || null,
      transport_type: transport.transport_type || null,
      source_location: transport.source_location || null,
      amount: transport.amount || null,
    });
  }

  // Insert weighbridge if provided
  if (weighbridge && (weighbridge.gross_weight || weighbridge.fee)) {
    await supabase.from('weighbridge_records').insert({
      delivery_id: deliveryId,
      weighbridge_id: weighbridge.weighbridge_id || null,
      gross_weight: weighbridge.gross_weight || null,
      tare_weight: weighbridge.tare_weight || null,
      fee: weighbridge.fee || null,
    });
  }
}

```

```

}

// Transactions
if (delivery.material_cost && delivery.material_cost > 0) {
  await supabase.from('transactions').insert({
    project_id: DEFAULT_PROJECT_ID,
    type: 'supplier_payment',
    date: delivery.date,
    amount: delivery.material_cost,
    party_id: delivery.supplier_id || null,
    account_id: account_id || null,
    reference_table: 'material_deliveries',
    reference_id: deliveryId,
    description: `Material payment on ${delivery.date}`,
  });
}

if (transport && transport.amount && transport.amount > 0) {
  await supabase.from('transactions').insert({
    project_id: DEFAULT_PROJECT_ID,
    type: 'operational_expense',
    date: delivery.date,
    amount: transport.amount,
    account_id: transport.account_id || account_id || null,
    reference_table: 'transport_records',
    reference_id: deliveryId,
    description: `Transport payment for material on ${delivery.date}`,
  });
}

if (weighbridge && weighbridge.fee && weighbridge.fee > 0) {
  await supabase.from('transactions').insert({
    project_id: DEFAULT_PROJECT_ID,
    type: 'operational_expense',
    date: delivery.date,
    amount: weighbridge.fee,
    account_id: weighbridge.account_id || account_id || null,
    reference_table: 'weighbridge_records',
    reference_id: deliveryId,
    description: `Weighbridge fee for material on ${delivery.date}`,
  });
}

revalidatePath('/');
revalidatePath('/materials');
}

export async function getMaterialDeliveries(limit = 50) {
  const supabase = await createClient();

  const { data, error } = await supabase
    .from('material_deliveries')
    .select(`
      *,
      material:materials(name),
      variant:material_variants(name),
      supplier:parties!material_deliveries_supplier_id_fkey(name),
      building:buildings(display_name)
    `)
    .eq('project_id', DEFAULT_PROJECT_ID)
    .order('date', { ascending: false })

```

```

        .order('created_at', { ascending: false })
        .limit(limit);

    if (error) throw new Error('Failed to load material deliveries');
    return data;
}

export async function getMaterials() {
    const supabase = await createClient();
    const { data, error } = await supabase
        .from('materials')
        .select('*')
        .eq('project_id', DEFAULT_PROJECT_ID);
    if (error) throw new Error('Failed to fetch materials');
    return data;
}

export async function getMaterialVariants(materialId: string) {
    const supabase = await createClient();
    const { data, error } = await supabase
        .from('material_variants')
        .select('*')
        .eq('material_id', materialId);
    if (error) throw new Error('Failed to fetch material variants');
    return data;
}

...

## `src\actions\money.ts`

```ts
'use server';

import { createClient } from '@lib/supabase/server';
import { DEFAULT_PROJECT_ID } from '@lib/constants';
import { revalidatePath } from 'next/cache';

export async function addMoneyIn(data: {
 amount: number;
 account_id: string;
 party_id?: string;
 date: string;
 payment_method?: string;
 description?: string;
}) {
 const supabase = await createClient();

 const { error } = await supabase.from('transactions').insert({
 project_id: DEFAULT_PROJECT_ID,
 type: 'money_in',
 amount: data.amount,
 account_id: data.account_id,
 party_id: data.party_id || null,
 date: data.date,
 payment_method: data.payment_method || 'cash',
 description: data.description || 'Cash received',
 });

 if (error) {
 console.error('Failed to record money in:', error);
 }
}

```

```

 throw new Error('Failed to record money in');
 }

 revalidatePath('/');
 revalidatePath('/money');
}

export async function createTransfer(data: {
 amount: number;
 from_account_id: string;
 to_account_id: string;
 date: string;
 description?: string;
}) {
 const supabase = await createClient();

 const { error } = await supabase.from('transactions').insert({
 project_id: DEFAULT_PROJECT_ID,
 type: 'transfer',
 amount: data.amount,
 account_id: data.from_account_id,
 to_account_id: data.to_account_id,
 date: data.date,
 payment_method: 'bank_transfer',
 description: data.description || 'Transfer between accounts',
 });

 if (error) {
 console.error('Failed to create transfer:', error);
 throw new Error('Failed to create transfer');
 }

 revalidatePath('/');
 revalidatePath('/money');
}

export async function getRecentTransactions(limit = 20) {
 const supabase = await createClient();

 const { data, error } = await supabase
 .from('transactions')
 .select('*', party:parties(name))
 .eq('project_id', DEFAULT_PROJECT_ID)
 .order('date', { ascending: false })
 .order('created_at', { ascending: false })
 .limit(limit);

 if (error) throw new Error('Failed to load transactions');
 return data || [];
}

...

`src\actions\parties.ts`

```ts
'use server';

import { createClient } from '@lib/supabase/server';
import { DEFAULT_PROJECT_ID } from '@lib/constants';
import type { Party } from '@lib/types';

```

```

export async function getParties(search?: string, role?: string):
Promise<Party[]> {
  const supabase = await createClient();

  let query = supabase
    .from('parties')
    .select('*')
    .eq('project_id', DEFAULT_PROJECT_ID)
    .eq('is_active', true)
    .order('name');

  if (search) {
    query = query.ilike('name', `%${search}%`);
  }

  if (role) {
    query = query.eq('role', role);
  }

  const { data, error } = await query;
  if (error) throw new Error(error.message);
  return (data as Party[]) || [];
}

export async function createParty(data: Partial<Party>): Promise<Party> {
  const supabase = await createClient();

  const { data: party, error } = await supabase
    .from('parties')
    .insert({
      project_id: DEFAULT_PROJECT_ID,
      name: data.name?.trim(),
      class: data.class || 'person',
      role: data.role || 'general',
      worker_type_id: data.worker_type_id || null,
      phone: data.phone || null,
      notes: data.notes || null,
    })
    .select()
    .single();

  if (error) throw new Error(error.message);
  return party as Party;
}

export async function updateParty(id: string, data: Partial<Party>):
Promise<Party> {
  const supabase = await createClient();

  const { data: party, error } = await supabase
    .from('parties')
    .update(data)
    .eq('id', id)
    .select()
    .single();

  if (error) throw new Error(error.message);
  return party as Party;
}

```

```

export async function getParty(id: string): Promise<Party> {
  const supabase = await createClient();
  const { data, error } = await supabase
    .from('parties')
    .select('*')
    .eq('id', id)
    .single();

  if (error) throw new Error(error.message);
  return data as Party;
}

export async function getPartyTransactions(partyId: string, dateFrom?:
string, dateTo?: string) {
  const supabase = await createClient();

  // Get all transactions where this party is involved
  let query = supabase
    .from('transactions')
    .select('*', account:accounts(name, type)')
    .eq('party_id', partyId)
    .order('date', { ascending: true })
    .order('created_at', { ascending: true });

  if (dateFrom) query = query.gte('date', dateFrom);
  if (dateTo) query = query.lte('date', dateTo);

  const { data, error } = await query;
  if (error) throw new Error(error.message);
  return data;
}

export async function deleteParty(id: string) {
  const supabase = await createClient();
  const { error } = await supabase
    .from('parties')
    .update({ is_active: false })
    .eq('id', id);

  if (error) throw new Error(error.message);
}

...

## `src\actions\reports.ts`

```ts
'use server';

import { createClient } from '@lib/supabase/server';
import { DEFAULT_PROJECT_ID } from '@lib/constants';

export interface ModuleSpend {
 module: string;
 label: string;
 icon: string;
 total: number;
}

export async function getMonthlyReport(year: number, month: number):
Promise<{ modules: ModuleSpend[]; grandTotal: number }> {

```

```

const supabase = await createClient();

const firstDay = `${year}-${String(month).padStart(2, '0')}-01`;
const lastDay = new Date(year, month, 0).toLocaleDateString('en-CA');

// Fetch all record types for the month in parallel
const [labourRes, foodRes, machineryRes, salaryRes, materialRes,
transportRes] = await Promise.all([
 supabase
 .from('labour_records')
 .select('amount')
 .eq('project_id', DEFAULT_PROJECT_ID)
 .gte('date', firstDay)
 .lte('date', lastDay)
 .not('amount', 'is', null),
 supabase
 .from('food_records')
 .select('amount')
 .eq('project_id', DEFAULT_PROJECT_ID)
 .gte('start_date', firstDay)
 .lte('start_date', lastDay)
 .not('amount', 'is', null),
 supabase
 .from('machinery_records')
 .select('amount')
 .eq('project_id', DEFAULT_PROJECT_ID)
 .gte('date', firstDay)
 .lte('date', lastDay)
 .not('amount', 'is', null),
 supabase
 .from('salary_records')
 .select('amount')
 .eq('project_id', DEFAULT_PROJECT_ID)
 .gte('payment_date', firstDay)
 .lte('payment_date', lastDay),
 supabase
 .from('material_deliveries')
 .select('material_cost')
 .eq('project_id', DEFAULT_PROJECT_ID)
 .gte('date', firstDay)
 .lte('date', lastDay)
 .not('material_cost', 'is', null),
 supabase
 .from('transport_records')
 .select('amount, delivery:material_deliveries!inner(project_id, date)')
 .not('amount', 'is', null),
 // Note: transport_records link through material_deliveries for project/
 // date context.
 // We filter after fetch since Supabase nested filters have limitations.
]);

const sum = (rows: any[] | null, field = 'amount') =>
 (rows || []).reduce((s: number, r: any) => s + (Number(r[field]) || 0),
0);

// Filter transport records by project and date range via joined delivery
const filteredTransport = (transportRes.data || []).filter((r: any) => {
 const delivery = r.delivery as any;
 if (!delivery) return false;
 return delivery.project_id === DEFAULT_PROJECT_ID &&
 delivery.date >= firstDay &&

```



```

 delivery.date <= lastDay;
 });

 const modules: ModuleSpend[] = [
 { module: 'labour', label: 'Labour', icon: 'Ø=Üw', total:
sum(labourRes.data) },
 { module: 'food', label: 'Food & Groceries', icon: 'Ø<ßZ', total:
sum(foodRes.data) },
 { module: 'machinery', label: 'Machinery', icon: 'Ø=þæ', total:
sum(machineryRes.data) },
 { module: 'salary', label: 'Salary', icon: 'Ø=Ü°', total:
sum(salaryRes.data) },
 { module: 'materials', label: 'Materials', icon: 'Ø>Ýñ', total:
sum(materialRes.data, 'material_cost') },
 { module: 'transport', label: 'Transport', icon: 'Ø=þš', total:
sum(filteredTransport) },
];

 const grandTotal = modules.reduce((s, m) => s + m.total, 0);

 return { modules, grandTotal };
}

export async function getDayBook(date: string) {
 const supabase = await createClient();

 const { data, error } = await supabase
 .from('transactions')
 .select('*', party:parties(name), account:accounts(name))
 .eq('project_id', DEFAULT_PROJECT_ID)
 .eq('date', date)
 .order('created_at', { ascending: true });

 if (error) throw new Error(error.message);
 return data || [];
}

export async function getBuildingReport(year: number, month: number) {
 const supabase = await createClient();

 const firstDay = `${year}-${String(month).padStart(2, '0')}-01`;
 const lastDay = new Date(year, month, 0).toLocaleDateString('en-CA');

 const [buildingsRes, labourRes, materialRes, machineryRes] = await
Promise.all([
 supabase.from('buildings').select('id, display_name').eq('project_id',
DEFAULT_PROJECT_ID),
 supabase
 .from('labour_records')
 .select('building_id, amount')
 .eq('project_id', DEFAULT_PROJECT_ID)
 .gte('date', firstDay)
 .lte('date', lastDay)
 .not('amount', 'is', null)
 .not('building_id', 'is', null),
 supabase
 .from('material_deliveries')
 .select('building_id, material_cost')
 .eq('project_id', DEFAULT_PROJECT_ID)
 .gte('date', firstDay)
 .lte('date', lastDay)

```

```

 .not('material_cost', 'is', null)
 .not('building_id', 'is', null),
 supabase
 .from('machinery_records')
 .select('building_id, amount')
 .eq('project_id', DEFAULT_PROJECT_ID)
 .gte('date', firstDay)
 .lte('date', lastDay)
 .not('amount', 'is', null)
 .not('building_id', 'is', null)
]]);

const buildings = buildingsRes.data || [];
const labour = labourRes.data || [];
const materials = materialRes.data || [];
const machinery = machineryRes.data || [];

const buildingCosts = buildings.map((b: any) => {
 const lCost = labour.filter((r: any) => r.building_id ===
b.id).reduce((s: number, r: any) => s + (Number(r.amount) || 0), 0);
 const mCost = materials.filter((r: any) => r.building_id ===
b.id).reduce((s: number, r: any) => s + (Number(r.material_cost) || 0), 0);
 const machCost = machinery.filter((r: any) => r.building_id ===
b.id).reduce((s: number, r: any) => s + (Number(r.amount) || 0), 0);

 return {
 id: b.id,
 name: b.display_name,
 total: lCost + mCost + machCost,
 breakdown: {
 labour: lCost,
 materials: mCost,
 machinery: machCost
 }
 };
}).filter((b: any) => b.total > 0);

buildingCosts.sort((a: any, b: any) => b.total - a.total);

return buildingCosts;
}
...

```

```

`src\actions\salary.ts`

```

```

``ts
'use server';

import { createClient } from '@lib/supabase/server';
import { DEFAULT_PROJECT_ID } from '@lib/constants';
import type { SalaryRecord } from '@lib/types';
import { revalidatePath } from 'next/cache';

export async function createSalaryRecord(data: Partial<SalaryRecord> &
{ account_id?: string }) {
 const supabase = await createClient();

 const { data: record, error } = await
supabase.from('salary_records').insert({
 project_id: DEFAULT_PROJECT_ID,

```

```

 employee_id: data.employee_id,
 worker_type_id: data.worker_type_id || null,
 start_date: data.start_date,
 end_date: data.end_date,
 amount: data.amount,
 payment_date: data.payment_date,
 payment_method: data.payment_method || 'cash',
 cash_provider_id: data.cash_provider_id || null,
 comments: data.comments || null,
 }).select('id').single();

 if (error) {
 console.error('Failed to create salary record:', error);
 throw new Error('Failed to save salary record');
 }

 // Generate transaction if there is an amount paid
 if (data.amount && data.amount > 0) {
 await supabase.from('transactions').insert({
 project_id: DEFAULT_PROJECT_ID,
 type: 'operational_expense',
 date: data.payment_date || data.start_date,
 amount: data.amount,
 party_id: data.employee_id,
 account_id: data.account_id || null,
 payment_method: data.payment_method || 'cash',
 reference_table: 'salary_records',
 reference_id: record.id,
 description: `Salary payment for ${data.start_date}`,
 });
 }

 revalidatePath('/');
 revalidatePath('/salary');
}

export async function getSalaryRecords(limit = 50) {
 const supabase = await createClient();

 const { data, error } = await supabase
 .from('salary_records')
 .select(`
 *,
 employee:parties!salary_records_employee_id_fkey(name),
 worker_type:worker_types(name),
 cash_provider:parties!salary_records_cash_provider_id_fkey(name)
 `)
 .eq('project_id', DEFAULT_PROJECT_ID)
 .order('payment_date', { ascending: false })
 .order('created_at', { ascending: false })
 .limit(limit);

 if (error) throw new Error('Failed to load salary records');
 return data;
}

...

`src\actions\transactions.ts`

```ts

```

```

'use server';

import { createClient } from '@lib/supabase/server';
import { DEFAULT_PROJECT_ID } from '@lib/constants';
import { revalidatePath } from 'next/cache';

export interface ActivityItem {
  id: string;
  module: 'labour' | 'food' | 'machinery' | 'salary' | 'transport' |
'materials' | 'money_in' | 'transfer';
  date: string;
  amount: number | null;
  title: string;
  subtitle: string;
  icon: string;
}

export async function getActivityFeed(
  limit = 50,
  filters?: { module?: string; search?: string; dateFrom?: string; dateTo?:
string }
): Promise<ActivityItem[]> {
  const supabase = await createClient();
  const items: ActivityItem[] = [];

  // Fetch labour records
  const labourQuery = supabase
    .from('labour_records')
    .select('id, date, amount, comments, worker:parties!
labour_records_worker_id_fkey(name), work_type:work_types(name)')
    .eq('project_id', DEFAULT_PROJECT_ID)
    .order('date', { ascending: false })
    .limit(filters?.search ? 1000 : limit);

  if (filters?.dateFrom) labourQuery.gte('date', filters.dateFrom);
  if (filters?.dateTo) labourQuery.lte('date', filters.dateTo);

  // Fetch food records
  const foodQuery = supabase
    .from('food_records')
    .select('id, start_date, amount, comments,
food_category:food_categories(name), shop:parties!
food_records_shop_id_fkey(name)')
    .eq('project_id', DEFAULT_PROJECT_ID)
    .order('start_date', { ascending: false })
    .limit(filters?.search ? 1000 : limit);

  if (filters?.dateFrom) foodQuery.gte('start_date', filters.dateFrom);
  if (filters?.dateTo) foodQuery.lte('start_date', filters.dateTo);

  // Fetch machinery records
  const machineryQuery = supabase
    .from('machinery_records')
    .select('id, date, amount, hours, comments, machine:machinery(machine_id,
machine_type)')
    .eq('project_id', DEFAULT_PROJECT_ID)
    .order('date', { ascending: false })
    .limit(filters?.search ? 1000 : limit);

  if (filters?.dateFrom) machineryQuery.gte('date', filters.dateFrom);
  if (filters?.dateTo) machineryQuery.lte('date', filters.dateTo);

```

```

// Fetch salary records
const salaryQuery = supabase
  .from('salary_records')
  .select('id, payment_date, amount, comments, employee:parties!
salary_records_employee_id_fkey(name)')
  .eq('project_id', DEFAULT_PROJECT_ID)
  .order('payment_date', { ascending: false })
  .limit(filters?.search ? 1000 : limit);

if (filters?.dateFrom) salaryQuery.gte('payment_date', filters.dateFrom);
if (filters?.dateTo) salaryQuery.lte('payment_date', filters.dateTo);

// Fetch material deliveries
const materialQuery = supabase
  .from('material_deliveries')
  .select('id, date, material_cost, quantity, unit, comments,
material:materials(name), supplier:parties!
material_deliveries_supplier_id_fkey(name)')
  .eq('project_id', DEFAULT_PROJECT_ID)
  .order('date', { ascending: false })
  .limit(filters?.search ? 1000 : limit);

if (filters?.dateFrom) materialQuery.gte('date', filters.dateFrom);
if (filters?.dateTo) materialQuery.lte('date', filters.dateTo);

// Fetch money-in and transfer transactions
const moneyQuery = supabase
  .from('transactions')
  .select('id, date, amount, type, description, party:parties(name)')
  .eq('project_id', DEFAULT_PROJECT_ID)
  .in('type', ['money_in', 'transfer'])
  .order('date', { ascending: false })
  .limit(filters?.search ? 1000 : limit);

if (filters?.dateFrom) moneyQuery.gte('date', filters.dateFrom);
if (filters?.dateTo) moneyQuery.lte('date', filters.dateTo);

// Execute all in parallel
const [labourRes, foodRes, machineryRes, salaryRes, materialRes, moneyRes]
= await Promise.all([
  labourQuery,
  foodQuery,
  machineryQuery,
  salaryQuery,
  materialQuery,
  moneyQuery,
]);

// Map labour
for (const r of labourRes.data || []) {
  const workerName = (r.worker as any)?.name || 'Worker';
  const workType = (r.work_type as any)?.name || '';
  if (filters?.search && !
workerName.toLowerCase().includes(filters.search.toLowerCase()) && !
workType.toLowerCase().includes(filters.search.toLowerCase())) continue;
  items.push({
    id: r.id,
    module: 'labour',
    date: r.date,
    amount: r.amount,

```

```

        title: workerName,
        subtitle: workType || 'Labour',
        icon: 'Ø=Üw',
    });
}

// Map food
for (const r of foodRes.data || []) {
    const category = (r.food_category as any)?.name || 'Food';
    const shop = (r.shop as any)?.name || '';
    if (filters?.search && !
category.toLowerCase().includes(filters.search.toLowerCase()) && !
shop.toLowerCase().includes(filters.search.toLowerCase())) continue;
    items.push({
        id: r.id,
        module: 'food',
        date: r.start_date,
        amount: r.amount,
        title: category,
        subtitle: shop || 'Food & Groceries',
        icon: 'Ø<ßZ',
    });
}

// Map machinery
for (const r of machineryRes.data || []) {
    const machine = r.machine as any;
    const name = machine?.machine_id || 'Machine';
    const type = machine?.machine_type || '';
    if (filters?.search && !
name.toLowerCase().includes(filters.search.toLowerCase()) && !
type.toLowerCase().includes(filters.search.toLowerCase())) continue;
    items.push({
        id: r.id,
        module: 'machinery',
        date: r.date,
        amount: r.amount,
        title: name,
        subtitle: r.hours ? `${r.hours}h – ${type}` : type || 'Machinery',
        icon: 'Ø=þæ',
    });
}

// Map salary
for (const r of salaryRes.data || []) {
    const empName = (r.employee as any)?.name || 'Employee';
    if (filters?.search && !
empName.toLowerCase().includes(filters.search.toLowerCase())) continue;
    items.push({
        id: r.id,
        module: 'salary',
        date: r.payment_date,
        amount: r.amount,
        title: empName,
        subtitle: 'Salary',
        icon: 'Ø=Ü°',
    });
}

// Map materials
for (const r of materialRes.data || []) {

```

```

    const matName = (r.material as any)?.name || 'Material';
    const supplier = (r.supplier as any)?.name || '';
    const qtyStr = r.quantity ? `${r.quantity} ${r.unit || ''}` : '';
    if (filters?.search && !
matName.toLowerCase().includes(filters.search.toLowerCase()) && !
supplier.toLowerCase().includes(filters.search.toLowerCase())) continue;
    items.push({
      id: r.id,
      module: 'materials',
      date: r.date,
      amount: r.material_cost,
      title: matName,
      subtitle: [qtyStr, supplier].filter(Boolean).join(' - ') || 'Material
Delivery',
      icon: 'Ø>Ýñ',
    });
  }

  // Map money-in / transfer
  for (const r of moneyRes.data || []) {
    const partyName = (r.party as any)?.name || '';
    const isTransfer = r.type === 'transfer';
    if (filters?.search && !
partyName.toLowerCase().includes(filters.search.toLowerCase()) && !
(r.description || '').toLowerCase().includes(filters.search.toLowerCase()))
continue;
    items.push({
      id: r.id,
      module: isTransfer ? 'transfer' : 'money_in',
      date: r.date,
      amount: r.amount,
      title: isTransfer ? 'Transfer' : 'Money In',
      subtitle: r.description || partyName || '',
      icon: isTransfer ? '!"p ' : 'Ø=Ûµ',
    });
  }

  // Filter by module if specified
  const filtered = filters?.module
    ? items.filter(i => i.module === filters.module)
    : items;

  // Sort by date descending
  filtered.sort((a, b) => b.date.localeCompare(a.date));

  return filtered.slice(0, limit);
}

export async function deleteRecord(tableName: string, id: string) {
  const supabase = await createClient();

  // If this is a transaction, just delete it directly
  if (tableName === 'transactions') {
    const { error } = await supabase.from('transactions').delete().eq('id',
id);
    if (error) throw new Error('Failed to delete transaction');
    revalidatePath('/');
    return;
  }

  // For material_deliveries, cascade-delete linked transport and weighbridge

```

```

records first
  if (tableName === 'material_deliveries') {
    // Clean up transport_records and their transactions
    const { data: transportRecords } = await supabase
      .from('transport_records')
      .select('id')
      .eq('delivery_id', id);

    if (transportRecords && transportRecords.length > 0) {
      for (const tr of transportRecords) {
        await supabase.from('transactions').delete()
          .eq('reference_table', 'transport_records')
          .eq('reference_id', tr.id);
      }
      await supabase.from('transport_records').delete().eq('delivery_id', id);
    }

    // Clean up weighbridge_records and their transactions
    const { data: weighbridgeRecords } = await supabase
      .from('weighbridge_records')
      .select('id')
      .eq('delivery_id', id);

    if (weighbridgeRecords && weighbridgeRecords.length > 0) {
      for (const wb of weighbridgeRecords) {
        await supabase.from('transactions').delete()
          .eq('reference_table', 'weighbridge_records')
          .eq('reference_id', wb.id);
      }
      await supabase.from('weighbridge_records').delete().eq('delivery_id',
id);
    }
  }

  // Delete the operational record
  const { error } = await supabase.from(tableName).delete().eq('id', id);
  if (error) throw new Error(`Failed to delete from ${tableName}`);

  // Also delete the associated transaction if one exists
  const { error: txnError } = await supabase
    .from('transactions')
    .delete()
    .eq('reference_table', tableName)
    .eq('reference_id', id);

  if (txnError) {
    console.error('Failed to clean up transaction', txnError);
  }

  revalidatePath('/');
  revalidatePath(`/${tableName.split('_')[0]}`);
}
...

## `src\actions\transport.ts`

```ts
'use server';

import { createClient } from '@lib/supabase/server';

```



```

import { DEFAULT_PROJECT_ID, DEFAULT_CASH_ACCOUNT_ID } from '@lib/constants';

export async function createStandaloneTransportRecord(data: any &
{ account_id?: string }) {
 const supabase = await createClient();

 const { data: record, error } = await supabase
 .from('transport_records')
 .insert({
 project_id: DEFAULT_PROJECT_ID,
 vehicle_id: data.vehicle_id,
 amount: data.amount,
 payment_method: data.payment_method,
 status: 'completed',
 })
 .select()
 .single();

 if (error) throw new Error(error.message);

 // Auto-deduct from cash/bank if paid
 if (data.amount && data.payment_method && data.payment_method !== 'credit')
 {
 const { error: txnError } = await supabase.from('transactions').insert({
 project_id: DEFAULT_PROJECT_ID,
 account_id: data.account_id || null,
 amount: data.amount,
 type: 'operational_expense',
 reference_table: 'transport_records',
 reference_id: record.id,
 date: data.date,
 description: `Transport fee (standalone)`,
 });
 if (txnError) console.error('Failed to create transport txn', txnError);
 }

 return record;
}

...

`src\app\api\backup\route.ts`

```ts
import { NextResponse } from 'next/server';
import { createClient } from '@lib/supabase/server';

const TABLES = [
  'parties',
  'transactions',
  'labour_records',
  'food_records',
  'machinery_records',
  'salary_records',
  'material_deliveries',
  'transport_records',
  'weighbridge_records',
  'fuel_records',
  'buildings',
  'machinery',
  'worker_types',

```

```

    'work_types',
    'food_categories'
  ];

export async function GET() {
  try {
    const supabase = await createClient();
    const backupData: Record<string, any[]> = {};

    // Fetch all tables
    for (const table of TABLES) {
      const { data, error } = await supabase.from(table).select('*');
      if (error) {
        console.error(`Error fetching ${table}:`, error);
        backupData[table] = []; // fallback
      } else {
        backupData[table] = data || [];
      }
    }

    // Add metadata
    const finalBackup = {
      timestamp: new Date().toISOString(),
      version: '1.0',
      data: backupData
    };

    return new NextResponse(JSON.stringify(finalBackup, null, 2), {
      status: 200,
      headers: {
        'Content-Type': 'application/json',
        'Content-Disposition': `attachment; filename="shambala-backup-${new
Date().toLocaleDateString('en-CA')}.json"`,
      },
    });
  } catch (error) {
    console.error('Backup error:', error);
    return NextResponse.json({ error: 'Internal Server Error' }, { status:
500 });
  }
}
...

```

```

## `src/app/api/export/route.ts`

```

```

````ts
import { NextRequest, NextResponse } from 'next/server';
import * as XLSX from 'xlsx';
import { createClient } from '@lib/supabase/server';
import { DEFAULT_PROJECT_ID } from '@lib/constants';

export async function GET(request: NextRequest) {
 try {
 const searchParams = request.nextUrl.searchParams;
 const type = searchParams.get('type') || 'all';
 const month = searchParams.get('month');
 const year = searchParams.get('year');

 const supabase = await createClient();

```

```

let query = supabase
 .from('transactions')
 .select(`
 *,
 category:categories(name),
 party:parties(name),
 account:accounts!transactions_account_id_fkey(name)
 `)
 .eq('project_id', DEFAULT_PROJECT_ID)
 .is('deleted_at', null)
 .order('date', { ascending: true });

// Apply filters
if (type !== 'all') {
 query = query.eq('type', type);
}
if (month && year) {
 const firstDay = `${year}-${month.padStart(2, '0')}-01`;
 const lastDay = new Date(parseInt(year), parseInt(month),
0).toLocaleDateString('en-CA');
 query = query.gte('date', firstDay).lte('date', lastDay);
}

const { data: transactions, error } = await query;
if (error) throw new Error(error.message);

// Build Excel data
// eslint-disable-next-line @typescript-eslint/no-explicit-any
const rows = (transactions || []).map((txn: any) => ({
 Date: txn.date,
 Type: txn.type,
 Category: (Array.isArray(txn.category) ? txn.category[0]?.name :
txn.category?.name) || '',
 'Paid To / From': (Array.isArray(txn.party) ? txn.party[0]?.name :
txn.party?.name) || '',
 Amount: txn.amount,
 'Payment Method': txn.payment_method,
 Account: (Array.isArray(txn.account) ? txn.account[0]?.name :
txn.account?.name) || '',
 Description: txn.description || '',
})));

const ws = XLSX.utils.json_to_sheet(rows);
const wb = XLSX.utils.book_new();
XLSX.utils.book_append_sheet(wb, ws, 'Transactions');

// Set column widths
ws['!cols'] = [
 { wch: 12 }, // Date
 { wch: 16 }, // Type
 { wch: 18 }, // Category
 { wch: 25 }, // Paid To
 { wch: 14 }, // Amount
 { wch: 15 }, // Payment Method
 { wch: 15 }, // Account
 { wch: 40 }, // Description
];

const buf = XLSX.write(wb, { type: 'buffer', bookType: 'xlsx' });
return new NextResponse(buf, {

```

```

 headers: {
 'Content-Type': 'application/vnd.openxmlformats-officedocument.spreadsheetml.sheet',
 'Content-Disposition': `attachment; filename="shambala-export-${new Date().toLocaleDateString('en-CA')}.xlsx"`,
 },
 });
} catch (err) {
 console.error('Export error:', err);
 return NextResponse.json(
 { error: err instanceof Error ? err.message : 'Export failed' },
 { status: 500 }
);
}
}
}

```

```

`src/app/api/import/route.ts`

```

```

` ``ts
import { NextRequest, NextResponse } from 'next/server';
import { createClient } from '@lib/supabase/server';
import { DEFAULT_PROJECT_ID, DEFAULT_CASH_ACCOUNT_ID } from '@lib/constants';
import type { ParsedRow } from '@lib/excel-parser';

export async function POST(request: NextRequest) {
 try {
 const { rows } = (await request.json()) as { rows: ParsedRow[] };

 if (!rows || !Array.isArray(rows) || rows.length === 0) {
 return NextResponse.json({ error: 'No rows to import' }, { status:
400 });
 }

 const supabase = await createClient();

 // Get existing parties for name matching
 const { data: existingParties } = await supabase
 .from('parties')
 .select('id, name')
 .eq('project_id', DEFAULT_PROJECT_ID);

 const partyMap = new Map<string, string>();
 for (const p of existingParties || []) {
 partyMap.set(p.name.toUpperCase(), p.id);
 }

 let imported = 0;
 let failed = 0;

 for (const row of rows) {
 try {
 const d = row.parsedData;
 if (!d.amount || d.amount <= 0) {
 failed++;
 continue;
 }

 // Resolve or create party
 let partyId: string | null = null;
 }
 }
 }
}

```

```

 if (d.party) {
 const upperName = d.party.toUpperCase();
 if (partyMap.has(upperName)) {
 partyId = partyMap.get(upperName)!;
 } else {
 const { data: newParty } = await supabase
 .from('parties')
 .insert({
 project_id: DEFAULT_PROJECT_ID,
 name: d.party,
 type: 'other',
 })
 .select('id')
 .single();
 if (newParty) {
 partyId = newParty.id;
 partyMap.set(upperName, newParty.id);
 }
 }
 }

 // Determine payment method and account
 const paymentMethod = d.paymentMethod || 'cash';
 const accountId = paymentMethod === 'bank_transfer'
 ? '00000000-0000-0000-0000-000000000011' // IDBI Bank
 : DEFAULT_CASH_ACCOUNT_ID;

 // Map transaction type
 const type = d.type || 'expense';

 const { error: txnError } = await supabase
 .from('transactions')
 .insert({
 project_id: DEFAULT_PROJECT_ID,
 type,
 date: d.date || new Date().toLocaleDateString('en-CA'),
 amount: d.amount,
 account_id: accountId,
 category_id: d.categoryId || null,
 party_id: partyId,
 description: d.description || null,
 payment_method: paymentMethod,
 });

 if (txnError) {
 console.error('Failed to import row:', txnError);
 failed++;
 } else {
 imported++;
 }
 } catch (err) {
 console.error('Row import error:', err);
 failed++;
 }
}

return NextResponse.json({ imported, failed, total: rows.length });
} catch (err) {
 console.error('Import error:', err);
 return NextResponse.json(
 { error: err instanceof Error ? err.message : 'Import failed' },
);
}

```

```

 { status: 500 }
);
 }
 }
 ...

`src\app\error.tsx`

```tsx
'use client';

import { useEffect } from 'react';

export default function GlobalError({
  error,
  reset,
}): {
  error: Error & { digest?: string };
  reset: () => void;
} {
  useEffect(() => {
    console.error('Unhandled error:', error);
  }, [error]);

  return (
    <div className="px-4 pt-16 pb-24 max-w-lg mx-auto text-center animate-fade-in">
      <div className="text-5xl mb-4">& p </div>
      <h1 className="text-xl font-bold mb-2">Something went wrong</h1>
      <p className="text-text-muted text-sm mb-6">
        {error.message || 'An unexpected error occurred. Please try again.'}
      </p>
      <button
        onClick={reset}
        className="px-6 py-3 bg-accent text-white font-semibold rounded-xl
hover:brightness-110 transition-all active:scale-[0.98]"
        >
        Try Again
      </button>
    </div>
  );
}

...

## `src\app\expenses\page.tsx`

```tsx
'use client';

import { useState, useEffect, useCallback } from 'react';
import { getActivityFeed, type ActivityItem } from '@actions/transactions';
import { TransactionCard } from '@components/TransactionCard';

const MODULE_FILTERS = [
 { key: '', label: 'All' },
 { key: 'labour', label: 'Ø=Üw Labour' },
 { key: 'food', label: 'Ø<ßZ Food' },
 { key: 'machinery', label: 'Ø=Þæ Machinery' },
 { key: 'salary', label: 'Ø=Ü° Salary' },

```

```

 { key: 'materials', label: 'Ø>Ýñ Materials' },
 { key: 'transport', label: 'Ø=Þš Transport' },
];

export default function ExpensesPage() {
 const [items, setItems] = useState<ActivityItem[]>([]);
 const [loading, setLoading] = useState(true);
 const [search, setSearch] = useState('');
 const [moduleFilter, setModuleFilter] = useState('');

 const loadData = useCallback(async () => {
 setLoading(true);
 try {
 const data = await getActivityFeed(100, {
 module: moduleFilter || undefined,
 search: search || undefined,
 });
 setItems(data);
 } catch (err) {
 console.error('Failed to load activity', err);
 } finally {
 setLoading(false);
 }
 }, [moduleFilter, search]);

 useEffect(() => {
 loadData();
 }, [loadData]);

 // Debounced search
 const [searchInput, setSearchInput] = useState('');
 useEffect(() => {
 const timer = setTimeout(() => setSearch(searchInput), 300);
 return () => clearTimeout(timer);
 }, [searchInput]);

 // Group items by date
 const grouped = items.reduce<Record<string, ActivityItem[]>>((acc, item) =>
 {
 const key = item.date;
 if (!acc[key]) acc[key] = [];
 acc[key].push(item);
 return acc;
 }, {});

 const sortedDates = Object.keys(grouped).sort((a, b) => b.localeCompare(a));

 return (
 <div className="px-4 pt-6 pb-24 max-w-lg mx-auto">
 <h1 className="text-xl font-bold mb-4 animate-fade-in">Transaction
History</h1>

 {/* Search Bar */}
 <div className="relative mb-3 animate-fade-in stagger-1">
 <input
 type="text"
 value={searchInput}
 onChange={(e) => setSearchInput(e.target.value)}
 placeholder="Ø=Ý Search transactions..."
 className="w-full bg-bg-card border border-border rounded-2xl px-4
py-3 text-text-primary placeholder:text-text-muted focus:outline-none

```

```

focus:border-accent transition-colors"
 />
 {searchInput && (
 <button
 onClick={() => { setSearchInput(''); setSearch(''); }}
 className="absolute right-3 top-1/2 -translate-y-1/2 text-text-
muted hover:text-text-secondary"
 >
 ,
 </button>
)}
</div>

{/* Module Filter Chips */}
<div className="flex gap-2 overflow-x-auto pb-3 mb-4 animate-fade-in
stagger-2 scrollbar-hide">
 {MODULE_FILTERS.map((f) => (
 <button
 key={f.key}
 onClick={() => setModuleFilter(f.key)}
 className={`px-3 py-1.5 rounded-full text-xs font-semibold
whitespace-nowrap border transition-all ${
 moduleFilter === f.key
 ? 'bg-accent text-white border-accent shadow-glow-accent'
 : 'bg-bg-card text-text-secondary border-border hover:border-
accent/30'
 }}
 >
 {f.label}
 </button>
))}
</div>

{/* Results */}
{loading ? (
 <div className="space-y-3">
 {[...Array(5)].map((_, i) => (
 <div key={i} className="bg-bg-card border border-border
rounded-2xl p-4 h-16 animate-pulse-subtle" />
))}
 </div>
) : items.length === 0 ? (
 <div className="text-center py-16">
 <p className="text-3xl mb-2">Ø=ÜË</p>
 <p className="text-text-muted">No transactions found</p>
 <p className="text-xs text-text-muted mt-1">
 {search || moduleFilter ? 'Try different filters' : 'Start adding
records from the home screen'}
 </p>
 </div>
) : (
 <div className="space-y-5 animate-fade-in stagger-3">
 {sortedDates.map((date) => (
 <div key={date}>
 <h2 className="text-xs font-semibold text-text-muted uppercase
tracking-wider mb-2 px-1">
 {formatGroupDate(date)}
 </h2>
 <div className="space-y-2">
 {grouped[date].map((item) => (
 <TransactionCard

```



```

 key={`${item.module}-${item.id}`}
 icon={item.icon}
 title={item.title}
 subtitle={item.subtitle}
 amount={item.amount}
 date={item.date}
 module={item.module}
 isIncome={item.module === 'money_in'}
 />
))}
 </div>
</div>
))}
</div>
)}
</div>
);
}

function formatGroupDate(dateStr: string): string {
 const date = new Date(dateStr + 'T00:00:00');
 const today = new Date();
 today.setHours(0, 0, 0, 0);
 const yesterday = new Date(today);
 yesterday.setDate(yesterday.getDate() - 1);

 if (date.getTime() === today.getTime()) return 'Today';
 if (date.getTime() === yesterday.getTime()) return 'Yesterday';

 return date.toLocaleDateString('en-IN', { day: 'numeric', month: 'short',
 year: 'numeric' });
}

...

`src\app\food\add\FoodForm.tsx`

```tsx
'use client';

import { useState } from 'react';
import { useRouter } from 'next/navigation';
import { createFoodRecord } from '@actions/food';
import { SearchableSelect } from '@components/SearchableSelect';
import { PAYMENT_METHOD_LABELS } from '@lib/constants';

export function FoodForm({
  initialCategories,
  initialShops,
  initialProviders,
  accounts
}: {
  initialCategories: any[];
  initialShops: any[];
  initialProviders: any[];
  accounts: any[];
}) {
  const router = useRouter();
  const [saving, setSaving] = useState(false);

  const [categoryId, setCategoryId] = useState('');

```

```

const [shopId, setShopId] = useState('');
const [date, setDate] = useState(new Date().toLocaleDateString('en-CA'));
const [amount, setAmount] = useState('');
const [paymentMethod, setPaymentMethod] = useState('cash');
const [cashProviderId, setCashProviderId] = useState('');
const [accountId, setAccountId] = useState(accounts.length > 0 ?
(accounts.find((a: any) => a.is_default)?.id || accounts[0].id) : '');
const [comments, setComments] = useState('');

async function handleSubmit(e: React.FormEvent) {
  e.preventDefault();
  setSaving(true);
  try {
    await createFoodRecord({
      food_category_id: categoryId || undefined,
      shop_id: shopId || undefined,
      start_date: date,
      end_date: date,
      amount: amount ? Number(amount) : undefined,
      payment_method: paymentMethod as any,
      cash_provider_id: cashProviderId || undefined,
      comments: comments || undefined,
      account_id: accountId || undefined,
    } as any);
    router.push('/');
  } catch (err) {
    console.error(err);
    alert('Failed to save food record');
  } finally {
    setSaving(false);
  }
}

const categoryOptions = initialCategories.map(c => ({ id: c.id, label:
c.name }));
const shopOptions = initialShops.map(s => ({ id: s.id, label: s.name }));
const providerOptions = initialProviders.map(p => ({ id: p.id, label:
p.name }));
const accountOptions = accounts.map((a: any) => ({ id: a.id, label: a.name,
subLabel: a.type.toUpperCase() }));

return (
  <div className="px-4 pt-6 pb-24 max-w-lg mx-auto animate-fade-in">
    <div className="flex items-center mb-6">
      <button onClick={() => router.back()} className="mr-3 text-2xl text-
text-secondary active:scale-90 transition-transform">
        !•
      </button>
      <h1 className="text-2xl font-bold">Add Food / Groceries</h1>
    </div>

    <form onSubmit={handleSubmit} className="space-y-4">
      <div className="bg-bg-card border border-border rounded-2xl p-5
shadow-sm space-y-4">
        <SearchableSelect
          label="Category"
          options={categoryOptions}
          value={categoryId}
          onChange={setCategoryId}
          placeholder="Ø=Ý Select Category (e.g. Groceries)"

```

```

/>

<SearchableSelect
  label="Shop"
  options={shopOptions}
  value={shopId}
  onChange={setShopId}
  placeholder="Ø=Ÿ Select Shop"
/>

<div>
  <label className="block text-xs font-semibold text-text-muted
uppercase tracking-wider mb-2">
    Date
  </label>
  <input
    type="date"
    required
    value={date}
    onChange={(e) => setDate(e.target.value)}
    className="w-full bg-bg border border-border rounded-xl px-4
py-3 text-text focus:outline-none focus:border-accent transition-colors"
  />
</div>

<div>
  <label className="block text-xs font-semibold text-text-muted
uppercase tracking-wider mb-2">
    Amount ( ¹)
  </label>
  <input
    type="number"
    value={amount}
    onChange={(e) => setAmount(e.target.value)}
    className="w-full bg-bg border border-border rounded-xl px-4
py-3 text-text focus:outline-none focus:border-accent transition-colors text-
xl font-bold"
    placeholder="0.00"
  />
</div>

</div>

<div className="bg-bg-card border border-border rounded-2xl p-5
shadow-sm space-y-4">

  <SearchableSelect
    label="Paid From Account"
    options={accountOptions}
    value={accountId}
    onChange={setAccountId}
    placeholder="Ø=Ÿ Select Account"
  />

  <SearchableSelect
    label="Cash Given By"
    options={providerOptions}
    value={cashProviderId}
    onChange={setCashProviderId}
    placeholder="Ø=Ÿ Search person"
  />

```

```

        <div>
          <label className="block text-xs font-semibold text-text-muted
uppercase tracking-wider mb-2">
            Payment Method
          </label>
          <select
            value={paymentMethod}
            onChange={e => setPaymentMethod(e.target.value)}
            className="w-full bg-bg border border-border rounded-xl px-4
py-3 text-text focus:outline-none focus:border-accent appearance-none"
          >
            {Object.entries(PAYMENT_METHOD_LABELS).map(([key, label]) => (
              <option key={key} value={key}>{label}</option>
            ))}
          </select>
        </div>

        <div>
          <label className="block text-xs font-semibold text-text-muted
uppercase tracking-wider mb-2">
            Comments
          </label>
          <input
            type="text"
            value={comments}
            onChange={(e) => setComments(e.target.value)}
            className="w-full bg-bg border border-border rounded-xl px-4
py-3 text-text focus:outline-none focus:border-accent"
            placeholder="Optional comments"
          />
        </div>
      </div>

      <button
        type="submit"
        disabled={saving || !categoryId}
        className="block w-full py-4 bg-accent text-white text-center text-
lg font-bold rounded-2xl shadow-glow-accent hover:brightness-110 transition-
all active:scale-[0.98] disabled:opacity-50 mt-4"
      >
        {saving ? 'Saving...' : 'SAVE FOOD EXPENSE'}
      </button>
    </form>
  </div>
);
}
...

## `src\app\food\add\loading.tsx`

```tsx
import { SkeletonFormPage } from '@components/Skeletons';

export default function FoodAddLoading() {
 return <SkeletonFormPage />;
}
...

```

```
`src\app\food\add\page.tsx`
```

```
````tsx
import { getParties } from '@actions/parties';
import { getFoodCategories } from '@actions/master';
import { getAccounts } from '@actions/accounts';
import { FoodForm } from './FoodForm';

export default async function AddFoodPage() {
  const [categories, parties, accs] = await Promise.all([
    getFoodCategories(),
    getParties(),
    getAccounts(),
  ]);

  const shops = parties.filter(p => p.class === 'shop' || p.class ===
'supplier');
  const cashProviders = parties.filter(p => p.class === 'person');

  return (
    <FoodForm
      initialCategories={categories}
      initialShops={shops}
      initialProviders={cashProviders}
      accounts={accs}
    />
  );
}
```

```
## `src\app\food\loading.tsx`
```

```
````tsx
import { SkeletonListPage } from '@components/Skeletons';

export default function FoodLoading() {
 return <SkeletonListPage />;
}
```

```
`src\app\food\page.tsx`
```

```
````tsx
import Link from 'next/link';
import { getFoodRecords } from '@actions/food';
import { deleteRecord } from '@actions/transactions';
import { TransactionCard } from '@components/TransactionCard';

export default async function FoodPage() {
  const records = await getFoodRecords();

  return (
    <main className="p-4 max-w-lg mx-auto pb-24">
      <div className="flex justify-between items-center mb-6">
        <h1 className="text-2xl font-semibold">Food Records</h1>
      </div>

      <div className="space-y-4">
        {records.map((r) => (
```

```

        <TransactionCard
            key={r.id}
            title={r.food_category?.name || 'Food'}
            subtitle={`For ${r.worker?.name || r.worker_type?.name || 'Site'}`}
from ${r.shop?.name || 'Unknown'}`}
            amount={r.amount}
            date={r.start_date}
            module="food"
            icon="🍽️"
            onDelete={async () => {
                'use server';
                await deleteRecord('food_records', r.id);
            }}
        />
    ))}

    {records.length === 0 && (
        <div className="text-center text-text-muted py-8">
            No food records found.
        </div>
    )}
</div>

    { /* Floating Add Button */ }
    <Link
        href="/food/add"
        className="fixed bottom-20 right-4 w-14 h-14 bg-accent text-white
rounded-full flex items-center justify-center shadow-lg hover:brightness-110
transition-all z-50"
    >
        <svg xmlns="http://www.w3.org/2000/svg" className="h-8 w-8"
fill="none" viewBox="0 0 24 24" stroke="currentColor">
            <path strokeLinecap="round" strokeLinejoin="round" strokeWidth={2}
d="M12 4v16m8-8H4" />
        </svg>
    </Link>
</main>
    );
}
...

```

```
## `src\app\globals.css`
```

```
```css
```

```
@import "tailwindcss";
```

```
/* =====
SHAMBALA – Design System
Mobile-first construction site expense manager
===== */
```

```
@theme {
 /* Core palette – warm, earthy tones for construction context */
 --color-bg-primary: #0a0a0f;
 --color-bg-secondary: #12121a;
 --color-bg-card: #1a1a25;
 --color-bg-elevated: #222233;
 --color-bg-input: #1e1e2d;

 --color-accent: #6c5ce7;

```

```

--color-accent-light: #a29bfe;
--color-accent-glow: rgba(108, 92, 231, 0.15);

--color-green: #00b894;
--color-green-light: #55efc4;
--color-green-glow: rgba(0, 184, 148, 0.15);

--color-red: #e17055;
--color-red-light: #fab1a0;

--color-amber: #fdbc6e;
--color-amber-dark: #e17055;

--color-text-primary: #f1f1f5;
--color-text-secondary: #8e8ea0;
--color-text-muted: #5a5a6e;

--color-border: #2a2a3a;
--color-border-light: #333348;

/* Shadows */
--shadow-card: 0 2px 8px rgba(0, 0, 0, 0.3);
--shadow-elevated: 0 8px 32px rgba(0, 0, 0, 0.4);
--shadow-glow-accent: 0 0 20px rgba(108, 92, 231, 0.3);
--shadow-glow-green: 0 0 20px rgba(0, 184, 148, 0.3);
}

/* =====
BASE
===== */

html {
 -webkit-tap-highlight-color: transparent;
 -webkit-text-size-adjust: 100%;
}

body {
 background: var(--color-bg-primary);
 color: var(--color-text-primary);
 font-family: 'Inter', system-ui, -apple-system, sans-serif;
 overflow-x: hidden;
}

/* =====
SMOOTH SCROLLING & SCROLLBAR
===== */

* {
 scrollbar-width: thin;
 scrollbar-color: var(--color-border) transparent;
}

*::-webkit-scrollbar {
 width: 4px;
}

*::-webkit-scrollbar-track {
 background: transparent;
}

*::-webkit-scrollbar-thumb {
 background: var(--color-border);
 border-radius: 999px;
}

```

```

}

/* =====
 ANIMATIONS
 ===== */

@keyframes fade-in {
 from { opacity: 0; transform: translateY(8px); }
 to { opacity: 1; transform: translateY(0); }
}

@keyframes slide-up {
 from { opacity: 0; transform: translateY(100%); }
 to { opacity: 1; transform: translateY(0); }
}

@keyframes pulse-subtle {
 0%, 100% { opacity: 1; }
 50% { opacity: 0.7; }
}

@keyframes scale-in {
 from { opacity: 0; transform: scale(0.95); }
 to { opacity: 1; transform: scale(1); }
}

.animate-fade-in {
 animation: fade-in 0.3s ease-out;
}

.animate-slide-up {
 animation: slide-up 0.3s cubic-bezier(0.16, 1, 0.3, 1);
}

.animate-scale-in {
 animation: scale-in 0.2s ease-out;
}

.animate-pulse-subtle {
 animation: pulse-subtle 2s ease-in-out infinite;
}

/* Staggered animation delays for list items */
.stagger-1 { animation-delay: 0.05s; animation-fill-mode: backwards; }
.stagger-2 { animation-delay: 0.1s; animation-fill-mode: backwards; }
.stagger-3 { animation-delay: 0.15s; animation-fill-mode: backwards; }
.stagger-4 { animation-delay: 0.2s; animation-fill-mode: backwards; }

/* =====
 GLASSMORPHISM
 ===== */

.glass {
 background: rgba(26, 26, 37, 0.8);
 backdrop-filter: blur(12px);
 -webkit-backdrop-filter: blur(12px);
 border: 1px solid var(--color-border);
}

.glass-elevated {
 background: rgba(34, 34, 51, 0.85);

```



```

 backdrop-filter: blur(16px);
 -webkit-backdrop-filter: blur(16px);
 border: 1px solid var(--color-border-light);
}

/* =====
 MOBILE NUMERIC INPUT – remove spin buttons
 ===== */

input[type="number"]::-webkit-inner-spin-button,
input[type="number"]::-webkit-outer-spin-button {
 -webkit-appearance: none;
 margin: 0;
}
input[type="number"] {
 -moz-appearance: textfield;
}

...

`src\app\import\page.tsx`

```tsx
'use client';

import { useState, useCallback } from 'react';
import { useRouter } from 'next/navigation';
import { parseWorkbook, type ParseResult, type ParsedRow } from '@lib/excel-parser';
import { formatCurrency } from '@lib/utils';

export default function ImportPage() {
    const router = useRouter();
    const [parsing, setParsing] = useState(false);
    const [result, setResult] = useState<ParseResult | null>(null);
    const [error, setError] = useState('');
    const [tab, setTab] = useState<'auto' | 'review' | 'rejected'>('auto');
    const [rows, setRows] = useState<ParsedRow[]>([]);
    const [importing, setImporting] = useState(false);
    const [importDone, setImportDone] = useState(false);

    const handleFile = useCallback(async (file: File) => {
        setParsing(true);
        setError('');
        try {
            const buffer = await file.arrayBuffer();
            const parsed = parseWorkbook(buffer);
            setResult(parsed);
            setRows(parsed.rows);
        } catch (err) {
            setError(err instanceof Error ? err.message : 'Failed to parse file');
        } finally {
            setParsing(false);
        }
    }, []);

    const handleDrop = useCallback((e: React.DragEvent) => {
        e.preventDefault();
        const file = e.dataTransfer.files[0];
        if (file) handleFile(file);
    }, [handleFile]);

```

```

    const handleFileInput = useCallback((e:
React.ChangeEvent<HTMLInputElement>) => {
    const file = e.target.files?.[0];
    if (file) handleFile(file);
  }, [handleFile]);

    const updateRowStatus = (index: number, status: ParsedRow['status']) => {
      setRows(prev => {
        const updated = [...prev];
        updated[index] = { ...updated[index], status };
        return updated;
      });
    };

    const filteredRows = rows.filter(r => r.status === tab);

    const handleImport = async () => {
      setImporting(true);
      try {
        // Import all auto + manually approved rows
        const toImport = rows.filter(r => r.status === 'auto');

        // Use the import API
        const response = await fetch('/api/import', {
          method: 'POST',
          headers: { 'Content-Type': 'application/json' },
          body: JSON.stringify({ rows: toImport }),
        });

        if (!response.ok) {
          throw new Error(await response.text());
        }

        setImportDone(true);
      } catch (err) {
        setError(err instanceof Error ? err.message : 'Import failed');
      } finally {
        setImporting(false);
      }
    };

    if (importDone) {
      const importedCount = rows.filter(r => r.status === 'auto').length;
      return (
        <div className="flex flex-col items-center justify-center min-h-[60vh]
px-4 animate-scale-in">
          <div className="w-20 h-20 rounded-full bg-green-glow flex items-
center justify-center text-4xl mb-4">
            |
          </div>
          <h2 className="text-xl font-bold text-green">Import Complete!</h2>
          <p className="text-text-secondary text-sm mt-2">{importedCount}
transactions imported</p>
          <button
            onClick={() => { router.push('/expenses'); router.refresh(); }}
            className="mt-6 px-8 py-3 bg-accent text-white rounded-2xl font-
semibold active:scale-95 transition-transform"
          >
            View Expenses
          </button>
        </div>
      );
    }
  );
}

```

```

    </div>
  );
}

// =====
// UPLOAD SCREEN
// =====
if (!result) {
  return (
    <div className="px-4 pt-6 max-w-lg mx-auto">
      <h1 className="text-xl font-bold mb-2">Import Excel</h1>
      <p className="text-text-secondary text-sm mb-6">
        Upload your EXPENDITURE 2025.xlsx file. The system will analyze and
        classify each transaction.
      </p>

      <div
        onDragOver={e => e.preventDefault()}
        onDrop={handleDrop}
        className="border-2 border-dashed border-border rounded-2xl p-12
        text-center hover:border-accent transition-colors"
      >
        {parsing ? (
          <div className="animate-pulse-subtle">
            <div className="text-4xl mb-3">&™p </div>
            <p className="text-text-secondary text-sm">Analyzing
            workbook...</p>
            <p className="text-text-muted text-xs mt-1">Detecting sheets,
            dates, amounts, categories</p>
          </div>
        ) : (
          <>
            <div className="text-4xl mb-3">Ø=Üå</div>
            <p className="text-text-secondary text-sm mb-4">Drag & drop
            your Excel file here</p>
            <label className="inline-block px-6 py-3 bg-accent text-white
            rounded-xl font-semibold cursor-pointer active:scale-95 transition-transform">
              Choose File
              <input
                type="file"
                accept=".xlsx,.xls"
                onChange={handleFileInput}
                className="hidden"
              />
            </label>
          </>
        )}
      </div>

      {error && (
        <div className="mt-4 bg-red/10 border border-red/30 rounded-xl px-4
        py-3 text-sm text-red">
          {error}
        </div>
      )}
    </div>
  );
}

// =====
// REVIEW SCREEN

```

```

// =====
const autoCount = rows.filter(r => r.status === 'auto').length;
const reviewCount = rows.filter(r => r.status === 'review').length;
const rejectedCount = rows.filter(r => r.status === 'rejected').length;

return (
  <div className="px-4 pt-6 max-w-lg mx-auto pb-24">
    <h1 className="text-xl font-bold mb-2">Review Import</h1>
    <p className="text-text-secondary text-sm mb-6">
      Found {result.total} potential transactions
    </p>

    {/* Summary cards */}
    <div className="grid grid-cols-3 gap-2 mb-6">
      <div className="bg-green-glow border border-green/20 rounded-xl p-3
text-center">
        <p className="text-2xl font-bold text-green">{autoCount}</p>
        <p className="text-[10px] text-text-muted uppercase">Ready</p>
      </div>
      <div className="bg-amber/10 border border-amber/20 rounded-xl p-3
text-center">
        <p className="text-2xl font-bold text-amber">{reviewCount}</p>
        <p className="text-[10px] text-text-muted uppercase">Review</p>
      </div>
      <div className="bg-red/10 border border-red/20 rounded-xl p-3 text-
center">
        <p className="text-2xl font-bold text-red">{rejectedCount}</p>
        <p className="text-[10px] text-text-muted uppercase">Rejected</p>
      </div>
    </div>

    {/* Tab switcher */}
    <div className="flex gap-2 mb-4">
      {[
        { key: 'auto' as const, label: 'Ready', count: autoCount },
        { key: 'review' as const, label: 'Review', count: reviewCount },
        { key: 'rejected' as const, label: 'Rejected', count:
rejectedCount },
      ]}.map(t => (
        <button
          key={t.key}
          onClick={() => setTab(t.key)}
          className={`flex-1 py-2.5 rounded-xl text-xs font-semibold
transition-all ${
            tab === t.key
              ? 'bg-accent text-white'
              : 'bg-bg-card border border-border text-text-secondary'
          }`}
        >
          {t.label} ({t.count})
        </button>
      )))
    </div>

    {/* Row list */}
    <div className="space-y-2 mb-6">
      {filteredRows.length === 0 ? (
        <div className="text-center py-8">
          <p className="text-text-muted text-sm">No rows in this category</p>
        </div>
      )
    </div>
  </div>
)

```

```

) : (
  filteredRows.map((row, idx) => {
    const globalIdx = rows.indexOf(row);
    return (
      <div key={`_${row.sheetName}-${row.rowNumber}`} className="bg-bg-
card border border-border rounded-2xl p-4 animate-fade-in">
        <div className="flex items-start justify-between gap-2 mb-2">
          <div className="flex-1 min-w-0">
            <p className="text-sm font-medium truncate">
              {row.parsedData.description || '-'}
            </p>
            <div className="flex items-center gap-2 mt-1 flex-wrap">
              {row.parsedData.date && (
                <span className="text-[10px] text-text-muted bg-bg-
elevated px-2 py-0.5 rounded">
                  {row.parsedData.date}
                </span>
              )}
              {row.parsedData.category && (
                <span className="text-[10px] text-accent bg-accent-
glow px-2 py-0.5 rounded">
                  {row.parsedData.category}
                </span>
              )}
              {row.parsedData.party && (
                <span className="text-[10px] text-text-secondary bg-
bg-elevated px-2 py-0.5 rounded">
                  {row.parsedData.party}
                </span>
              )}
            </div>
            <div>
              <span className="text-sm font-bold whitespace-nowrap">
                {row.parsedData.amount ?
formatCurrency(row.parsedData.amount) : '-'}
              </span>
            </div>
          </div>
          <p>
            {/* Reason */}
            <p className="text-[10px] text-text-muted mb-2">{row.reason}</
p>

            {/* Source info */}
            <p className="text-[10px] text-text-muted mb-2">
              Sheet: {row.sheetName} · Row {row.rowNumber} · Confidence:
{Math.round(row.confidence * 100)}%
            </p>

            {/* Actions for review items */}
            {tab === 'review' && (
              <div className="flex gap-2 mt-2">
                <button
                  onClick={() => updateRowStatus(globalIdx, 'auto')}
                  className="flex-1 py-1.5 text-xs font-semibold bg-
green/10 text-green border border-green/20 rounded-lg active:scale-95
transition-transform"
                >
                  ' Approve
                </button>
                <button
                  onClick={() => updateRowStatus(globalIdx, 'rejected')}

```

```

        className="flex-1 py-1.5 text-xs font-semibold bg-
red/10 text-red border border-red/20 rounded-lg active:scale-95 transition-
transform"
        >
        ' Reject
        </button>
    </div>
    })

    {tab === 'rejected' && (
        <button
            onClick={() => updateRowStatus(globalIdx, 'auto')}
            className="w-full py-1.5 text-xs font-semibold bg-
green/10 text-green border border-green/20 rounded-lg active:scale-95
transition-transform mt-2"
            >
            ' Approve Anyway
            </button>
        )}
    </div>
    );
    })
    })
</div>

{error && (
    <div className="mb-4 bg-red/10 border border-red/30 rounded-xl px-4
py-3 text-sm text-red">
        {error}
    </div>
)}

{/* Import button – fixed at bottom */}
<div className="fixed bottom-20 left-0 right-0 px-4 pb-4">
    <div className="max-w-lg mx-auto">
        <button
            onClick={handleImport}
            disabled={importing || autoCount === 0}
            className={`w-full py-4 rounded-2xl text-base font-bold
transition-all active:scale-[0.98] ${
                importing || autoCount === 0
                ? 'bg-bg-elevated text-text-muted cursor-not-allowed'
                : 'bg-green text-white shadow-glow-green hover:brightness-110'
            }`}
            >
            {importing ? (
                <span className="animate-pulse-subtle">Importing {autoCount}
transactions...</span>
            ) : (
                `Import ${autoCount} Transactions`
            )}
        </button>
    </div>
</div>
</div>
);
}
...

## `src\app\labour\add\LabourForm.tsx`

```

```

````tsx
'use client';

import { useState, useEffect } from 'react';
import { useRouter } from 'next/navigation';
import { createLabourRecord } from '@actions/labour';
import { createParty } from '@actions/parties';
import { SearchableSelect } from '@components/SearchableSelect';
import { PAYMENT_METHOD_LABELS } from '@lib/constants';

export function LabourForm({
 initialWorkers,
 initialProviders,
 initialBuildings,
 initialWorkTypes,
 accounts
}: {
 initialWorkers: any[];
 initialProviders: any[];
 initialBuildings: any[];
 initialWorkTypes: any[];
 accounts: any[];
}) {
 const router = useRouter();
 const [saving, setSaving] = useState(false);

 // Master Data
 const [workers, setWorkers] = useState<any[]>(initialWorkers);
 const [cashProviders, setCashProviders] = useState<any[]>(initialProviders);

 // Form State
 const [workerId, setWorkerId] = useState('');
 const [workTypeId, setWorkTypeId] = useState('');
 const [buildingId, setBuildingId] = useState('');
 const [date, setDate] = useState(new Date().toLocaleDateString('en-CA'));
 const [amount, setAmount] = useState('');
 const [paymentMethod, setPaymentMethod] = useState('cash');
 const [cashProviderId, setCashProviderId] = useState('');
 const [accountId, setAccountId] = useState(accounts.length > 0 ?
(accounts.find((a: any) => a.is_default)?.id || accounts[0].id) : '');
 const [comments, setComments] = useState('');
 const [showMore, setShowMore] = useState(false);

 // Form persistence
 useEffect(() => {
 try {
 const draft = sessionStorage.getItem('draft_labour');
 if (draft) {
 const parsed = JSON.parse(draft);
 if (parsed.workerId) setWorkerId(parsed.workerId);
 if (parsed.workTypeId) setWorkTypeId(parsed.workTypeId);
 if (parsed.buildingId) setBuildingId(parsed.buildingId);
 if (parsed.amount) setAmount(parsed.amount);
 if (parsed.comments) setComments(parsed.comments);
 if (parsed.accountId) setAccountId(parsed.accountId);
 }
 } catch (e) {}
 }, []);

 useEffect(() => {

```

```

 sessionStorage.setItem('draft_labour', JSON.stringify({
 workerId, workTypeId, buildingId, amount, comments, accountId
 }));
 }, [workerId, workTypeId, buildingId, amount, comments, accountId]);

 async function handleAddWorker(name: string) {
 if (!name.trim()) return;
 setSaving(true);
 try {
 const party = await createParty({ name, class: 'person', role:
'labour' });
 setWorkers(prev => [...prev, party]);
 setWorkerId(party.id);
 } catch (err) {
 alert('Failed to add worker');
 } finally {
 setSaving(false);
 }
 }

 async function handleAddCashProvider(name: string) {
 if (!name.trim()) return;
 setSaving(true);
 try {
 const party = await createParty({ name, class: 'person', role:
'investor' });
 setCashProviders(prev => [...prev, party]);
 setCashProviderId(party.id);
 } catch (err) {
 alert('Failed to add cash provider');
 } finally {
 setSaving(false);
 }
 }

 async function handleSubmit(e: React.FormEvent) {
 e.preventDefault();
 setSaving(true);
 try {
 await createLabourRecord({
 worker_id: workerId,
 work_type_id: workTypeId || undefined,
 building_id: buildingId || undefined,
 date: date,
 amount: amount ? Number(amount) : undefined,
 payment_method: paymentMethod as any,
 cash_provider_id: cashProviderId || undefined,
 comments: comments || undefined,
 account_id: accountId || undefined,
 });
 sessionStorage.removeItem('draft_labour');
 router.push('/');
 } catch (err) {
 console.error(err);
 alert('Failed to save labour record');
 } finally {
 setSaving(false);
 }
 }

 const workerOptions = workers.map(w => ({ id: w.id, label: w.name,

```



```

subLabel: w.phone }));
 const providerOptions = cashProviders.map(p => ({ id: p.id, label:
p.name }));
 const buildingOptions = initialBuildings.map(b => ({ id: b.id, label:
b.display_name }));
 const workTypeOptions = initialWorkTypes.map(w => ({ id: w.id, label:
w.name }));
 const accountOptions = accounts.map((a: any) => ({ id: a.id, label: a.name,
subLabel: a.type.toUpperCase() }));

 return (
 <div className="px-4 pt-6 pb-24 max-w-lg mx-auto animate-fade-in">
 <div className="flex items-center mb-6">
 <button onClick={() => router.back()} className="mr-3 text-2xl text-
text-secondary active:scale-90 transition-transform">
 !•
 </button>
 <h1 className="text-2xl font-bold">Add Labour</h1>
 </div>

 <form onSubmit={handleSubmit} className="space-y-4">

 <div className="bg-bg-card border border-border rounded-2xl p-5
shadow-sm space-y-4">
 <SearchableSelect
 label="Worker"
 options={workerOptions}
 value={workerId}
 onChange={setWorkerId}
 placeholder="Ø=Ý Search worker"
 onAddNew={(name) => handleAddWorker(name || 'New Worker')}
 addNewLabel="Add Worker"
 storageKey="workers"
 />

 <SearchableSelect
 label="Work Type"
 options={workTypeOptions}
 value={workTypeId}
 onChange={setWorkTypeId}
 placeholder="Ø=Ý Search work type"
 storageKey="workTypes"
 />

 <SearchableSelect
 label="Building / Location"
 options={buildingOptions}
 value={buildingId}
 onChange={setBuildingId}
 placeholder="Ø=Ý Search building"
 storageKey="buildings"
 />

 <div>
 <label className="block text-xs font-semibold text-text-muted
uppercase tracking-wider mb-2">
 Date
 </label>
 <input
 type="date"
 required

```

```

 value={date}
 onChange={(e) => setDate(e.target.value)}
 className="w-full bg-bg border border-border rounded-xl px-4
py-3 text-text focus:outline-none focus:border-accent transition-colors"
 />
 </div>

 <div>
 <label className="block text-xs font-semibold text-text-muted
uppercase tracking-wider mb-2">
 Amount (1)
 </label>
 <input
 type="number"
 value={amount}
 onChange={(e) => setAmount(e.target.value)}
 className="w-full bg-bg border border-border rounded-xl px-4
py-3 text-text focus:outline-none focus:border-accent transition-colors text-
xl font-bold"
 placeholder="Amount paid"
 />
 </div>

 </div>

 {!showMore ? (
 <button
 type="button"
 onClick={() => setShowMore(true)}
 className="w-full py-3 text-sm font-semibold text-accent"
 >
 + More Details (Account, Method)
 </button>
) : (
 <div className="bg-bg-card border border-border rounded-2xl p-5
shadow-sm space-y-4 animate-fade-in">

 <SearchableSelect
 label="Paid From Account"
 options={accountOptions}
 value={accountId}
 onChange={setAccountId}
 placeholder="Ø=Ý Select Account"
 />

 <div>
 <label className="block text-xs font-semibold text-text-muted
uppercase tracking-wider mb-2">
 Payment Method
 </label>
 <select
 value={paymentMethod}
 onChange={e => setPaymentMethod(e.target.value)}
 className="w-full bg-bg border border-border rounded-xl px-4
py-3 text-text focus:outline-none focus:border-accent appearance-none"
 >
 {Object.entries(PAYMENT_METHOD_LABELS).map(([key, label]) => (
 <option key={key} value={key}>{label}</option>
))}
 </select>
 </div>
 </div>
)
}

```

```

 <SearchableSelect
 label="Cash Given By"
 options={providerOptions}
 value={cashProviderId}
 onChange={setCashProviderId}
 placeholder="Ø=Ý Search person"
 onAddNew={(name) => handleAddCashProvider(name || 'New Person')}
 addNewLabel="Add Person"
 />

 <div>
 <label className="block text-xs font-semibold text-text-muted
uppercase tracking-wider mb-2">
 Comments
 </label>
 <input
 type="text"
 value={comments}
 onChange={(e) => setComments(e.target.value)}
 className="w-full bg-bg border border rounded-xl px-4
py-3 text-text focus:outline-none focus:border-accent"
 placeholder="Optional comments"
 />
 </div>

 <button
 type="button"
 onClick={() => setShowMore(false)}
 className="w-full pt-2 text-sm font-semibold text-text-muted
text-center"
 >
 Hide Details
 </button>
 </div>
)}

 <button
 type="submit"
 disabled={saving || !workerId}
 className="block w-full py-4 bg-accent text-white text-center text-
lg font-bold rounded-2xl shadow-glow-accent hover:brightness-110 transition-
all active:scale-[0.98] disabled:opacity-50 mt-4"
 >
 {saving ? 'Saving...' : 'SAVE LABOUR'}
 </button>
 </form>
</div>
);
}
...

`src\app\labour\add\loading.tsx`

```tsx
import { SkeletonFormPage } from '@components/Skeletons';

export default function LabourAddLoading() {
  return <SkeletonFormPage />;
}

```

...

`src\app\labour\add\page.tsx`

```
````tsx
import { getParties } from '@actions/parties';
import { getBuildings, getWorkTypes } from '@actions/master';
import { getAccounts } from '@actions/accounts';
import { LabourForm } from './LabourForm';

export default async function AddLabourPage() {
 const [labourParties, investorParties, bldgs, works, accs] = await
Promise.all([
 getParties(undefined, 'labour'),
 getParties(undefined, 'investor'),
 getBuildings(),
 getWorkTypes(),
 getAccounts(),
]);

 return (
 <LabourForm
 initialWorkers={labourParties}
 initialProviders={investorParties}
 initialBuildings={bldgs}
 initialWorkTypes={works}
 accounts={accs}
 />
);
}
```

...

## `src\app\labour\loading.tsx`

```
````tsx
import { SkeletonListPage } from '@components/Skeletons';

export default function LabourLoading() {
  return <SkeletonListPage />;
}
```

...

`src\app\labour\page.tsx`

```
````tsx
import Link from 'next/link';
import { getLabourRecords } from '@actions/labour';
import { deleteRecord } from '@actions/transactions';
import { TransactionCard } from '@components/TransactionCard';

export default async function LabourPage() {
 const records = await getLabourRecords();

 return (
 <main className="p-4 max-w-lg mx-auto pb-24">
 <div className="flex justify-between items-center mb-6">
 <h1 className="text-2xl font-semibold">Labour Records</h1>

```

```

</div>

<div className="space-y-4">
 {records.map((r) => (
 <TransactionCard
 key={r.id}
 title={r.worker?.name || 'Unknown Worker'}
 subtitle={` ${r.work_type?.name || 'Work'} at
${r.building?.display_name || 'Site'} `}
 amount={r.amount}
 date={r.date}
 module="labour"
 icon="👷"
 onDelete={async () => {
 'use server';
 await deleteRecord('labour_records', r.id);
 }}
 />
))}

 {records.length === 0 && (
 <div className="text-center text-text-muted py-8">
 No labour records found.
 </div>
)}
</div>

{/* Floating Add Button */}
<Link
 href="/labour/add"
 className="fixed bottom-20 right-4 w-14 h-14 bg-accent text-white
rounded-full flex items-center justify-center shadow-lg hover:brightness-110
transition-all z-50"
>
 <svg xmlns="http://www.w3.org/2000/svg" className="h-8 w-8"
fill="none" viewBox="0 0 24 24" stroke="currentColor">
 <path strokeLinecap="round" strokeLinejoin="round" strokeWidth={2}
d="M12 4v16m8-8H4" />
 </svg>
</Link>
</main>
);
}

```

```

...

`src\app\layout.tsx`

```

```

````tsx
import type { Metadata, Viewport } from "next";
import { Inter } from "next/font/google";
import "@/globals.css";
import { Navbar } from "@/components/Navbar";
import { APP_NAME } from "@/lib/constants";
import { ToastProvider } from "@/components/Toast";

const inter = Inter({ subsets: ["latin"], variable: '--font-inter' });

export const metadata: Metadata = {
  title: `${APP_NAME} - Site Expenses`,
  description: "Manage construction site expenses, labour, and materials.",

```

```

    manifest: '/manifest.json',
    appleWebApp: {
      capable: true,
      statusBarStyle: 'default',
      title: APP_NAME,
    },
    formatDetection: {
      telephone: false,
    },
  };

export const viewport: Viewport = {
  width: "device-width",
  initialScale: 1,
  maximumScale: 1,
  userScalable: false,
  themeColor: "#0a0a0f",
};

export default function RootLayout({ children }: LayoutProps<"/">) {
  return (
    <html lang="en" className={`${inter.variable} h-full antialiased`} >
      <body className={`${inter.variable} font-sans bg-bg text-text-primary
antialiased min-h-screen selection:bg-accent/20 selection:text-accent-light`} >
        <ToastProvider>
          {children}
          <Navbar />
        </ToastProvider>
      </body>
    </html>
  );
}

...

## `src\app\loading.tsx`

```tsx
import { SkeletonHomePage } from '@components/Skeletons';

export default function HomeLoading() {
 return <SkeletonHomePage />;
}

...

`src\app\login\page.tsx`

```tsx
'use client';

import { useState } from 'react';
import { useRouter } from 'next/navigation';
import { createClient } from '@lib/supabase/client';

export default function LoginPage() {
  const router = useRouter();
  const [email, setEmail] = useState('');
  const [password, setPassword] = useState('');
  const [error, setError] = useState<string | null>(null);
  const [loading, setLoading] = useState(false);

```

```

const supabase = createClient();

async function handleLogin(e: React.FormEvent) {
  e.preventDefault();
  setLoading(true);
  setError(null);

  const { error } = await supabase.auth.signInWithPassword({
    email,
    password,
  });

  if (error) {
    setError(error.message);
    setLoading(false);
  } else {
    router.push('/');
    router.refresh();
  }
}

return (
  <div className="min-h-screen flex items-center justify-center p-4 bg-bg">
    <div className="w-full max-w-sm bg-bg-card border border-border
rounded-3xl p-6 shadow-2xl animate-fade-in">
      <div className="text-center mb-8">
        <h1 className="text-3xl font-bold mb-2">Shambala</h1>
        <p className="text-text-muted text-sm">Sign in to manage your
project</p>
      </div>

      <form onSubmit={handleLogin} className="space-y-4">
        {error && (
          <div className="p-3 bg-red-light/10 border border-red-light/20
text-red-light text-sm rounded-xl">
            {error}
          </div>
        )}

        <div>
          <label className="block text-xs font-semibold text-text-muted
uppercase tracking-wider mb-2">
            Email
          </label>
          <input
            type="email"
            required
            value={email}
            onChange={(e) => setEmail(e.target.value)}
            className="w-full bg-bg border border-border rounded-xl px-4
py-3 text-text focus:outline-none focus:border-accent transition-colors"
            placeholder="admin@example.com"
          />
        </div>

        <div>
          <label className="block text-xs font-semibold text-text-muted
uppercase tracking-wider mb-2">
            Password
          </label>

```

```

        <input
          type="password"
          required
          value={password}
          onChange={(e) => setPassword(e.target.value)}
          className="w-full bg-bg border border-border rounded-xl px-4
py-3 text-text focus:outline-none focus:border-accent transition-colors"
          placeholder="....."
        />
      </div>

      <button
        type="submit"
        disabled={loading}
        className="w-full py-3.5 bg-accent text-white font-bold rounded-
xl active:scale-[0.98] transition-transform shadow-glow-accent mt-4
disabled:opacity-50"
      >
        {loading ? 'Signing in...' : 'Sign In'}
      </button>
    </form>
  </div>
</div>
);
}
...

```

```
## `src\app\machinery\add\loading.tsx`
```

```

````tsx
import { SkeletonFormPage } from '@components/Skeletons';

export default function MachineryAddLoading() {
 return <SkeletonFormPage />;
}
...

```

```
`src\app\machinery\add\MachineryForm.tsx`
```

```

````tsx
'use client';

import { useState, useEffect } from 'react';
import { useRouter } from 'next/navigation';
import { createMachineryRecord } from '@actions/machinery';
import { SearchableSelect } from '@components/SearchableSelect';
import { PAYMENT_METHOD_LABELS } from '@lib/constants';

export function MachineryForm({
  initialMachines,
  initialFuelTypes,
  initialSuppliers,
  initialProviders,
  accounts
}): {
  initialMachines: any[];
  initialFuelTypes: any[];
  initialSuppliers: any[];
  initialProviders: any[];

```



```

    accounts: any[];
  }) {
    const router = useRouter();
    const [saving, setSaving] = useState(false);

    // Machinery State
    const [machineryId, setMachineryId] = useState('');
    const [date, setDate] = useState(new Date().toLocaleDateString('en-CA'));
    const [startMeter, setStartMeter] = useState('');
    const [endMeter, setEndMeter] = useState('');
    const [totalHours, setTotalHours] = useState('');
    const [amount, setAmount] = useState('');
    const [paymentMethod, setPaymentMethod] = useState('cash');
    const [cashProviderId, setCashProviderId] = useState('');
    const [accountId, setAccountId] = useState(accounts.length > 0 ?
(accounts.find((a: any) => a.is_default)?.id || accounts[0].id) : '');
    const [comments, setComments] = useState('');

    // Fuel State
    const [showFuel, setShowFuel] = useState(false);
    const [fuelTypeId, setFuelTypeId] = useState('');
    const [quantity, setQuantity] = useState('');
    const [fuelCost, setFuelCost] = useState('');
    const [supplierId, setSupplierId] = useState('');

    // Auto-calculate hours if meters change
    useEffect(() => {
      if (startMeter && endMeter) {
        const diff = Number(endMeter) - Number(startMeter);
        if (diff > 0) setTotalHours(diff.toString());
      }
    }, [startMeter, endMeter]);

    async function handleSubmit(e: React.FormEvent) {
      e.preventDefault();
      setSaving(true);
      try {
        await createMachineryRecord(
          {
            machine_id: machineryId || undefined,
            date,
            start_time: startMeter || undefined,
            end_time: endMeter || undefined,
            hours: totalHours ? Number(totalHours) : undefined,
            amount: amount ? Number(amount) : undefined,
            payment_method: paymentMethod as any,
            cash_provider_id: cashProviderId || undefined,
            comments: comments || undefined,
            account_id: accountId || undefined,
          } as any,
          showFuel ? {
            fuel_type: fuelTypeId || undefined,
            quantity: quantity ? Number(quantity) : undefined,
            unit: 'Liters',
            amount: fuelCost ? Number(fuelCost) : undefined,
            provider_id: supplierId || undefined,
            account_id: accountId || undefined,
          } as any : undefined
        );
        router.push('/');
      } catch (err) {

```

```

        console.error(err);
        alert('Failed to save machinery record');
    } finally {
        setSaving(false);
    }
}

const machineOptions = initialMachines.map(m => ({ id: m.id, label: m.name,
subLabel: m.type }));
const fuelOptions = initialFuelTypes.map(f => ({ id: f.id, label:
f.name }));
const supplierOptions = initialSuppliers.map(s => ({ id: s.id, label:
s.name }));
const providerOptions = initialProviders.map(p => ({ id: p.id, label:
p.name }));
const accountOptions = accounts.map((a: any) => ({ id: a.id, label: a.name,
subLabel: a.type.toUpperCase() }));

return (
    <div className="px-4 pt-6 pb-24 max-w-lg mx-auto animate-fade-in">
        <div className="flex items-center mb-6">
            <button onClick={() => router.back()} className="mr-3 text-2xl text-
text-secondary active:scale-90 transition-transform">
                !•
            </button>
            <h1 className="text-2xl font-bold">Machinery & Fuel</h1>
        </div>

        <form onSubmit={handleSubmit} className="space-y-4">
            <div className="bg-bg-card border border-border rounded-2xl p-5
shadow-sm space-y-4">

                <SearchableSelect
                    label="Machine"
                    options={machineOptions}
                    value={machineryId}
                    onChange={setMachineryId}
                    placeholder="Ø=Ý Select Machine (e.g. JCB-01)"
                />

                <div>
                    <label className="block text-xs font-semibold text-text-muted
uppercase tracking-wider mb-2">
                        Date
                    </label>
                    <input
                        type="date"
                        required
                        value={date}
                        onChange={(e) => setDate(e.target.value)}
                        className="w-full bg-bg border border-border rounded-xl px-4
py-3 text-text focus:outline-none focus:border-accent"
                    />
                </div>

                <div className="flex gap-3">
                    <div className="flex-1">
                        <label className="block text-xs font-semibold text-text-muted
uppercase tracking-wider mb-2">
                            Start Meter
                        </label>

```

```

        <input
            type="number"
            value={startMeter}
            onChange={e => setStartMeter(e.target.value)}
            className="w-full bg-bg border border-border rounded-xl px-4
py-3 text-text outline-none focus:border-accent"
            placeholder="0.0"
        />
    </div>
    <div className="flex-1">
        <label className="block text-xs font-semibold text-text-muted
uppercase tracking-wider mb-2">
            End Meter
        </label>
        <input
            type="number"
            value={endMeter}
            onChange={e => setEndMeter(e.target.value)}
            className="w-full bg-bg border border-border rounded-xl px-4
py-3 text-text outline-none focus:border-accent"
            placeholder="0.0"
        />
    </div>
</div>

<div>
    <label className="block text-xs font-semibold text-text-muted
uppercase tracking-wider mb-2">
        Total Hours
    </label>
    <input
        type="number"
        value={totalHours}
        onChange={e => setTotalHours(e.target.value)}
        className="w-full bg-bg-elevated border border-border rounded-
xl px-4 py-3 text-accent font-bold outline-none"
        placeholder="0"
    />
</div>

<div>
    <label className="block text-xs font-semibold text-text-muted
uppercase tracking-wider mb-2">
        Machine Rent/Cost ( 1)
    </label>
    <input
        type="number"
        value={amount}
        onChange={(e) => setAmount(e.target.value)}
        className="w-full bg-bg border border-border rounded-xl px-4
py-3 text-text focus:outline-none focus:border-accent text-xl font-bold"
        placeholder="0.00"
    />
</div>
</div>

{ /* Fuel Link */ }
<div className="bg-bg-card border border-border rounded-2xl p-5
shadow-sm">
    { !showFuel ? (
        <button

```

```

        type="button"
        onClick={() => setShowFuel(true)}
        className="w-full text-accent font-semibold flex items-center
justify-center gap-2"
      >
        <span className="text-xl">&ý</span> Add Fuel Record
      </button>
    ) : (
      <div className="space-y-4 animate-fade-in">
        <div className="flex justify-between items-center mb-2">
          <h3 className="font-bold text-text flex items-center
gap-2"><span className="text-xl">&ý</span> Fuel Entry</h3>
          <button type="button" onClick={() => setShowFuel(false)}
className="text-text-muted text-sm">Remove</button>
        </div>

        <SearchableSelect
          label="Fuel Type"
          options={fuelOptions}
          value={fuelTypeId}
          onChange={setFuelTypeId}
          placeholder="Ø=Ý Select Fuel"
        />

        <div className="flex gap-3">
          <div className="flex-1">
            <label className="block text-xs font-semibold text-text-
muted uppercase tracking-wider mb-2">
              Quantity (Liters)
            </label>
            <input
              type="number"
              value={quantity}
              onChange={e => setQuantity(e.target.value)}
              className="w-full bg-bg border border-border rounded-xl
px-4 py-3 text-text outline-none focus:border-accent"
              placeholder="0"
            />
          </div>
          <div className="flex-1">
            <label className="block text-xs font-semibold text-text-
muted uppercase tracking-wider mb-2">
              Fuel Cost
            </label>
            <input
              type="number"
              value={fuelCost}
              onChange={e => setFuelCost(e.target.value)}
              className="w-full bg-bg border border-border rounded-xl
px-4 py-3 text-text outline-none focus:border-accent font-bold text-accent"
              placeholder="¹0.00"
            />
          </div>
        </div>

        <SearchableSelect
          label="Fuel Station/Supplier"
          options={supplierOptions}
          value={supplierId}
          onChange={setSupplierId}
          placeholder="Ø=Ý Select Supplier"
        />
      </div>
    )
  }
}

```

```

    />
  </div>
)}
</div>

<div className="bg-bg-card border border-border rounded-2xl p-5
shadow-sm space-y-4">

  <SearchableSelect
    label="Paid From Account"
    options={accountOptions}
    value={accountId}
    onChange={setAccountId}
    placeholder="Ø=Ý Select Account"
  />

  <SearchableSelect
    label="Cash Given By"
    options={providerOptions}
    value={cashProviderId}
    onChange={setCashProviderId}
    placeholder="Ø=Ý Search person"
  />

  <div className="flex gap-3">
    <div className="flex-1">
      <label className="block text-xs font-semibold text-text-muted
uppercase tracking-wider mb-2">
        Payment Method
      </label>
      <select
        value={paymentMethod}
        onChange={e => setPaymentMethod(e.target.value)}
        className="w-full bg-bg border border-border rounded-xl px-4
py-3 text-text focus:outline-none focus:border-accent appearance-none"
      >
        {Object.entries(PAYMENT_METHOD_LABELS).map(([key, label]) => (
          <option key={key} value={key}>{label}</option>
        ))}
      </select>
    </div>
  </div>

  <div>
    <label className="block text-xs font-semibold text-text-muted
uppercase tracking-wider mb-2">
      Comments
    </label>
    <input
      type="text"
      value={comments}
      onChange={(e) => setComments(e.target.value)}
      className="w-full bg-bg border border-border rounded-xl px-4
py-3 text-text focus:outline-none focus:border-accent"
      placeholder="Optional comments"
    />
  </div>
</div>

<button
  type="submit"

```

```

        disabled={saving || !machineryId}
        className="block w-full py-4 bg-accent text-white text-center text-
lg font-bold rounded-2xl shadow-glow-accent hover:brightness-110 transition-
all active:scale-[0.98] disabled:opacity-50 mt-4"
      >
        {saving ? 'Saving...' : 'SAVE MACHINERY RECORD'}
      </button>
    </form>
  </div>
);
}
...

```

```

## `src\app\machinery\add\page.tsx`

```

```

```tsx
import { getParties } from '@actions/parties';
import { getMachinery, getFuelTypes } from '@actions/master';
import { getAccounts } from '@actions/accounts';
import { MachineryForm } from './MachineryForm';

export default async function AddMachineryPage() {
 const [mach, fuels, parties, accs] = await Promise.all([
 getMachinery(),
 getFuelTypes(),
 getParties(),
 getAccounts(),
]);

 const suppliers = parties.filter(p => p.class === 'shop' || p.class ===
'supplier');
 const cashProviders = parties.filter(p => p.class === 'person');

 return (
 <MachineryForm
 initialMachines={mach}
 initialFuelTypes={fuels}
 initialSuppliers={suppliers}
 initialProviders={cashProviders}
 accounts={accs}
 />
);
}
...

```

```

`src\app\machinery\loading.tsx`

```

```

```tsx
import { SkeletonListPage } from '@components/Skeletons';

export default function MachineryLoading() {
  return <SkeletonListPage />;
}
...

```

```

## `src\app\machinery\page.tsx`

```

```

```tsx

```

```

import Link from 'next/link';
import { getMachineryRecords } from '@actions/machinery';
import { deleteRecord } from '@actions/transactions';
import { TransactionCard } from '@components/TransactionCard';

export default async function MachineryPage() {
 const records = await getMachineryRecords();

 return (
 <main className="p-4 max-w-lg mx-auto pb-24">
 <div className="flex justify-between items-center mb-6">
 <h1 className="text-2xl font-semibold">Machinery Records</h1>
 </div>

 <div className="space-y-4">
 {records.map((r) => (
 <TransactionCard
 key={r.id}
 title={` ${r.machine?.machine_type} (${r.machine?.machine_id}) `}
 subtitle={`Op: ${r.operator?.name || 'Unknown'} - ${r.hours ?
r.hours + ' hrs' : 'Flat'} `}
 amount={r.amount}
 date={r.date}
 module="machinery"
 icon="ðŸšª"
 onDelete={async () => {
 'use server';
 await deleteRecord('machinery_records', r.id);
 }}
 </>
))}

 {records.length === 0 && (
 <div className="text-center text-text-muted py-8">
 No machinery records found.
 </div>
)}
 </div>

 { /* Floating Add Button */ }
 <Link
 href="/machinery/add"
 className="fixed bottom-20 right-4 w-14 h-14 bg-accent text-white
rounded-full flex items-center justify-center shadow-lg hover:brightness-110
transition-all z-50"
 >
 <svg xmlns="http://www.w3.org/2000/svg" className="h-8 w-8"
fill="none" viewBox="0 0 24 24" stroke="currentColor">
 <path strokeLinecap="round" strokeLinejoin="round" strokeWidth={2}
d="M12 4v16m8-8H4" />
 </svg>
 </Link>
 </main>
);
}

...

`src\app\materials\add\loading.tsx`

````tsx

```

```

import { SkeletonFormPage } from '@components/Skeletons';

export default function MaterialsAddLoading() {
  return <SkeletonFormPage />;
}

...

## `src\app\materials\add\MaterialsForm.tsx`

```tsx
'use client';

import { useState, useEffect } from 'react';
import { useRouter } from 'next/navigation';
import { getMaterialVariants, createMaterialDelivery } from '@actions/materials';
import { SearchableSelect } from '@components/SearchableSelect';

export function MaterialsForm({
 initialMaterials,
 initialSuppliers,
 initialBuildings,
 initialVehicles,
 initialWeighbridges,
 accounts
}): {
 initialMaterials: any[];
 initialSuppliers: any[];
 initialBuildings: any[];
 initialVehicles: any[];
 initialWeighbridges: any[];
 accounts: any[];
} {
 const router = useRouter();
 const [saving, setSaving] = useState(false);

 const [variants, setVariants] = useState<any[]>([]);

 // Delivery State
 const [materialId, setMaterialId] = useState('');
 const [variantId, setVariantId] = useState('');
 const [supplierId, setSupplierId] = useState('');
 const [buildingId, setBuildingId] = useState('');
 const [date, setDate] = useState(new Date().toLocaleDateString('en-CA'));
 const [quantity, setQuantity] = useState('');
 const [unit, setUnit] = useState('Tonnes');
 const [materialCost, setMaterialCost] = useState('');

 // Transport State
 const [showTransport, setShowTransport] = useState(false);
 const [vehicleId, setVehicleId] = useState('');
 const [transportCost, setTransportCost] = useState('');

 // Weighbridge State
 const [showWeighbridge, setShowWeighbridge] = useState(false);
 const [weighbridgeId, setWeighbridgeId] = useState('');
 const [grossWeight, setGrossWeight] = useState('');
 const [tareWeight, setTareWeight] = useState('');
 const [weighbridgeFee, setWeighbridgeFee] = useState('');

```



```

// Account State
const [accountId, setAccountId] = useState(accounts.length > 0 ?
(accounts.find((a: any) => a.is_default)?.id || accounts[0].id) : '');

// Load variants when material changes
useEffect(() => {
 if (materialId) {
 getMaterialVariants(materialId).then(setVariants).catch(console.error);
 const mat = initialMaterials.find(m => m.id === materialId);
 if (mat?.default_unit) setUnit(mat.default_unit);
 } else {
 setVariants([]);
 }
}, [materialId, initialMaterials]);

const netWeight = (Number(grossWeight) || 0) - (Number(tareWeight) || 0);
const landedCost = (Number(materialCost) || 0) + (Number(transportCost) ||
0) + (Number(weighbridgeFee) || 0);

async function handleSubmit(e: React.FormEvent) {
 e.preventDefault();
 setSaving(true);
 try {
 await createMaterialDelivery(
 {
 material_id: materialId,
 variant_id: variantId || undefined,
 supplier_id: supplierId || undefined,
 building_id: buildingId || undefined,
 date,
 quantity: quantity ? Number(quantity) : undefined,
 unit,
 material_cost: materialCost ? Number(materialCost) : undefined,
 },
 showTransport ? {
 vehicle_id: vehicleId || undefined,
 amount: transportCost ? Number(transportCost) : undefined,
 account_id: accountId || undefined,
 } : undefined,
 showWeighbridge ? {
 weighbridge_id: weighbridgeId || undefined,
 gross_weight: grossWeight ? Number(grossWeight) : undefined,
 tare_weight: tareWeight ? Number(tareWeight) : undefined,
 fee: weighbridgeFee ? Number(weighbridgeFee) : undefined,
 account_id: accountId || undefined,
 } : undefined,
 accountId || undefined
);
 router.push('/');
 } catch (err) {
 alert('Failed to save material delivery');
 console.error(err);
 } finally {
 setSaving(false);
 }
}

const materialOptions = initialMaterials.map(m => ({ id: m.id, label:
m.name }));
const variantOptions = variants.map(v => ({ id: v.id, label: v.name }));
const supplierOptions = initialSuppliers.map(s => ({ id: s.id, label:
s.name }));

```

```

 const buildingOptions = initialBuildings.map(b => ({ id: b.id, label:
b.display_name }));
 const vehicleOptions = initialVehicles.map(v => ({ id: v.id, label:
v.vehicle_number, subLabel: v.vehicle_type }));
 const weighbridgeOptions = initialWeighbridges.map(w => ({ id: w.id, label:
w.name }));
 const accountOptions = accounts.map((a: any) => ({ id: a.id, label: a.name,
subLabel: a.type.toUpperCase() }));

 return (
 <div className="px-4 pt-6 pb-24 max-w-lg mx-auto animate-fade-in">
 <div className="flex items-center mb-6">
 <button onClick={() => router.back()} className="mr-3 text-2xl text-
text-secondary active:scale-90 transition-transform">
 !•
 </button>
 <h1 className="text-2xl font-bold">Material Delivery</h1>
 </div>

 <form onSubmit={handleSubmit} className="space-y-4">

 {/* Core Material Details */}
 <div className="bg-bg-card border border-border rounded-2xl p-5
shadow-sm space-y-4">
 <SearchableSelect
 label="Material"
 options={materialOptions}
 value={materialId}
 onChange={setMaterialId}
 placeholder="Ø=Ý Select Material"
 />

 {variants.length > 0 && (
 <SearchableSelect
 label="Size / Variant"
 options={variantOptions}
 value={variantId}
 onChange={setVariantId}
 placeholder="Ø=Ý Select Size"
 />
)}

 <SearchableSelect
 label="Supplier"
 options={supplierOptions}
 value={supplierId}
 onChange={setSupplierId}
 placeholder="Ø=Ý Select Supplier"
 />

 <SearchableSelect
 label="Destination"
 options={buildingOptions}
 value={buildingId}
 onChange={setBuildingId}
 placeholder="Ø=Ý Select Building"
 />

 <div className="flex gap-3">
 <div className="flex-1">
 <label className="block text-xs font-semibold text-text-muted

```

```

 uppercase tracking-wider mb-2">
 Quantity/Weight
 </label>
 <input
 type="number"
 value={quantity}
 onChange={e => setQuantity(e.target.value)}
 className="w-full bg-bg border border-border rounded-xl px-4
py-3 text-text outline-none focus:border-accent"
 placeholder="0"
 />
</div>
<div className="flex-1">
 <label className="block text-xs font-semibold text-text-muted
uppercase tracking-wider mb-2">
 Unit
 </label>
 <input
 type="text"
 value={unit}
 onChange={e => setUnit(e.target.value)}
 className="w-full bg-bg border border-border rounded-xl px-4
py-3 text-text outline-none focus:border-accent"
 />
</div>
</div>

<div>
 <label className="block text-xs font-semibold text-text-muted
uppercase tracking-wider mb-2">
 Material Cost (Optional)
 </label>
 <input
 type="number"
 value={materialCost}
 onChange={e => setMaterialCost(e.target.value)}
 className="w-full bg-bg border border-border rounded-xl px-4
py-3 text-text outline-none focus:border-accent text-xl font-bold"
 placeholder=" 10.00"
 />
</div>
</div>

{/* Transport Link */}
<div className="bg-bg-card border border-border rounded-2xl p-5
shadow-sm">
 {!showTransport ? (
 <button
 type="button"
 onClick={() => setShowTransport(true)}
 className="w-full text-accent font-semibold flex items-center
justify-center gap-2"
 >
 + Add Transport Record
 </button>
) : (
 <div className="space-y-4 animate-fade-in">
 <div className="flex justify-between items-center mb-2">
 <h3 className="font-bold text-text">Transport Details</h3>
 <button type="button" onClick={() => setShowTransport(false)}
className="text-text-muted text-sm">Remove</button>

```

```

 </div>
 <SearchableSelect
 label="Vehicle"
 options={vehicleOptions}
 value={vehicleId}
 onChange={setVehicleId}
 placeholder="Ø=Ÿ Select Vehicle"
 />
 <div>
 <label className="block text-xs font-semibold text-text-muted
uppercase tracking-wider mb-2">
 Transport Cost
 </label>
 <input
 type="number"
 value={transportCost}
 onChange={e => setTransportCost(e.target.value)}
 className="w-full bg-bg border border-border rounded-xl
px-4 py-3 text-text outline-none focus:border-accent"
 placeholder=" 10.00"
 />
 </div>
 </div>
)}
</div>

{/* Weighbridge Link */}
<div className="bg-bg-card border border-border rounded-2xl p-5
shadow-sm">
 {!showWeighbridge ? (
 <button
 type="button"
 onClick={() => setShowWeighbridge(true)}
 className="w-full text-accent font-semibold flex items-center
justify-center gap-2"
 >
 + Add Weighbridge Record
 </button>
) : (
 <div className="space-y-4 animate-fade-in">
 <div className="flex justify-between items-center mb-2">
 <h3 className="font-bold text-text">Weighbridge Details</h3>
 <button type="button" onClick={() =>
setShowWeighbridge(false)} className="text-text-muted text-sm">Remove</button>
 </div>

 <SearchableSelect
 label="Weighbridge Location"
 options={weighbridgeOptions}
 value={weighbridgeId}
 onChange={setWeighbridgeId}
 placeholder="Ø=Ÿ Select Weighbridge"
 />

 <div className="flex gap-3">
 <div className="flex-1">
 <label className="block text-[10px] font-semibold text-text-
muted uppercase tracking-wider mb-1">
 Gross
 </label>
 <input

```

```

 type="number"
 value={grossWeight}
 onChange={e => setGrossWeight(e.target.value)}
 className="w-full bg-bg border border-border rounded-lg
px-3 py-2 text-text outline-none focus:border-accent"
 />
 </div>
 <div className="flex-1">
 <label className="block text-[10px] font-semibold text-text-
muted uppercase tracking-wider mb-1">
 Tare
 </label>
 <input
 type="number"
 value={tareWeight}
 onChange={e => setTareWeight(e.target.value)}
 className="w-full bg-bg border border-border rounded-lg
px-3 py-2 text-text outline-none focus:border-accent"
 />
 </div>
 <div className="flex-1 bg-bg-elevated rounded-lg p-2 flex
flex-col justify-center items-center">
 <span className="text-[10px] text-text-muted uppercase font-
semibold">Net
 {netWeight > 0 ?
netWeight : 0}
 </div>
 </div>

 <div>
 <label className="block text-xs font-semibold text-text-muted
uppercase tracking-wider mb-2">
 Weighbridge Fee
 </label>
 <input
 type="number"
 value={weighbridgeFee}
 onChange={e => setWeighbridgeFee(e.target.value)}
 className="w-full bg-bg border border-border rounded-xl
px-4 py-3 text-text outline-none focus:border-accent"
 placeholder=" 10.00"
 />
 </div>
</div>
 </div>
)}
</div>

<div className="bg-bg-card border border-border rounded-2xl p-5
shadow-sm space-y-4">
 <SearchableSelect
 label="Paid From Account"
 options={accountOptions}
 value={accountId}
 onChange={setAccountId}
 placeholder="ðŸ’ Select Account"
 />
</div>

 { /* Cost Summary */ }
 <div className="bg-accent/10 border border-accent/20 rounded-2xl p-5
shadow-sm flex justify-between items-center">

```

```

 <div>
 <p className="text-xs font-semibold text-accent uppercase
tracking-wider">Total Landed Cost</p>
 {quantity && netWeight > 0 && (
 <p className="text-xs text-text-muted mt-0.5"> 1{(landedCost /
netWeight).toFixed(2)} / unit</p>
)}
 </div>
 <div className="text-2xl font-bold text-accent">
1{landedCost.toFixed(2)}
 </div>
 </div>

 <button
 type="submit"
 disabled={saving || !materialId}
 className="block w-full py-4 bg-accent text-white text-center text-
lg font-bold rounded-2xl shadow-glow-accent hover:brightness-110 transition-
all active:scale-[0.98] disabled:opacity-50 mt-4"
 >
 {saving ? 'Saving...' : 'SAVE DELIVERY'}
 </button>
 </form>
 </div>
);
}
...

```

```

`src\app\materials\add\page.tsx`

```

```

````tsx
import { getMaterials } from '@actions/materials';
import { getParties } from '@actions/parties';
import { getBuildings, getTransportVehicles, getWeighbridges } from '@actions/master';
import { getAccounts } from '@actions/accounts';
import { MaterialsForm } from './MaterialsForm';

export default async function AddMaterialPage() {
  const [mats, parts, bldgs, vehs, wbs, accs] = await Promise.all([
    getMaterials(),
    getParties(),
    getBuildings(),
    getTransportVehicles(),
    getWeighbridges(),
    getAccounts(),
  ]);

  const suppliers = parts.filter(p => p.class === 'supplier' || p.class === 'shop');

  return (
    <MaterialsForm
      initialMaterials={mats}
      initialSuppliers={suppliers}
      initialBuildings={bldgs}
      initialVehicles={vehs}
      initialWeighbridges={wbs}
      accounts={accs}
    />
  );
}

```

```

    });
}
...

## `src\app\materials\loading.tsx`

```tsx
import { SkeletonListPage } from '@components/Skeletons';

export default function MaterialsLoading() {
 return <SkeletonListPage />;
}
...

`src\app\materials\page.tsx`

```tsx
import Link from 'next/link';
import { getMaterialDeliveries } from '@actions/materials';
import { deleteRecord } from '@actions/transactions';
import { TransactionCard } from '@components/TransactionCard';

export default async function MaterialsPage() {
  const records = await getMaterialDeliveries();

  return (
    <main className="p-4 max-w-lg mx-auto pb-24">
      <div className="flex justify-between items-center mb-6">
        <h1 className="text-2xl font-semibold">Materials & Deliveries</h1>
      </div>

      <div className="space-y-4">
        {records.map((r) => (
          <TransactionCard
            key={r.id}
            title={` ${r.material?.name} ${r.variant?.name ?
              ` ($ {r.variant.name}) ` : ' ' }`
            subtitle={` ${r.quantity} ${r.unit} from ${r.supplier?.name ||
              'Unknown'} `}
            amount={r.material_cost}
            date={r.date}
            module="materials"
            icon="ðŸŒˆ"
            onDelete={async () => {
              'use server';
              await deleteRecord('material_deliveries', r.id);
            }}
          />
        ))}

        {records.length === 0 && (
          <div className="text-center text-text-muted py-8">
            No material deliveries found.
          </div>
        )}
      </div>

      { /* Floating Add Button */ }
      <Link

```

```

        href="/materials/add"
        className="fixed bottom-20 right-4 w-14 h-14 bg-accent text-white
rounded-full flex items-center justify-center shadow-lg hover:brightness-110
transition-all z-50"
      >
        <svg xmlns="http://www.w3.org/2000/svg" className="h-8 w-8"
fill="none" viewBox="0 0 24 24" stroke="currentColor">
          <path strokeLinecap="round" strokeLinejoin="round" strokeWidth={2}
d="M12 4v16m8-8H4" />
        </svg>
      </Link>
    </main>
  );
}

```

```

...

```

```

## `src\app\money\add\loading.tsx`

```

```

````tsx
import { SkeletonFormPage } from '@components/Skeletons';

export default function MoneyAddLoading() {
 return <SkeletonFormPage />;
}
...

```

```

`src\app\money\add\MoneyInForm.tsx`

```

```

````tsx
'use client';

import { useState } from 'react';
import { useRouter } from 'next/navigation';
import { addMoneyIn } from '@actions/money';
import { createParty } from '@actions/parties';
import { SearchableSelect } from '@components/SearchableSelect';

export function MoneyInForm({ initialAccounts, initialPeople }:
{ initialAccounts: any[], initialPeople: any[] }) {
  const router = useRouter();
  const [saving, setSaving] = useState(false);

  const [accounts, setAccounts] = useState<any[]>(initialAccounts);
  const [people, setPeople] = useState<any[]>(initialPeople);

  const [amount, setAmount] = useState('');
  const [accountId, setAccountId] = useState(initialAccounts.length > 0 ?
(initialAccounts.find((a: any) => a.is_default)?.id ||
initialAccounts[0].id) : '');
  const [partyId, setPartyId] = useState('');
  const [date, setDate] = useState(new Date().toLocaleDateString('en-CA'));
  const [description, setDescription] = useState('');

  async function handleSubmit(e: React.FormEvent) {
    e.preventDefault();
    if (!amount || !accountId) return;
    setSaving(true);
    try {
      await addMoneyIn({

```



```

        amount: Number(amount),
        account_id: accountId,
        party_id: partyId || undefined,
        date,
        description: description || undefined,
    });
    router.push('/money');
} catch (err) {
    console.error(err);
    alert('Failed to record money in');
} finally {
    setSaving(false);
}
}

async function handleAddPerson(name: string) {
    if (!name.trim()) return;
    setSaving(true);
    try {
        const newParty = await createParty({ name, class: 'person', role:
'investor' });
        setPeople([...people, newParty]);
        setPartyId(newParty.id);
    } catch (err) {
        console.error(err);
        alert('Failed to create person');
    } finally {
        setSaving(false);
    }
}

const accountOptions = accounts.map((a: any) => ({
    id: a.id,
    label: a.name,
    subLabel: a.type.toUpperCase(),
}));
const personOptions = people.map((p: any) => ({ id: p.id, label: p.name }));

return (
    <div className="px-4 pt-6 pb-24 max-w-lg mx-auto animate-fade-in">
        <div className="flex items-center mb-6">
            <button onClick={() => router.back()} className="mr-3 text-2xl text-
text-secondary active:scale-90 transition-transform">
                !•
            </button>
            <h1 className="text-2xl font-bold">Money In</h1>
        </div>

        <form onSubmit={handleSubmit} className="space-y-4">
            <div className="bg-bg-card border border-border rounded-2xl p-5
shadow-sm space-y-4">
                <div>
                    <label className="block text-xs font-semibold text-text-muted
uppercase tracking-wider mb-2">
                        Amount ( 1 )
                    </label>
                    <input
                        type="number"
                        required
                        value={amount}
                        onChange={(e) => setAmount(e.target.value)}

```

```

        className="w-full bg-bg border border-border rounded-xl px-4
py-4 text-green focus:outline-none focus:border-green transition-colors
text-3xl font-bold text-center"
        placeholder="0"
      />
    </div>

    <SearchableSelect
      label="To Account"
      options={accountOptions}
      value={accountId}
      onChange={setAccountId}
      placeholder="0=Ý Select Account"
    />

    <div>
      <label className="block text-xs font-semibold text-text-muted
uppercase tracking-wider mb-2">
        Date
      </label>
      <input
        type="date"
        required
        value={date}
        onChange={(e) => setDate(e.target.value)}
        className="w-full bg-bg border border-border rounded-xl px-4
py-3 text-text focus:outline-none focus:border-accent transition-colors"
      />
    </div>
  </div>

  <div className="bg-bg-card border border-border rounded-2xl p-5
shadow-sm space-y-4">
    <SearchableSelect
      label="Received From"
      options={personOptions}
      value={partyId}
      onChange={setPartyId}
      placeholder="0=Ý Search person (optional)"
      onAddNew={(name) => handleAddPerson(name || 'New Person')}
      addNewLabel="Add Person"
    />

    <div>
      <label className="block text-xs font-semibold text-text-muted
uppercase tracking-wider mb-2">
        Description
      </label>
      <input
        type="text"
        value={description}
        onChange={(e) => setDescription(e.target.value)}
        className="w-full bg-bg border border-border rounded-xl px-4
py-3 text-text focus:outline-none focus:border-accent"
        placeholder="e.g. Varun Sir Cash, Bank withdrawal"
      />
    </div>
  </div>

  <button
    type="submit"

```

```

        disabled={saving || !amount || !accountId}
        className="block w-full py-4 bg-green text-white text-center text-
lg font-bold rounded-2xl shadow-glow-green hover:brightness-110 transition-
all active:scale-[0.98] disabled:opacity-50 mt-4"
      >
        {saving ? 'Saving...' : 'RECORD MONEY IN'}
      </button>
    </form>
  </div>
);
}
...

## `src\app\money\add\page.tsx`

```tsx
import { getAccounts } from '@actions/accounts';
import { getParties } from '@actions/parties';
import { MoneyInForm } from './MoneyInForm';

export default async function MoneyInPage() {
 const [accounts, people] = await Promise.all([
 getAccounts(),
 getParties(undefined, 'investor'),
]);

 return <MoneyInForm initialAccounts={accounts} initialPeople={people} />;
}

...

`src\app\money\loading.tsx`

```tsx
'use client';

import { SkeletonCard, SkeletonLine } from '@components/Skeletons';

export default function MoneyLoading() {
  return (
    <div className="px-4 pt-6 pb-24 max-w-lg mx-auto animate-fade-in">
      <div className="flex items-center justify-between mb-6">
        <SkeletonLine width="w-40" height="h-7" />
        <div className="flex gap-2">
          <div className="w-24 h-8 bg-bg-elevated rounded-full animate-pulse-
subtle" />
          <div className="w-24 h-8 bg-bg-elevated rounded-full animate-pulse-
subtle" />
        </div>
      </div>

      {/* Total balance */}
      <SkeletonCard height="h-24" />

      {/* Accounts */}
      <div className="mt-4 space-y-2">
        <SkeletonLine width="w-20" height="h-3" />
        {[...Array(3)].map((_, i) => (
          <SkeletonCard key={i} height="h-[72px]" />
        ))}
      </div>
    </div>
  );
}

```

```

    ))}
  </div>

  { /* Recent */ }
  <div className="mt-6 space-y-2">
    <SkeletonLine width="w-32" height="h-3" />
    {[...Array(4)].map((_, i) => (
      <SkeletonCard key={i} height="h-[72px]" />
    ))}
  </div>
</div>
);
}
...

## `src\app\money\page.tsx`

```tsx
import Link from 'next/link';
import { getAccountBalances } from '@actions/accounts';
import { getRecentTransactions } from '@actions/money';
import { formatCurrency, formatDateShort } from '@lib/utils';

const TXN_TYPE_LABELS: Record<string, { label: string; icon: string; color:
string }> = {
 money_in: { label: 'Money In', icon: 'Ø=Üµ', color: 'text-green' },
 transfer: { label: 'Transfer', icon: '!"p ', color: 'text-accent-light' },
 operational_expense: { label: 'Expense', icon: 'Ø=Üä', color: 'text-red-
light' },
 general_expense: { label: 'Expense', icon: 'Ø=Üä', color: 'text-red-light' },
 supplier_payment: { label: 'Supplier', icon: 'Ø<ßê', color: 'text-red-light' },
 opening_balance: { label: 'Opening', icon: 'Ø=ÜÊ', color: 'text-accent-light' },
};

const ACCOUNT_TYPE_BADGES: Record<string, { bg: string; text: string }> = {
 cash: { bg: 'bg-green/10', text: 'text-green' },
 bank: { bg: 'bg-accent/10', text: 'text-accent-light' },
 upi: { bg: 'bg-amber/10', text: 'text-amber' },
 other: { bg: 'bg-bg-elevated', text: 'text-text-muted' },
};

export default async function MoneyPage() {
 const [balances, recentTxns] = await Promise.all([
 getAccountBalances(),
 getRecentTransactions(15),
]);

 const totalBalance = balances.reduce((sum, b) => sum + b.balance, 0);

 return (
 <div className="px-4 pt-6 pb-24 max-w-lg mx-auto">
 { /* Header */ }
 <div className="flex items-center justify-between mb-6 animate-fade-in">
 <h1 className="text-xl font-bold">Money & Accounts</h1>
 <div className="flex gap-2">
 <Link
 href="/money/add"
 className="px-3 py-1.5 rounded-full bg-green/10 text-green text-
xs font-bold border border-green/20"
 >

```

```

 + Money In
 </Link>
 <Link
 href="/money/transfer"
 className="px-3 py-1.5 rounded-full bg-accent/10 text-accent-
light text-xs font-bold border border-accent/20"
 >
 !"p Transfer
 </Link>
 </div>
 </div>

 { /* Total Balance */ }
 <div className="bg-bg-card border border-border rounded-2xl p-5 mb-4
animate-fade-in stagger-1">
 <p className="text-xs text-text-muted font-medium uppercase tracking-
wider">Total Balance</p>
 <p className="text-3xl font-bold text-text-primary
mt-1">{formatCurrency(totalBalance)}</p>
 </div>

 { /* Account Cards */ }
 <div className="space-y-2 mb-6 animate-fade-in stagger-2">
 <h2 className="text-xs font-semibold text-text-muted uppercase
tracking-wider mb-2">Accounts</h2>
 {balances.map((account) => {
 const badge = ACCOUNT_TYPE_BADGES[account.account_type] ||
ACCOUNT_TYPE_BADGES.other;
 return (
 <div key={account.account_id} className="bg-bg-card border border-
border rounded-2xl p-4 flex items-center gap-3">
 <div className={`w-10 h-10 rounded-xl ${badge.bg} flex items-
center justify-center shrink-0`}>

 {account.account_type === 'cash' ? '₹' :
account.account_type === 'bank' ? '₹' : account.account_type === 'upi' ? '₹' :
'₹'}

 </div>
 <div className="flex-1 min-w-0">
 <h3 className="text-sm font-semibold">{account.account_name}</
h3>
 <span className={`text-[10px] px-2 py-0.5 rounded-full
${badge.bg} ${badge.text} font-semibold uppercase`}>
 {account.account_type}

 </div>
 <p className={`text-lg font-bold ${account.balance >= 0 ? 'text-
green' : 'text-red-light'}`}>
 {formatCurrency(account.balance)}
 </p>
 </div>
);
 })}
 </div>

 { /* Recent Transactions */ }
 <div className="animate-fade-in stagger-3">
 <h2 className="text-xs font-semibold text-text-muted uppercase
tracking-wider mb-3">Recent Movements</h2>
 {recentTxns.length === 0 ? (

```

```

 <div className="text-center py-8">
 <p className="text-text-muted text-sm">No transactions yet</p>
 </div>
) : (
 <div className="space-y-2">
 {recentTxns.map((txn: any) => {
 const meta = TXN_TYPE_LABELS[txn.type] ||
TXN_TYPE_LABELS.opening_balance;
 const partyName = txn.party?.name || '';
 const isIncome = txn.type === 'money_in' || txn.type ===
'opening_balance';

 return (
 <div key={txn.id} className="bg-bg-card border border-border
rounded-2xl p-4 flex items-center gap-3">
 <div className="w-9 h-9 rounded-xl bg-bg-elevated flex
items-center justify-center text-base shrink-0">
 {meta.icon}
 </div>
 <div className="flex-1 min-w-0">
 <h3 className="text-sm font-semibold">{meta.label}</h3>
 <p className="text-xs text-text-muted
truncate">{txn.description || partyName}</p>
 </div>
 <div className="text-right shrink-0">
 <p className={`text-sm font-bold ${meta.color}`}>
 {isIncome ? '+' : '-'}{formatCurrency(txn.amount)}
 </p>
 <p className="text-[10px] text-text-
muted">{formatDateShort(txn.date)}</p>
 </div>
 </div>
);
 })});
 </div>
);
}
...

```

```

`src/app/money/transfer/page.tsx`

```

```

````tsx
'use client';

import { useState, useEffect } from 'react';
import { useRouter } from 'next/navigation';
import { createTransfer } from '@actions/money';
import { getAccounts } from '@actions/accounts';
import { SearchableSelect } from '@components/SearchableSelect';

export default function TransferPage() {
  const router = useRouter();
  const [loading, setLoading] = useState(true);
  const [saving, setSaving] = useState(false);

  const [accounts, setAccounts] = useState<any[]>([]);

```

```

const [amount, setAmount] = useState('');
const [fromAccountId, setFromAccountId] = useState('');
const [toAccountId, setToAccountId] = useState('');
const [date, setDate] = useState(new Date().toLocaleDateString('en-CA'));
const [description, setDescription] = useState('');

useEffect(() => {
  async function loadData() {
    try {
      const accs = await getAccounts();
      setAccounts(accs);
    } catch (err) {
      console.error('Failed to load accounts', err);
    } finally {
      setLoading(false);
    }
  }
  loadData();
}, []);

async function handleSubmit(e: React.FormEvent) {
  e.preventDefault();
  if (!amount || !fromAccountId || !toAccountId) return;
  if (fromAccountId === toAccountId) {
    alert('Please select different accounts');
    return;
  }
  setSaving(true);
  try {
    await createTransfer({
      amount: Number(amount),
      from_account_id: fromAccountId,
      to_account_id: toAccountId,
      date,
      description: description || undefined,
    });
    router.push('/money');
  } catch (err) {
    console.error(err);
    alert('Failed to create transfer');
  } finally {
    setSaving(false);
  }
}

if (loading) return <div className="p-8 text-center text-text-muted">Loading...</div>;

const accountOptions = accounts.map((a: any) => ({
  id: a.id,
  label: a.name,
  subLabel: a.type.toUpperCase(),
})));

// Filter out selected account from the other dropdown
const fromOptions = accountOptions.filter(a => a.id !== toAccountId);
const toOptions = accountOptions.filter(a => a.id !== fromAccountId);

return (
  <div className="px-4 pt-6 pb-24 max-w-lg mx-auto animate-fade-in">
    <div className="flex items-center mb-6">

```

```

        <button onClick={() => router.back()} className="mr-3 text-2xl text-
text-secondary active:scale-90 transition-transform">
        !•
    </button>
    <h1 className="text-2xl font-bold">Transfer</h1>
</div>

    <form onSubmit={handleSubmit} className="space-y-4">
        <div className="bg-bg-card border border-border rounded-2xl p-5
shadow-sm space-y-4">
            {/* Amount */}
            <div>
                <label className="block text-xs font-semibold text-text-muted
uppercase tracking-wider mb-2">
                    Amount ( 1)
                </label>
                <input
                    type="number"
                    required
                    value={amount}
                    onChange={(e) => setAmount(e.target.value)}
                    className="w-full bg-bg border border-border rounded-xl px-4
py-4 text-accent-light focus:outline-none focus:border-accent transition-
colors text-3xl font-bold text-center"
                    placeholder="0"
                />
            </div>

            {/* From Account */}
            <SearchableSelect
                label="From Account"
                options={fromOptions}
                value={fromAccountId}
                onChange={setFromAccountId}
                placeholder="Ø=Ý Select source account"
            />

            {/* Arrow */}
            <div className="flex justify-center">
                <div className="w-10 h-10 rounded-full bg-accent/10 border border-
accent/20 flex items-center justify-center text-accent text-xl">
                    !"
                </div>
            </div>

            {/* To Account */}
            <SearchableSelect
                label="To Account"
                options={toOptions}
                value={toAccountId}
                onChange={setToAccountId}
                placeholder="Ø=Ý Select destination account"
            />

            {/* Date */}
            <div>
                <label className="block text-xs font-semibold text-text-muted
uppercase tracking-wider mb-2">
                    Date
                </label>
                <input

```



```

        type="date"
        required
        value={date}
        onChange={(e) => setDate(e.target.value)}
        className="w-full bg-bg border border-border rounded-xl px-4
py-3 text-text focus:outline-none focus:border-accent transition-colors"
      />
    </div>

    {/* Description */}
    <div>
      <label className="block text-xs font-semibold text-text-muted
uppercase tracking-wider mb-2">
        Description
      </label>
      <input
        type="text"
        value={description}
        onChange={(e) => setDescription(e.target.value)}
        className="w-full bg-bg border border-border rounded-xl px-4
py-3 text-text focus:outline-none focus:border-accent"
        placeholder="e.g. Cash deposit to bank"
      />
    </div>
  </div>

  <button
    type="submit"
    disabled={saving || !amount || !fromAccountId || !toAccountId}
    className="block w-full py-4 bg-accent text-white text-center text-
lg font-bold rounded-2xl shadow-glow-accent hover:brightness-110 transition-
all active:scale-[0.98] disabled:opacity-50 mt-4"
  >
    {saving ? 'Transferring...' : 'CONFIRM TRANSFER'}
  </button>
</form>
</div>
);
}
...

```

```
## `src\app\more\accounts\page.tsx`
```

```

````tsx
'use client';

import { useState, useEffect } from 'react';
import { getAccounts } from '@actions/accounts';
import type { Account } from '@lib/types';

const TYPE_ICONS: Record<string, string> = {
 cash: 'ðŸµ',
 bank: 'ðŸæ',
 upi: 'ðŸŃ',
 other: 'ðŸ³',
};

export default function AccountsPage() {
 const [accounts, setAccounts] = useState<Account[]>([]);
 const [loading, setLoading] = useState(true);

```

```

useEffect(() => {
 getAccounts().then(a => { setAccounts(a); setLoading(false); });
}, []);

return (
 <div className="px-4 pt-6 max-w-lg mx-auto">
 <h1 className="text-xl font-bold mb-5">Accounts</h1>

 {loading ? (
 <div className="space-y-2">
 {[1, 2, 3].map(i => (
 <div key={i} className="bg-bg-card rounded-2xl p-4 animate-pulse-
subtle">
 <div className="w-24 h-4 bg-bg-elevated rounded" />
 </div>
))}
 </div>
) : (
 <div className="space-y-2">
 {accounts.map(acc => (
 <div key={acc.id} className="bg-bg-card border border-border
rounded-2xl p-4">
 <div className="flex items-center gap-3">
 <span className="text-xl w-10 h-10 flex items-center justify-
center bg-bg-elevated rounded-xl">
 {TYPE_ICONS[acc.type] || 'Ø=Ü³'}

 <div>
 <h3 className="text-sm font-semibold">{acc.name}</h3>
 <p className="text-xs text-text-muted mt-0.5 capitalize">
 {acc.type}{acc.is_default ? ' . Default' : ''}
 </p>
 </div>
 </div>
 </div>
))}
 </div>
)}
 </div>
);
}

```

```

`src\app\more\categories\page.tsx`

```

```

````tsx
'use client';

import { useState, useEffect } from 'react';
import { useRouter } from 'next/navigation';
import { getFoodCategories, getWorkTypes, getWorkerTypes } from '@actions/
master';
import { Modal } from '@components/Modal';
import { createClient } from '@lib/supabase/client';
import { DEFAULT_PROJECT_ID } from '@lib/constants';

```

```

type Tab = 'food' | 'work' | 'worker';

```

```

export default function CategoriesPage() {

```

```

const router = useRouter();
const [tab, setTab] = useState<Tab>('food');
const [loading, setLoading] = useState(true);

const [foodCategories, setFoodCategories] = useState<any[]>([]);
const [workTypes, setWorkTypes] = useState<any[]>([]);
const [workerTypes, setWorkerTypes] = useState<any[]>([]);

const [isModalOpen, setIsModalOpen] = useState(false);
const [newName, setNewName] = useState('');
const [saving, setSaving] = useState(false);

useEffect(() => {
  async function loadData() {
    try {
      const [food, work, worker] = await Promise.all([
        getFoodCategories(),
        getWorkTypes(),
        getWorkerTypes(),
      ]);
      setFoodCategories(food || []);
      setWorkTypes(work || []);
      setWorkerTypes(worker || []);
    } catch (err) {
      console.error('Failed to load categories', err);
    } finally {
      setLoading(false);
    }
  }
  loadData();
}, []);

async function handleAdd(e: React.FormEvent) {
  e.preventDefault();
  if (!newName.trim()) return;
  setSaving(true);

  try {
    const supabase = createClient();
    const tableName = tab === 'food' ? 'food_categories' : tab === 'work' ?
'work_types' : 'worker_types';

    const { data, error } = await supabase
      .from(tableName)
      .insert({ project_id: DEFAULT_PROJECT_ID, name: newName.trim() })
      .select()
      .single();

    if (error) throw error;

    if (tab === 'food') setFoodCategories(prev => [...prev, data]);
    else if (tab === 'work') setWorkTypes(prev => [...prev, data]);
    else setWorkerTypes(prev => [...prev, data]);

    setNewName('');
    setIsModalOpen(false);
  } catch (err) {
    alert('Failed to add item');
  } finally {
    setSaving(false);
  }
}

```

```

}

const TABS: { key: Tab; label: string; icon: string }[] = [
  { key: 'food', label: 'Food', icon: '🍔' },
  { key: 'work', label: 'Work Types', icon: '👤' },
  { key: 'worker', label: 'Worker Types', icon: '👷' },
];

const currentItems = tab === 'food' ? foodCategories : tab === 'work' ?
workTypes : workerTypes;
const addLabel = tab === 'food' ? 'Food Category' : tab === 'work' ? 'Work
Type' : 'Worker Type';

return (
  <div className="px-4 pt-6 pb-24 max-w-lg mx-auto">
    <div className="flex items-center mb-5 animate-fade-in">
      <button onClick={() => router.back()} className="mr-3 text-2xl text-
text-secondary active:scale-90 transition-transform">
        !•
      </button>
      <h1 className="text-xl font-bold">Categories</h1>
    </div>

    { /* Tabs */ }
    <div className="flex gap-2 mb-5 animate-fade-in stagger-1">
      {TABS.map((t) => (
        <button
          key={t.key}
          onClick={() => setTab(t.key)}
          className={`flex-1 py-2.5 rounded-xl text-xs font-semibold
transition-all ${
            tab === t.key
              ? 'bg-accent text-white shadow-glow-accent'
              : 'bg-bg-card text-text-secondary border border-border
hover:border-accent/30'
          }`}
        >
          {t.icon} {t.label}
        </button>
      ))}
    </div>

    { /* Items List */ }
    <div className="animate-fade-in stagger-2">
      {loading ? (
        <div className="space-y-2">
          {[...Array(4)].map((_, i) => (
            <div key={i} className="bg-bg-card border border-border rounded-
xl p-4 h-12 animate-pulse-subtle" />
          ))}
        </div>
      ) : currentItems.length === 0 ? (
        <div className="text-center py-12">
          <p className="text-text-muted">No {addLabel.toLowerCase()}s yet</
p>
        </div>
      ) : (
        <div className="space-y-2">
          {currentItems.map((item: any) => (
            <div key={item.id} className="bg-bg-card border border-border
rounded-xl p-4 flex items-center gap-3">

```

```

        <div className="w-8 h-8 rounded-lg bg-bg-elevated flex items-
center justify-center text-sm">
            {TABS.find(t => t.key === tab)?.icon}
        </div>
        <span className="text-sm font-medium">{item.name}</span>
    </div>
    )))
</div>
)}
</div>

{/* Add Button */}
<button
    onClick={() => setIsModalOpen(true)}
    className="fixed bottom-24 right-4 w-14 h-14 bg-accent text-white
rounded-full shadow-glow-accent flex items-center justify-center text-2xl
active:scale-90 transition-transform z-40"
>
    +
</button>

{/* Add Modal */}
<Modal
    isOpen={isModalOpen}
    onClose={() => setIsModalOpen(false)}
    title={`Add ${addLabel}`}
>
    <form onSubmit={handleAdd} className="space-y-4">
        <input
            type="text"
            required
            placeholder={` ${addLabel} name`}
            value={newName}
            onChange={(e) => setNewName(e.target.value)}
            className="w-full bg-bg border border-border rounded-xl px-4 py-3
text-text focus:outline-none focus:border-accent"
        />
        <button
            type="submit"
            disabled={saving}
            className="w-full py-3 bg-accent text-white font-bold rounded-xl
active:scale-[0.98] transition-transform disabled:opacity-50"
        >
            {saving ? 'Adding...' : `Add ${addLabel}`}
        </button>
    </form>
</Modal>
</div>
    );
}

...

## `src\app\more\export\page.tsx`

```tsx
'use client';

import { useState } from 'react';

export default function ExportPage() {

```

```

const [exporting, setExporting] = useState(false);
const [exportType, setExportType] = useState('all');
const [month, setMonth] = useState(String(new Date().getMonth() + 1));
const [year, setYear] = useState(String(new Date().getFullYear()));

const monthNames = ['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun', 'Jul', 'Aug', 'Sep', 'Oct', 'Nov', 'Dec'];

const handleExport = async () => {
 setExporting(true);
 try {
 const params = new URLSearchParams({ type: exportType });
 if (exportType !== 'all') {
 params.set('month', month);
 params.set('year', year);
 }

 const response = await fetch(`/api/export?${params}`);
 if (!response.ok) throw new Error('Export failed');

 const blob = await response.blob();
 const url = URL.createObjectURL(blob);
 const a = document.createElement('a');
 a.href = url;
 a.download = `shambala-${exportType}-${new Date().toLocaleDateString('en-CA')}.xlsx`;
 a.click();
 URL.revokeObjectURL(url);
 } catch (err) {
 console.error('Export failed:', err);
 } finally {
 setExporting(false);
 }
};

return (
 <div className="px-4 pt-6 max-w-lg mx-auto">
 <h1 className="text-xl font-bold mb-2">Export Data</h1>
 <p className="text-text-secondary text-sm mb-6">Download your transactions as an Excel file</p>

 <div className="mb-5">
 <label className="text-xs font-semibold text-text-muted uppercase tracking-wider mb-2 block">
 Export Type
 </label>
 <div className="space-y-2">
 {[
 { value: 'all', label: 'All Transactions', icon: '☐=ÜË' },
 { value: 'expense', label: 'Expenses Only', icon: '☐=Ü.' },
 { value: 'income', label: 'Money In Only', icon: '☐=Üµ' },
 { value: 'salary', label: 'Salaries Only', icon: '☐=Ü°' },
]}.map(opt => (
 <button
 key={opt.value}
 onClick={() => setExportType(opt.value)}
 className={`w-full flex items-center gap-3 p-4 rounded-2xl border text-left transition-all ${
 exportType === opt.value
 ? 'border-accent bg-accent-glow'
 : 'border-border bg-bg-card'
 }`
 >

```

```

 }}
 >
 {opt.icon}
 {opt.label}
 </button>
)}}
 </div>
</div>

{exportType !== 'all' && (
 <div className="flex gap-2 mb-8 animate-fade-in">
 <select
 value={month}
 onChange={e => setMonth(e.target.value)}
 className="flex-1 bg-bg-input border border-border rounded-xl
px-3 py-2.5 text-sm text-text-primary outline-none"
 >
 {monthNames.map((name, i) => (
 <option key={i} value={i + 1}>{name}</option>
))}
 </select>
 <select
 value={year}
 onChange={e => setYear(e.target.value)}
 className="bg-bg-input border border-border rounded-xl px-3
py-2.5 text-sm text-text-primary outline-none"
 >
 {[2024, 2025, 2026, 2027].map(y => (
 <option key={y} value={y}>{y}</option>
))}
 </select>
 </div>
)}

<button
 onClick={handleExport}
 disabled={exporting}
 className={`w-full py-4 rounded-2xl text-base font-bold transition-
all active:scale-[0.98] mb-8 ${
 exporting
 ? 'bg-bg-elevated text-text-muted'
 : 'bg-accent text-white shadow-glow-accent hover:brightness-110'
 }}
>
 {exporting ? 'Exporting...' : 'ðŸ“š Download Excel'}
</button>

<div className="border-t border-border pt-8">
 <h2 className="text-lg font-bold mb-2 text-text-primary">Full
Database Backup</h2>
 <p className="text-text-secondary text-sm mb-4">
 Download a complete raw JSON backup of all your data (people,
records, transactions, master data). Keep this safe in case you need to
restore your system.
 </p>
 <button
 onClick={() => {
 window.location.href = '/api/backup';
 }}
 className="w-full py-4 rounded-2xl text-base font-bold transition-
all active:scale-[0.98] border border-border bg-bg-card hover:border-accent

```

```

text-text-primary flex items-center justify-center gap-2"
 >
 ð=Û¼ Download Full Backup (.json)
 </button>
 </div>
</div>
);
}
...

`src\app\more\page.tsx`

````tsx
import Link from 'next/link';

const MENU_ITEMS = [
  {
    section: 'Data',
    items: [
      { href: '/more/categories', icon: 'ð=ÛÂ', label: 'Categories', desc:
'Manage expense categories' },
      { href: '/more/people', icon: 'ð=Ûe', label: 'People & Suppliers', desc:
'Manage workers, suppliers, contractors' },
      { href: '/more/accounts', icon: 'ð<ßæ', label: 'Accounts', desc: 'Cash,
bank, UPI accounts' },
    ],
  },
  {
    section: 'Import & Export',
    items: [
      { href: '/import', icon: 'ð=Ûå', label: 'Import Excel', desc: 'Import from
EXPENDITURE 2025.xlsx' },
      { href: '/more/export', icon: 'ð=Ûä', label: 'Export Data', desc: 'Download
as Excel/CSV' },
    ],
  },
  {
    section: 'About',
    items: [
      { href: '', icon: 'ð<ßxp ', label: 'Shambala v1.0', desc: 'Construction Site
Expense Manager' },
    ],
  },
];

export default function MorePage() {
  return (
    <div className="px-4 pt-6 max-w-lg mx-auto">
      <h1 className="text-xl font-bold mb-5">More</h1>

      {MENU_ITEMS.map(section => (
        <div key={section.section} className="mb-6">
          <h2 className="text-xs font-semibold text-text-muted uppercase
tracking-wider mb-3">
            {section.section}
          </h2>
          <div className="space-y-2">
            {section.items.map(item => {
              const content = (
                <>

```



```

        <span className="text-xl w-10 h-10 flex items-center
justify-center bg-bg-elevated rounded-xl">
            {item.icon}
        </span>
        <div className="flex-1 min-w-0">
            <h3 className="text-sm font-semibold">{item.label}</h3>
            <p className="text-xs text-text-muted mt-0.5">{item.desc}
</p>
        </div>
        {item.href && (
            <svg className="w-4 h-4 text-text-muted shrink-0"
fill="none" viewBox="0 0 24 24" stroke="currentColor" strokeWidth={2}>
                <path d="M9 5l7 7 7-7" strokeLinecap="round"
strokeLinejoin="round" />
            </svg>
        )}
    </>
    );

    if (!item.href) {
        return (
            <div
                key={item.label}
                className="flex items-center gap-4 bg-bg-card border
border-border rounded-2xl p-4"
            >
                {content}
            </div>
        );
    }

    return (
        <Link
            key={item.href}
            href={item.href}
            className="flex items-center gap-4 bg-bg-card border border-
border rounded-2xl p-4 hover:bg-bg-elevated transition-all active:scale-
[0.98]"
        >
            {content}
        </Link>
    );
    )}
</div>
</div>
    )}
</div>
);
}
...

```

```

## `src/app/more/people/loading.tsx`

```

```

` ``tsx
'use client';

import { SkeletonCard, SkeletonLine } from '@components/Skeletons';

export default function PeopleLoading() {
    return (

```

```

    <div className="px-4 pt-6 max-w-lg mx-auto animate-fade-in">
      <div className="flex items-center justify-between mb-5">
        <SkeletonLine width="w-44" height="h-7" />
        <div className="w-10 h-10 bg-bg-elevated rounded-xl animate-pulse-
subtle" />
      </div>

      {/* Search */}
      <div className="h-12 bg-bg-card border border-border rounded-xl mb-4
animate-pulse-subtle" />

        {/* List */}
        <div className="space-y-2">
          {[...Array(8)].map( (_, i) => (
            <SkeletonCard key={i} height="h-[64px]" />
          ))}
        </div>
      </div>
    );
  }
  ...

## `src\app\more\people\page.tsx`

```tsx
'use client';

import { useState, useEffect } from 'react';
import { getParties, createParty } from '@actions/parties';
import { PARTY_CLASS_LABELS } from '@lib/constants';
import type { Party, PartyClass } from '@lib/types';
import Link from 'next/link';

export default function PeoplePage() {
 const [parties, setParties] = useState<Party[]>([]);
 const [search, setSearch] = useState('');
 const [showAdd, setShowAdd] = useState(false);
 const [name, setName] = useState('');
 const [partyClass, setPartyClass] = useState<PartyClass>('person');
 const [phone, setPhone] = useState('');
 const [loading, setLoading] = useState(true);

 useEffect(() => {
 getParties().then(p => { setParties(p); setLoading(false); });
 }, []);

 const filtered = parties.filter(p =>
 p.name.toLowerCase().includes(search.toLowerCase())
);

 const handleAdd = async () => {
 if (!name.trim()) return;
 const party = await createParty({ name, class: partyClass, phone: phone
|| undefined });
 setParties([...parties, party]);
 setName('');
 setPhone('');
 setShowAdd(false);
 };
};

```

```

return (
 <div className="px-4 pt-6 max-w-lg mx-auto">
 <div className="flex items-center justify-between mb-5">
 <h1 className="text-xl font-bold">People & Suppliers</h1>
 <button
 onClick={() => setShowAdd(!showAdd)}
 className="w-10 h-10 flex items-center justify-center bg-accent
rounded-xl text-white text-xl font-bold active:scale-95 transition-transform"
 >
 +
 </button>
 </div>

 {showAdd && (
 <div className="bg-bg-card border border-border rounded-2xl p-4 mb-4
animate-fade-in">
 <input
 type="text"
 placeholder="Name"
 value={name}
 onChange={e => setName(e.target.value)}
 className="w-full bg-bg-input border border-border rounded-xl
px-4 py-3 text-sm mb-3 outline-none focus:border-accent"
 />
 <div className="flex gap-2 flex-wrap mb-3">
 {(Object.entries(PARTY_CLASS_LABELS) as [PartyClass, string]
[]).map(([key, label]) => (
 <button
 key={key}
 onClick={() => setPartyClass(key)}
 className={`px-3 py-2 rounded-lg text-xs font-medium border
transition-all ${
 partyClass === key
 ? 'border-accent bg-accent-glow text-accent-light'
 : 'border-border bg-bg-elevated text-text-secondary'
 }`}
 >
 {label}
 </button>
))}
 </div>
 <input
 type="tel"
 placeholder="Phone (optional)"
 value={phone}
 onChange={e => setPhone(e.target.value)}
 className="w-full bg-bg-input border border-border rounded-xl
px-4 py-3 text-sm mb-3 outline-none focus:border-accent"
 />
 <button
 onClick={handleAdd}
 disabled={!name.trim()}
 className="w-full py-3 bg-accent text-white rounded-xl font-
semibold active:scale-95 transition-transform disabled:opacity-40"
 >
 Add Person
 </button>
 </div>
)}
 </div>
 <input

```

```

 type="text"
 placeholder="Search people..."
 value={search}
 onChange={e => setSearch(e.target.value)}
 className="w-full bg-bg-input border border-border rounded-xl px-4
py-3 text-sm mb-4 outline-none focus:border-accent"
 />

 {loading ? (
 <div className="space-y-2">
 {[1, 2, 3].map(i => (
 <div key={i} className="bg-bg-card rounded-2xl p-4 animate-pulse-
subtle">
 <div className="w-24 h-4 bg-bg-elevated rounded" />
 </div>
))}
 </div>
) : filtered.length === 0 ? (
 <div className="text-center py-12">
 <div className="text-4xl mb-3">Ø=Üe</div>
 <p className="text-text-secondary text-sm">No people added yet</p>
 </div>
) : (
 <div className="space-y-2">
 {filtered.map(p => (
 <Link key={p.id} href={` /more/people/${p.id}`} className="block">
 <div className="bg-bg-card border border-border rounded-2xl p-4
hover:border-accent transition-colors">
 <div className="flex items-center justify-between">
 <div>
 <h3 className="text-sm font-semibold">{p.name}</h3>
 <p className="text-xs text-text-muted mt-0.5">
 {PARTY_CLASS_LABELS[p.class]}{p.phone ? ` · ${p.phone}`
: ''}
 </p>
 </div>
 <div className="text-accent">
 <svg xmlns="http://www.w3.org/2000/svg" className="h-5
w-5" viewBox="0 0 20 20" fill="currentColor">
 <path fillRule="evenodd" d="M7.293 14.707a1 1 0
010-1.414L10.586 10 7.293 6.707a1 1 0 011.414-1.414l4 4a1 1 0 010 1.414l-4
4a1 1 0 01-1.414 0z" clipRule="evenodd" />
 </svg>
 </div>
 </div>
 </div>
 </Link>
))}
 </div>
)}
 </div>
);
}

...

`src\app\more\people\[id]\page.tsx`

````tsx
'use client';

```

```

import { useState, useEffect, useMemo } from 'react';
import { getParty, getPartyTransactions } from '@actions/parties';
import { PARTY_CLASS_LABELS } from '@lib/constants';
import { DeletePartyButton } from '@components/DeletePartyButton';
import { formatCurrency, formatDate } from '@lib/utils';
import Link from 'next/link';
import { useParams } from 'next/navigation';

const TXN_TYPE_MAP: Record<string, { label: string; type: 'debit' | 'credit' }> = {
  operational_expense: { label: 'Payment', type: 'debit' },
  supplier_payment: { label: 'Supplier Payment', type: 'debit' },
  general_expense: { label: 'Expense', type: 'debit' },
  money_in: { label: 'Receipt', type: 'credit' },
  transfer: { label: 'Transfer', type: 'debit' },
  opening_balance: { label: 'Opening Balance', type: 'credit' },
};

export default function PartyLedgerPage() {
  const { id } = useParams<{ id: string }>();
  const [party, setParty] = useState<any>(null);
  const [transactions, setTransactions] = useState<any[]>([]);
  const [loading, setLoading] = useState(true);
  const [dateFrom, setDateFrom] = useState('');
  const [dateTo, setDateTo] = useState('');

  useEffect(() => {
    async function load() {
      setLoading(true);
      try {
        const [p, txns] = await Promise.all([
          getParty(id),
          getPartyTransactions(id, dateFrom || undefined, dateTo ||
undefined),
        ]);
        setParty(p);
        setTransactions(txns);
      } catch (err) {
        console.error('Failed to load party data', err);
      } finally {
        setLoading(false);
      }
    }
    load();
  }, [id, dateFrom, dateTo]);

  // Compute ledger rows with running balance
  const { ledgerRows, totalDebit, totalCredit, closingBalance } = useMemo(()
=> {
    let balance = 0;
    let debitSum = 0;
    let creditSum = 0;

    const rows = transactions.map((txn) => {
      const meta = TXN_TYPE_MAP[txn.type] || { label: txn.type, type: 'debit'
as const };
      const amount = Number(txn.amount) || 0;
      let debit = 0;
      let credit = 0;

      if (meta.type === 'debit') {

```

```

        debit = amount;
        balance += amount; // We paid them !' they received
    } else {
        credit = amount;
        balance -= amount; // They paid us !' we received
    }

    debitSum += debit;
    creditSum += credit;

    return {
        id: txn.id,
        date: txn.date,
        particulars: txn.description || meta.label,
        voucherType: meta.label,
        accountName: (txn.account as any)?.name || '',
        refTable: txn.reference_table,
        debit,
        credit,
        balance,
    };
});

return { ledgerRows: rows, totalDebit: debitSum, totalCredit: creditSum,
closingBalance: balance };
}, [transactions]);

if (loading && !party) {
    return (
        <main className="p-4 max-w-lg mx-auto pb-24 animate-fade-in">
            <div className="flex items-center gap-3 mb-6">
                <div className="w-10 h-10 bg-bg-elevated rounded-xl animate-pulse-
subtle" />
                <div className="space-y-2">
                    <div className="w-40 h-6 bg-bg-elevated rounded-lg animate-pulse-
subtle" />
                    <div className="w-24 h-3 bg-bg-elevated rounded animate-pulse-
subtle" />
                </div>
            </div>
            <div className="grid grid-cols-3 gap-2 mb-4">
                {[1, 2, 3].map(i => (
                    <div key={i} className="bg-bg-card border border-border
rounded-2xl p-4 h-20 animate-pulse-subtle" />
                ))}
            </div>
            {[1, 2, 3, 4, 5].map(i => (
                <div key={i} className="bg-bg-card border border-border rounded-2xl
p-3 h-14 animate-pulse-subtle mb-2" />
            ))}
        </main>
    );
}

if (!party) return null;

return (
    <main className="p-4 max-w-lg mx-auto pb-24">
        </* Header */>
        <div className="mb-5 flex items-center justify-between animate-fade-in">
            <div className="flex items-center gap-3">

```

```

        <Link href="/more/people" className="w-10 h-10 flex items-center
justify-center bg-bg-card border border-border rounded-xl text-text-secondary
active:scale-90 transition-transform">
            <svg xmlns="http://www.w3.org/2000/svg" className="h-5 w-5"
viewBox="0 0 20 20" fill="currentColor">
                <path fillRule="evenodd" d="M12.707 5.293a1 1 0 010 1.414L9.414
10l3.293 3.293a1 1 0 01-1.414 1.414l-4-4a1 1 0 010-1.414l4-4a1 1 0 011.414
0z" clipRule="evenodd" />
            </svg>
        </Link>
    </div>
    <div>
        <h1 className="text-xl font-bold">{party.name}</h1>
        <p className="text-xs text-text-
muted">{PARTY_CLASS_LABELS[party.class]}{party.phone ? ` · ${party.phone}` :
''}</p>
    </div>
</div>
<DeletePartyButton partyId={party.id} partyName={party.name} />
</div>

{/* Summary Cards */}
<div className="grid grid-cols-3 gap-2 mb-5 animate-fade-in stagger-1">
    <div className="bg-bg-card border border-border rounded-2xl p-3 text-
center">
        <p className="text-[10px] text-text-muted font-semibold uppercase
tracking-wider">Paid</p>
        <p className="text-base font-bold text-red-light
mt-1">{formatCurrency(totalDebit)}</p>
    </div>
    <div className="bg-bg-card border border-border rounded-2xl p-3 text-
center">
        <p className="text-[10px] text-text-muted font-semibold uppercase
tracking-wider">Received</p>
        <p className="text-base font-bold text-green
mt-1">{formatCurrency(totalCredit)}</p>
    </div>
    <div className={`border rounded-2xl p-3 text-center ${
        closingBalance > 0
        ? 'bg-red-light/5 border-red-light/20'
        : closingBalance < 0
        ? 'bg-green/5 border-green/20'
        : 'bg-bg-card border-border'
    }`>
        <p className="text-[10px] text-text-muted font-semibold uppercase
tracking-wider">Balance</p>
        <p className={`text-base font-bold mt-1 ${
            closingBalance > 0 ? 'text-red-light' : closingBalance < 0 ?
'text-green' : 'text-text-primary'
        }`>
            {closingBalance > 0 ? '' : closingBalance < 0 ? '' : ''}
            {formatCurrency(Math.abs(closingBalance))}
        </p>
        <p className="text-[9px] text-text-muted mt-0.5">
            {closingBalance > 0 ? 'We owe them' : closingBalance < 0 ? 'They
owe us' : 'Settled'}
        </p>
    </div>
</div>

{/* Date Filter */}
<div className="flex gap-2 mb-4 animate-fade-in stagger-2">

```

```

    <div className="flex-1">
      <label className="block text-[10px] text-text-muted font-semibold
uppercase tracking-wider mb-1">From</label>
      <input
        type="date"
        value={dateFrom}
        onChange={e => setDateFrom(e.target.value)}
        className="w-full bg-bg-card border border-border rounded-xl px-3
py-2 text-xs text-text-primary focus:outline-none focus:border-accent
transition-colors"
      />
    </div>
    <div className="flex-1">
      <label className="block text-[10px] text-text-muted font-semibold
uppercase tracking-wider mb-1">To</label>
      <input
        type="date"
        value={dateTo}
        onChange={e => setDateTo(e.target.value)}
        className="w-full bg-bg-card border border-border rounded-xl px-3
py-2 text-xs text-text-primary focus:outline-none focus:border-accent
transition-colors"
      />
    </div>
    {(dateFrom || dateTo) && (
      <button
        onClick={() => { setDateFrom(''); setDateTo(''); }}
        className="self-end px-3 py-2 text-xs text-accent font-semibold"
      >
        Clear
      </button>
    )}
  </div>

  {/* Ledger Header */}
  <div className="animate-fade-in stagger-3">
    <h2 className="text-xs font-semibold text-text-muted uppercase
tracking-wider mb-3 flex items-center gap-2">
      <span>ØÜØ</span> Ledger
      <span className="text-text-muted font-
normal">({transactions.length} entries)</span>
    </h2>

    {/* Column Headers */}
    <div className="grid grid-cols-[1fr_auto_auto_auto] gap-1 px-3 py-2
text-[10px] font-bold text-text-muted uppercase tracking-wider border-b
border-border mb-1">
      <span>Particulars</span>
      <span className="w-20 text-right">Debit</span>
      <span className="w-20 text-right">Credit</span>
      <span className="w-20 text-right">Balance</span>
    </div>

    {/* Ledger Rows */}
    {loading ? (
      <div className="space-y-1">
        {[...Array(5)].map((_, i) => (
          <div key={i} className="h-14 bg-bg-card border border-border
rounded-xl animate-pulse-subtle" />
        ))}
      </div>
    )}
  </div>

```



```

) : ledgerRows.length === 0 ? (
  <div className="text-center py-12">
    <p className="text-3xl mb-2">Ø=ÜØ</p>
    <p className="text-text-muted text-sm">No transactions found</p>
  </div>
) : (
  <div className="space-y-1">
    {ledgerRows.map((row, idx) => (
      <div
        key={row.id}
        className="grid grid-cols-[1fr_auto_auto_auto] gap-1 bg-bg-card border border-border rounded-xl px-3 py-2.5 items-center text-xs"
      >
        <div className="min-w-0">
          <p className="font-medium text-text-primary truncate">{row.particulars}</p>
          <div className="flex items-center gap-1.5 mt-0.5">
            <span className="text-[10px] text-text-muted">{formatDate(row.date)}</span>
            {row.accountName && (
              <>
                <span className="text-text-muted">.</span>
                <span className="text-[10px] text-text-muted truncate">{row.accountName}</span>
              </>
            )}
          </div>
          <span className={`w-20 text-right font-semibold tabular-nums ${row.debit > 0 ? 'text-red-light' : 'text-transparent'}}`>
            {row.debit > 0 ? formatCurrency(row.debit) : '-'}
          </span>
          <span className={`w-20 text-right font-semibold tabular-nums ${row.credit > 0 ? 'text-green' : 'text-transparent'}}`>
            {row.credit > 0 ? formatCurrency(row.credit) : '-'}
          </span>
          <span className={`w-20 text-right font-bold tabular-nums ${row.balance > 0 ? 'text-red-light' : row.balance < 0 ? 'text-green' : 'text-text-muted'}}`>
            {formatCurrency(Math.abs(row.balance))}
          </span>
        </div>
      </div>
    ))}

    {/* Totals Row */}
    <div className="grid grid-cols-[1fr_auto_auto_auto] gap-1 bg-bg-elevated border border-accent/20 rounded-xl px-3 py-3 items-center text-xs font-bold mt-2">
      <span className="text-text-primary uppercase tracking-wider text-[10px]">Total</span>
      <span className="w-20 text-right text-red-light tabular-nums">{formatCurrency(totalDebit)}</span>
      <span className="w-20 text-right text-green tabular-nums">{formatCurrency(totalCredit)}</span>
      <span className={`w-20 text-right tabular-nums ${closingBalance > 0 ? 'text-red-light' : closingBalance < 0 ? 'text-green' : 'text-text-primary'}}`>
        {formatCurrency(Math.abs(closingBalance))}
      </span>
    </div>
  </div>
)

```

```

        </div>
      </div>
    )}
  </div>
</main>
);
}
...

## `src\app\page.tsx`

````tsx
import Link from 'next/link';
import { getAccountBalances, getThisMonthSpent } from '@actions/accounts';
import { createClient } from '@lib/supabase/server';
import { redirect } from 'next/navigation';
import { formatCurrency, getGreeting, currentMonthLabel } from '@lib/utils';

export default async function HomePage() {
 const [balances, monthSpent] = await Promise.all([
 getAccountBalances(),
 getThisMonthSpent(),
]);

 const cashBalance = balances.find(b => b.account_type ===
'cash')?.balance ?? 0;
 const bankBalance = balances
 .filter(b => b.account_type === 'bank')
 .reduce((sum, b) => sum + b.balance, 0);

 return (
 <div className="px-4 pt-6 max-w-lg mx-auto pb-24">
 {/* =====
 GREETING
 ===== */}
 <div className="mb-6 animate-fade-in flex justify-between items-start">
 <div>
 <h1 className="text-2xl font-bold">{getGreeting()}</h1>
 </div>
 <div className="flex flex-col items-end gap-2">
 <Link href="/reports" className="text-sm font-medium text-accent">
 Reports
 </Link>
 <form action={async () => {
 'use server';
 const supabase = await createClient();
 await supabase.auth.signOut();
 redirect('/login');
 }}>
 <button type="submit" className="text-xs text-text-muted
hover:text-red-light transition-colors">
 Sign out
 </button>
 </form>
 </div>
 </div>

 {/* =====
 BALANCE CARDS
 ===== */}
 </div>
);
}

```

```

 <div className="flex justify-between items-end mb-3 animate-fade-in
stagger-1">
 <h2 className="text-sm font-semibold text-text-muted uppercase
tracking-wider">Balances</h2>
 <div className="flex gap-2">
 <Link href="/money/add" className="w-9 h-9 rounded-full bg-green/10
text-green flex items-center justify-center text-lg shadow-sm border border-
green/20" title="Money In">+</Link>
 <Link href="/money/transfer" className="w-9 h-9 rounded-full bg-
accent/10 text-accent-light flex items-center justify-center text-lg shadow-
sm border border-accent/20" title="Transfer">!</Link>
 </div>
 </div>
 <div className="grid grid-cols-2 gap-3 mb-4 animate-fade-in stagger-1">
 <div className="bg-bg-card border border-border rounded-2xl p-4">
 <p className="text-xs text-text-muted font-medium uppercase
tracking-wider">Cash</p>
 <p className="text-xl font-bold text-green
mt-1">{formatCurrency(cashBalance)}</p>
 </div>
 <div className="bg-bg-card border border-border rounded-2xl p-4">
 <p className="text-xs text-text-muted font-medium uppercase
tracking-wider">Bank</p>
 <p className="text-xl font-bold text-accent-light
mt-1">{formatCurrency(bankBalance)}</p>
 </div>
 </div>

 <div className="bg-bg-card border border-border rounded-2xl p-4 mb-6
animate-fade-in stagger-2">
 <p className="text-xs text-text-muted font-medium uppercase tracking-
wider">{currentMonthLabel()}</p>
 <p className="text-xl font-bold text-red-light
mt-1">{formatCurrency(monthSpent)} <span className="text-sm font-normal text-
text-muted">spent</p>
 </div>

 {/* =====
 QUICK ADD ACTIONS (ERP)
 ===== */}
 <div className="mb-4 animate-fade-in stagger-3">
 <h2 className="text-sm font-semibold text-text-muted uppercase
tracking-wider mb-3">
 Quick Entry
 </h2>
 <div className="grid grid-cols-2 gap-3">
 <Link
 href="/labour/add"
 className="flex flex-col items-center justify-center gap-2 bg-bg-
card border border-border rounded-2xl p-5 text-text-secondary hover:text-
accent-light hover:border-accent/30 transition-all active:scale-95 shadow-sm"
 >
 ØÜw
 LABOUR
 </Link>
 <Link
 href="/materials/add"
 className="flex flex-col items-center justify-center gap-2 bg-bg-
card border border-border rounded-2xl p-5 text-text-secondary hover:text-
accent-light hover:border-accent/30 transition-all active:scale-95 shadow-sm"
 >

```

```

 Ø>Ýñ
 MATERIAL
 </Link>
 <Link
 href="/food/add"
 className="flex flex-col items-center justify-center gap-2 bg-bg-
card border border-border rounded-2xl p-5 text-text-secondary hover:text-
accent-light hover:border-accent/30 transition-all active:scale-95 shadow-sm"
 >
 Ø<ßZ
 FOOD
 </Link>
 <Link
 href="/machinery/add"
 className="flex flex-col items-center justify-center gap-2 bg-bg-
card border border-border rounded-2xl p-5 text-text-secondary hover:text-
accent-light hover:border-accent/30 transition-all active:scale-95 shadow-sm"
 >
 Ø<ßæ
 MACHINERY
 </Link>
 <Link
 href="/salary/add"
 className="flex flex-col items-center justify-center gap-2 bg-bg-
card border border-border rounded-2xl p-5 text-text-secondary hover:text-
accent-light hover:border-accent/30 transition-all active:scale-95 shadow-sm"
 >
 Ø<Ü°
 SALARY
 </Link>
 <Link
 href="/transport/add"
 className="flex flex-col items-center justify-center gap-2 bg-bg-
card border border-border rounded-2xl p-5 text-text-secondary hover:text-
accent-light hover:border-accent/30 transition-all active:scale-95 shadow-sm"
 >
 Ø<ßš
 TRANSPORT
 </Link>
 </div>
 </div>
</div>
);
}

```

```

...

`src\app\reports\page.tsx`

```

```

```tsx
'use client';

import { useState, useEffect } from 'react';
import { getMonthlyReport, getDayBook, getBuildingReport, type ModuleSpend }
from '@actions/reports';
import { formatCurrency, formatDate } from '@lib/utils';
import { TransactionCard } from '@components/TransactionCard';

type Tab = 'overview' | 'daybook' | 'buildings';

export default function ReportsPage() {

```

```

const now = new Date();
const [tab, setTab] = useState<Tab>('overview');

// Overview / Building State
const [year, setYear] = useState(now.getFullYear());
const [month, setMonth] = useState(now.getMonth() + 1);

// Daybook State
const [dayDate, setDayDate] = useState(now.toLocaleDateString('en-CA'));

const [loading, setLoading] = useState(true);
const [modules, setModules] = useState<ModuleSpend[]>([]);
const [grandTotal, setGrandTotal] = useState(0);
const [dayTransactions, setDayTransactions] = useState<any[]>([]);
const [buildingCosts, setBuildingCosts] = useState<any[]>([]);

useEffect(() => {
  async function loadData() {
    setLoading(true);
    try {
      if (tab === 'overview') {
        const res = await getMonthlyReport(year, month);
        setModules(res.modules);
        setGrandTotal(res.grandTotal);
      } else if (tab === 'daybook') {
        const res = await getDayBook(dayDate);
        setDayTransactions(res);
      } else if (tab === 'buildings') {
        const res = await getBuildingReport(year, month);
        setBuildingCosts(res);
      }
    } catch (err) {
      console.error('Failed to load report data', err);
    } finally {
      setLoading(false);
    }
  }
  loadData();
}, [tab, year, month, dayDate]);

const monthLabel = new Date(year, month - 1).toLocaleDateString('en-IN',
{ month: 'long', year: 'numeric' });
const isCurrentMonth = year === now.getFullYear() && month ===
now.getMonth() + 1;

function prevMonth() {
  if (month === 1) { setMonth(12); setYear(y => y - 1); }
  else setMonth(m => m - 1);
}
function nextMonth() {
  if (month === 12) { setMonth(1); setYear(y => y + 1); }
  else setMonth(m => m + 1);
}

const TABS: { key: Tab; label: string; icon: string }[] = [
  { key: 'overview', label: 'Overview', icon: '📊' },
  { key: 'daybook', label: 'Day Book', icon: '📅' },
  { key: 'buildings', label: 'Buildings', icon: '🏠' },
];

return (

```

```

<div className="px-4 pt-6 pb-24 max-w-lg mx-auto">
  <h1 className="text-xl font-bold mb-4 animate-fade-in">Reports
Dashboard</h1>

  {/* Tabs */}
  <div className="flex gap-2 mb-6 animate-fade-in stagger-1">
    {TABS.map((t) => (
      <button
        key={t.key}
        onClick={() => setTab(t.key)}
        className={`flex-1 py-2.5 rounded-xl text-xs font-semibold
transition-all ${
          tab === t.key
            ? 'bg-accent text-white shadow-glow-accent'
            : 'bg-bg-card text-text-secondary border border-border
hover:border-accent/30'
        }`}
      >
        <span className="block text-lg mb-1">{t.icon}</span>
        {t.label}
      </button>
    ))}
  </div>

  {/* Tab Content */}
  <div className="animate-fade-in stagger-2">
    {tab === 'overview' && (
      <>
        <MonthSelector
          monthLabel={monthLabel}
          isCurrentMonth={isCurrentMonth}
          onPrev={prevMonth}
          onNext={nextMonth}
        />
        <div className="bg-bg-card border border-border rounded-2xl p-5
mb-5">
          <p className="text-xs text-text-muted font-medium uppercase
tracking-wider">Total Expenditure</p>
          {loading ? (
            <div className="h-8 w-32 bg-bg-elevated rounded-lg animate-
pulse-subtle mt-2" />
          ) : (
            <p className="text-3xl font-bold text-red-light
mt-1">{formatCurrency(grandTotal)}</p>
          )}
        </div>
        <div className="space-y-3">
          <h2 className="text-xs font-semibold text-text-muted uppercase
tracking-wider">By Category</h2>
          {loading ? <SkeletonList count={6} height="h-16" /> : (
            modules.sort((a, b) => b.total - a.total).map((mod) => {
              const pct = grandTotal > 0 ? (mod.total / grandTotal) *
100 : 0;
              return (
                <div key={mod.module} className="bg-bg-card border border-
border rounded-2xl p-4">
                  <div className="flex items-center justify-between mb-2">
                    <div className="flex items-center gap-2">
                      <span className="text-lg">{mod.icon}</span>
                      <span className="text-sm font-semibold">{mod.label}
</span>

```

```

        </div>
        <span className="text-sm font-
bold">{formatCurrency(mod.total)}</span>
    </div>
    <div className="h-2 bg-bg-elevated rounded-full
overflow-hidden">
        <div className="h-full rounded-full bg-accent
transition-all duration-500" style={{ width: `${Math.max(pct, 0.5)}%` }} />
    </div>
    <p className="text-[10px] text-text-muted mt-1 text-
right">{pct.toFixed(1)}%</p>
    </div>
    );
    })
    </div>
</>
)}

{tab === 'daybook' && (
    <>
        <div className="bg-bg-card border border-border rounded-2xl p-4
mb-5 flex items-center justify-between">
            <label className="text-sm font-semibold text-text-
secondary">Select Date:</label>
            <input
                type="date"
                value={dayDate}
                onChange={e => setDayDate(e.target.value)}
                className="bg-bg-input border border-border rounded-xl px-3
py-2 text-sm text-text-primary focus:outline-none focus:border-accent"
            />
        </div>

        {loading ? <SkeletonList count={5} height="h-[72px]" /> :
        dayTransactions.length === 0 ? (
            <div className="text-center py-12 border border-dashed border-
border rounded-2xl">
                <p className="text-3xl mb-2">Ø=ÜÅ</p>
                <p className="text-text-muted text-sm">No transactions for
this date.</p>
            </div>
        ) : (
            <div className="space-y-4">
                {dayTransactions.map(t => (
                    <TransactionCard
                        key={t.id}
                        title={t.description || t.type.replace('_', ' ')}
                        subtitle={`$${(t.party as any)?.name || 'Direct'}`
                        .
                        $${(t.account as any)?.name || 'Cash'}`}
                        amount={t.amount}
                        date={t.date}
                        module={t.type}
                        isIncome={t.type === 'money_in' || t.type ===
'opening_balance'}
                    />
                ))}
            </div>
        )}
    </>
)}

```

```

{tab === 'buildings' && (
  <>
    <MonthSelector
      monthLabel={monthLabel}
      isCurrentMonth={isCurrentMonth}
      onPrev={prevMonth}
      onNext={nextMonth}
    />
    <div className="space-y-3">
      <h2 className="text-xs font-semibold text-text-muted uppercase
tracking-wider mb-2">Cost by Building</h2>
      {loading ? <SkeletonList count={4} height="h-24" /> :
buildingCosts.length === 0 ? (
        <div className="text-center py-12 border border-dashed border-
border rounded-2xl">
          <p className="text-3xl mb-2">Ø<ßâ</p>
          <p className="text-text-muted text-sm">No building costs
found for this month.</p>
        </div>
      ) : (
        buildingCosts.map(b => (
          <div key={b.id} className="bg-bg-card border border-border
rounded-2xl p-4">
            <div className="flex justify-between items-center border-
b border-border pb-2 mb-2">
              <h3 className="font-bold text-base">{b.name}</h3>
              <span className="font-bold text-red-
light">{formatCurrency(b.total)}</span>
            </div>
            <div className="grid grid-cols-3 gap-2 text-center text-
xs">
              <div>
                <p className="text-text-muted mb-0.5">Labour</p>
                <p className="font-semibold text-text-
primary">{formatCurrency(b.breakdown.labour)}</p>
              </div>
              <div>
                <p className="text-text-muted mb-0.5">Materials</p>
                <p className="font-semibold text-text-
primary">{formatCurrency(b.breakdown.materials)}</p>
              </div>
              <div>
                <p className="text-text-muted mb-0.5">Machinery</p>
                <p className="font-semibold text-text-
primary">{formatCurrency(b.breakdown.machinery)}</p>
              </div>
            </div>
          </div>
        ))
      </div>
    </div>
  )}
</div>
</div>
);
}

```

```

// Subcomponents for reuse in this file
function MonthSelector({ monthLabel, isCurrentMonth, onPrev, onNext }: any) {

```



```

    return (
      <div className="flex items-center justify-between bg-bg-card border
border-border rounded-2xl p-4 mb-5">
        <button onClick={onPrev} className="w-10 h-10 rounded-xl bg-bg-elevated
flex items-center justify-center text-text-secondary hover:text-text-primary
active:scale-90 transition-all">!</button>
        <div className="text-center">
          <p className="text-base font-bold">{monthLabel}</p>
          {isCurrentMonth && <p className="text-[10px] text-accent font-
semibold">CURRENT MONTH</p>}
        </div>
        <button onClick={onNext} disabled={isCurrentMonth} className="w-10 h-10
rounded-xl bg-bg-elevated flex items-center justify-center text-text-
secondary hover:text-text-primary active:scale-90 transition-all
disabled:opacity-30">!</button>
      </div>
    );
  }
}

```

```

function SkeletonList({ count, height }: { count: number, height: string }) {
  return (
    <div className="space-y-3">
      {[...Array(count)].map((_, i) => (
        <div key={i} className={`bg-bg-card border border-border rounded-2xl
p-4 ${height} animate-pulse-subtle`} />
      ))}
    </div>
  );
}
...

```

```

## `src\app\salary\add\loading.tsx`

```

```

```tsx
import { SkeletonFormPage } from '@components/Skeletons';

export default function SalaryAddLoading() {
 return <SkeletonFormPage />;
}
...

```

```

`src\app\salary\add\page.tsx`

```

```

```tsx
import { getParties } from '@actions/parties';
import { getAccounts } from '@actions/accounts';
import { SalaryForm } from './SalaryForm';

export default async function AddSalaryPage() {
  const [salariedParties, investorParties, accs] = await Promise.all([
    getParties(undefined, 'salaried'),
    getParties(undefined, 'investor'),
    getAccounts(),
  ]);

  return (
    <SalaryForm
      initialEmployees={salariedParties}
      initialProviders={investorParties}
    />
  );
}

```

```

        accounts={accs}
      />
    );
  }
  ...

## `src\app\salary\add\SalaryForm.tsx`

````tsx
'use client';

import { useState } from 'react';
import { useRouter } from 'next/navigation';
import { createSalaryRecord } from '@actions/salary';
import { createParty } from '@actions/parties';
import { SearchableSelect } from '@components/SearchableSelect';
import { PAYMENT_METHOD_LABELS } from '@lib/constants';

export function SalaryForm({
 initialEmployees,
 initialProviders,
 accounts
}: {
 initialEmployees: any[];
 initialProviders: any[];
 accounts: any[];
}) {
 const router = useRouter();
 const [saving, setSaving] = useState(false);

 const [employees, setEmployees] = useState<any[]>(initialEmployees);
 const [cashProviders, setCashProviders] = useState<any[]>(initialProviders);

 const [employeeId, setEmployeeId] = useState('');
 const [date, setDate] = useState(new Date().toLocaleDateString('en-CA'));
 const [amount, setAmount] = useState('');
 const [paymentMethod, setPaymentMethod] = useState('cash');
 const [cashProviderId, setCashProviderId] = useState('');
 const [accountId, setAccountId] = useState(accounts.length > 0 ?
(accounts.find((a: any) => a.is_default)?.id || accounts[0].id) : '');
 const [comments, setComments] = useState('');
 const [period, setPeriod] = useState('');

 async function handleAddEmployee(name: string) {
 if (!name.trim()) return;
 setSaving(true);
 try {
 const party = await createParty({ name, class: 'person', role:
'salaried' });
 setEmployees(prev => [...prev, party]);
 setEmployeeId(party.id);
 } catch (err) {
 alert('Failed to add employee');
 } finally {
 setSaving(false);
 }
 }

 async function handleAddCashProvider(name: string) {
 if (!name.trim()) return;

```

```

 setSaving(true);
 try {
 const party = await createParty({ name, class: 'person', role:
'investor' });
 setCashProviders(prev => [...prev, party]);
 setCashProviderId(party.id);
 } catch (err) {
 alert('Failed to add cash provider');
 } finally {
 setSaving(false);
 }
 }

 async function handleSubmit(e: React.FormEvent) {
 e.preventDefault();
 setSaving(true);
 try {
 // Derive start_date and end_date from the period (month) if set
 let startDate = date;
 let endDate = date;
 if (period) {
 const [y, m] = period.split('-').map(Number);
 startDate = `${y}-${String(m).padStart(2, '0')}-01`;
 endDate = new Date(y, m, 0).toLocaleDateString('en-CA');
 }

 await createSalaryRecord({
 employee_id: employeeId || undefined,
 start_date: startDate,
 end_date: endDate,
 payment_date: date,
 amount: amount ? Number(amount) : undefined,
 payment_method: paymentMethod as any,
 cash_provider_id: cashProviderId || undefined,
 comments: comments || undefined,
 account_id: accountId || undefined,
 } as any);
 router.push('/');
 } catch (err) {
 console.error(err);
 alert('Failed to save salary record');
 } finally {
 setSaving(false);
 }
 }

 const employeeOptions = employees.map(e => ({ id: e.id, label: e.name }));
 const providerOptions = cashProviders.map(p => ({ id: p.id, label:
p.name }));
 const accountOptions = accounts.map((a: any) => ({ id: a.id, label: a.name,
subLabel: a.type.toUpperCase() }));

 return (
 <div className="px-4 pt-6 pb-24 max-w-lg mx-auto animate-fade-in">
 <div className="flex items-center mb-6">
 <button onClick={() => router.back()} className="mr-3 text-2xl text-
text-secondary active:scale-90 transition-transform">
 !•
 </button>
 <h1 className="text-2xl font-bold">Add Salary/Advance</h1>
 </div>
 </div>
)

```

```

 <form onSubmit={handleSubmit} className="space-y-4">
 <div className="bg-bg-card border border-border rounded-2xl p-5
shadow-sm space-y-4">

 <SearchableSelect
 label="Employee / Worker"
 options={employeeOptions}
 value={employeeId}
 onChange={setEmployeeId}
 placeholder="Ø=Ÿ Select Person"
 onAddNew={(name) => handleAddEmployee(name || 'New Employee')}
 addNewLabel="Add Employee"
 />

 <div>
 <label className="block text-xs font-semibold text-text-muted
uppercase tracking-wider mb-2">
 Date
 </label>
 <input
 type="date"
 required
 value={date}
 onChange={(e) => setDate(e.target.value)}
 className="w-full bg-bg border border-border rounded-xl px-4
py-3 text-text focus:outline-none focus:border-accent transition-colors"
 />
 </div>

 <div>
 <label className="block text-xs font-semibold text-text-muted
uppercase tracking-wider mb-2">
 Salary Month / Period
 </label>
 <input
 type="month"
 value={period}
 onChange={(e) => setPeriod(e.target.value)}
 className="w-full bg-bg border border-border rounded-xl px-4
py-3 text-text focus:outline-none focus:border-accent transition-colors"
 />
 </div>

 <div>
 <label className="block text-xs font-semibold text-text-muted
uppercase tracking-wider mb-2">
 Amount (¹)
 </label>
 <input
 type="number"
 value={amount}
 onChange={(e) => setAmount(e.target.value)}
 className="w-full bg-bg border border-border rounded-xl px-4
py-3 text-text focus:outline-none focus:border-accent transition-colors text-
xl font-bold"
 placeholder="0.00"
 />
 </div>

 </div>
 </form>

```

```

<div className="bg-bg-card border border-border rounded-2xl p-5
shadow-sm space-y-4">

 <SearchableSelect
 label="Paid From Account"
 options={accountOptions}
 value={accountId}
 onChange={setAccountId}
 placeholder="Ø=Ý Select Account"
 />

 <SearchableSelect
 label="Cash Given By"
 options={providerOptions}
 value={cashProviderId}
 onChange={setCashProviderId}
 placeholder="Ø=Ý Search person"
 onAddNew={(name) => handleAddCashProvider(name || 'New Person')}
 addNewLabel="Add Person"
 />

 <div>
 <label className="block text-xs font-semibold text-text-muted
uppercase tracking-wider mb-2">
 Payment Method
 </label>
 <select
 value={paymentMethod}
 onChange={e => setPaymentMethod(e.target.value)}
 className="w-full bg-bg border border-border rounded-xl px-4
py-3 text-text focus:outline-none focus:border-accent appearance-none"
 >
 {Object.entries(PAYMENT_METHOD_LABELS).map(([key, label]) => (
 <option key={key} value={key}>{label}</option>
))}
 </select>
 </div>

 <div>
 <label className="block text-xs font-semibold text-text-muted
uppercase tracking-wider mb-2">
 Comments (e.g. Advance, Final Settlement)
 </label>
 <input
 type="text"
 value={comments}
 onChange={(e) => setComments(e.target.value)}
 className="w-full bg-bg border border-border rounded-xl px-4
py-3 text-text focus:outline-none focus:border-accent"
 placeholder="Optional comments"
 />
 </div>
</div>

<button
 type="submit"
 disabled={saving || !employeeId}
 className="block w-full py-4 bg-accent text-white text-center text-
lg font-bold rounded-2xl shadow-glow-accent hover:brightness-110 transition-
all active:scale-[0.98] disabled:opacity-50 mt-4"

```

```

 >
 {saving ? 'Saving...' : 'SAVE SALARY'}
 </button>
 </form>
 </div>
);
}
...

`src\app\salary\loading.tsx`

```tsx
import { SkeletonListPage } from '@components/Skeletons';

export default function SalaryLoading() {
  return <SkeletonListPage />;
}
...

## `src\app\salary\page.tsx`

```tsx
import Link from 'next/link';
import { getSalaryRecords } from '@actions/salary';
import { deleteRecord } from '@actions/transactions';
import { TransactionCard } from '@components/TransactionCard';

export default async function SalaryPage() {
 const records = await getSalaryRecords();

 return (
 <main className="p-4 max-w-lg mx-auto pb-24">
 <div className="flex justify-between items-center mb-6">
 <h1 className="text-2xl font-semibold">Salary Records</h1>
 </div>

 <div className="space-y-4">
 {records.map((r) => (
 <TransactionCard
 key={r.id}
 title={r.employee?.name || 'Unknown Employee'}
 subtitle={`$${r.start_date} to $${r.end_date}`}
 amount={r.amount}
 date={r.payment_date}
 module="salary"
 icon="ðŸ“°"
 onDelete={async () => {
 'use server';
 await deleteRecord('salary_records', r.id);
 }}
 />
))}
 </div>

 {records.length === 0 && (
 <div className="text-center text-text-muted py-8">
 No salary records found.
 </div>
)}
 </div>
);
}

```

```

 { /* Floating Add Button */}
 <Link
 href="/salary/add"
 className="fixed bottom-20 right-4 w-14 h-14 bg-accent text-white
rounded-full flex items-center justify-center shadow-lg hover:brightness-110
transition-all z-50"
 >
 <svg xmlns="http://www.w3.org/2000/svg" className="h-8 w-8"
fill="none" viewBox="0 0 24 24" stroke="currentColor">
 <path strokeLinecap="round" strokeLinejoin="round" strokeWidth={2}
d="M12 4v16m8-8H4" />
 </svg>
 </Link>
 </main>
);
}
...

```

```

`src\app\transport\add\loading.tsx`

```

```

````tsx
import { SkeletonFormPage } from '@components/Skeletons';

export default function TransportAddLoading() {
  return <SkeletonFormPage />;
}
...

```

```

## `src\app\transport\add\page.tsx`

```

```

````tsx
import { getParties } from '@actions/parties';
import { getTransportVehicles } from '@actions/master';
import { getAccounts } from '@actions/accounts';
import { TransportForm } from '../TransportForm';

export default async function AddTransportPage() {
 const [vehs, investorParties, accs] = await Promise.all([
 getTransportVehicles(),
 getParties(undefined, 'investor'),
 getAccounts(),
]);

 return (
 <TransportForm
 initialVehicles={vehs}
 initialProviders={investorParties}
 accounts={accs}
 />
);
}
...

```

```

`src\app\transport\add\TransportForm.tsx`

```

```

````tsx
'use client';

```

```

import { useState } from 'react';
import { useRouter } from 'next/navigation';
import { createStandaloneTransportRecord } from '@actions/transport';
import { createParty } from '@actions/parties';
import { SearchableSelect } from '@components/SearchableSelect';
import { PAYMENT_METHOD_LABELS } from '@lib/constants';

export function TransportForm({
  initialVehicles,
  initialProviders,
  accounts
}: {
  initialVehicles: any[];
  initialProviders: any[];
  accounts: any[];
}) {
  const router = useRouter();
  const [saving, setSaving] = useState(false);

  const [cashProviders, setCashProviders] = useState<any[]>(initialProviders);

  const [vehicleId, setVehicleId] = useState('');
  const [date, setDate] = useState(new Date().toLocaleDateString('en-CA'));
  const [amount, setAmount] = useState('');
  const [paymentMethod, setPaymentMethod] = useState('cash');
  const [cashProviderId, setCashProviderId] = useState('');
  const [accountId, setAccountId] = useState(accounts.length > 0 ?
(accounts.find((a: any) => a.is_default)?.id || accounts[0].id) : '');
  const [comments, setComments] = useState('');

  const [source, setSource] = useState('');
  const [destination, setDestination] = useState('');

  async function handleAddCashProvider(name: string) {
    if (!name.trim()) return;
    setSaving(true);
    try {
      const party = await createParty({ name, class: 'person', role:
'investor' });
      setCashProviders(prev => [...prev, party]);
      setCashProviderId(party.id);
    } catch (err) {
      alert('Failed to add cash provider');
    } finally {
      setSaving(false);
    }
  }

  async function handleSubmit(e: React.FormEvent) {
    e.preventDefault();
    setSaving(true);
    try {
      await createStandaloneTransportRecord({
        vehicle_id: vehicleId || undefined,
        date: date,
        amount: amount ? Number(amount) : undefined,
        payment_method: paymentMethod as any,
        cash_provider_id: cashProviderId || undefined,
        comments: comments || undefined,
        source: source || undefined,

```



```

        destination: destination || undefined,
        account_id: accountId || undefined,
    });
    router.push('/');
} catch (err) {
    console.error(err);
    alert('Failed to save transport record');
} finally {
    setSaving(false);
}
}

const vehicleOptions = initialVehicles.map(v => ({ id: v.id, label:
v.vehicle_number, subLabel: v.vehicle_type }));
const providerOptions = cashProviders.map(p => ({ id: p.id, label:
p.name }));
const accountOptions = accounts.map((a: any) => ({ id: a.id, label: a.name,
subLabel: a.type.toUpperCase() }));

return (
    <div className="px-4 pt-6 pb-24 max-w-lg mx-auto animate-fade-in">
        <div className="flex items-center mb-6">
            <button onClick={() => router.back()} className="mr-3 text-2xl text-
text-secondary active:scale-90 transition-transform">
                !•
            </button>
            <h1 className="text-2xl font-bold">Add Transport Trip</h1>
        </div>

        <form onSubmit={handleSubmit} className="space-y-4">
            <div className="bg-bg-card border border-border rounded-2xl p-5
shadow-sm space-y-4">

                <SearchableSelect
                    label="Vehicle"
                    options={vehicleOptions}
                    value={vehicleId}
                    onChange={setVehicleId}
                    placeholder="Ø=Ý Select Vehicle"
                />

                <div>
                    <label className="block text-xs font-semibold text-text-muted
uppercase tracking-wider mb-2">
                        Date
                    </label>
                    <input
                        type="date"
                        required
                        value={date}
                        onChange={(e) => setDate(e.target.value)}
                        className="w-full bg-bg border border-border rounded-xl px-4
py-3 text-text focus:outline-none focus:border-accent transition-colors"
                    />
                </div>

                <div className="flex gap-3">
                    <div className="flex-1">
                        <label className="block text-xs font-semibold text-text-muted
uppercase tracking-wider mb-2">
                            From

```

```

        </label>
        <input
          type="text"
          value={source}
          onChange={(e) => setSource(e.target.value)}
          className="w-full bg-bg border border-border rounded-xl px-4
py-3 text-text focus:outline-none focus:border-accent transition-colors"
          placeholder="Source"
        />
      </div>
      <div className="flex-1">
        <label className="block text-xs font-semibold text-text-muted
uppercase tracking-wider mb-2">
          To
        </label>
        <input
          type="text"
          value={destination}
          onChange={(e) => setDestination(e.target.value)}
          className="w-full bg-bg border border-border rounded-xl px-4
py-3 text-text focus:outline-none focus:border-accent transition-colors"
          placeholder="Destination"
        />
      </div>
    </div>

    <div>
      <label className="block text-xs font-semibold text-text-muted
uppercase tracking-wider mb-2">
        Transport Cost ( 1)
      </label>
      <input
        type="number"
        value={amount}
        onChange={(e) => setAmount(e.target.value)}
        className="w-full bg-bg border border-border rounded-xl px-4
py-3 text-text focus:outline-none focus:border-accent transition-colors text-
xl font-bold"
        placeholder="0.00"
      />
    </div>

  </div>

  <div className="bg-bg-card border border-border rounded-2xl p-5
shadow-sm space-y-4">

    <SearchableSelect
      label="Paid From Account"
      options={accountOptions}
      value={accountId}
      onChange={setAccountId}
      placeholder="Ø=Ý Select Account"
    />

    <SearchableSelect
      label="Cash Given By"
      options={providerOptions}
      value={cashProviderId}
      onChange={setCashProviderId}
      placeholder="Ø=Ý Search person"
    />
  </div>

```

```

        onAddNew={ (name) => handleAddCashProvider(name || 'New Person')}
        addNewLabel="Add Person"
      />

      <div>
        <label className="block text-xs font-semibold text-text-muted
uppercase tracking-wider mb-2">
          Payment Method
        </label>
        <select
          value={paymentMethod}
          onChange={e => setPaymentMethod(e.target.value)}
          className="w-full bg-bg border border-border rounded-xl px-4
py-3 text-text focus:outline-none focus:border-accent appearance-none"
        >
          {Object.entries(PAYMENT_METHOD_LABELS).map(([key, label]) => (
            <option key={key} value={key}>{label}</option>
          ))}
        </select>
      </div>

      <div>
        <label className="block text-xs font-semibold text-text-muted
uppercase tracking-wider mb-2">
          Comments
        </label>
        <input
          type="text"
          value={comments}
          onChange={(e) => setComments(e.target.value)}
          className="w-full bg-bg border border-border rounded-xl px-4
py-3 text-text focus:outline-none focus:border-accent"
          placeholder="Optional comments"
        />
      </div>
    </div>

    <button
      type="submit"
      disabled={saving || !vehicleId}
      className="block w-full py-4 bg-accent text-white text-center text-
lg font-bold rounded-2xl shadow-glow-accent hover:brightness-110 transition-
all active:scale-[0.98] disabled:opacity-50 mt-4"
    >
      {saving ? 'Saving...' : 'SAVE TRANSPORT'}
    </button>
  </form>
</div>
  );
}

...

## `src\app\transport\loading.tsx`

```tsx
import { SkeletonListPage } from '@components/Skeletons';

export default function TransportLoading() {
 return <SkeletonListPage />;
}

```

....

```
`src\app\transport\page.tsx`
```

```
````tsx
import Link from 'next/link';
import { TransactionCard } from '@components/TransactionCard';
import { deleteRecord } from '@actions/transactions';
import { createClient } from '@lib/supabase/server';
import { DEFAULT_PROJECT_ID } from '@lib/constants';

async function getTransportRecords(limit = 50) {
  const supabase = await createClient();

  const { data, error } = await supabase
    .from('transport_records')
    .select(`
      *,
      vehicle:transport_vehicles(vehicle_number, vehicle_type),
      delivery:material_deliveries(date, material:materials(name))
    `)
    .order('created_at', { ascending: false })
    .limit(limit);

  if (error) throw new Error('Failed to load transport records');
  return data;
}

export default async function TransportPage() {
  const records = await getTransportRecords();

  return (
    <main className="p-4 max-w-lg mx-auto pb-24">
      <div className="flex justify-between items-center mb-6">
        <h1 className="text-2xl font-semibold">Transport Records</h1>
      </div>

      <div className="space-y-4">
        {records.map((r) => {
          const vehicle = r.vehicle as any;
          const delivery = r.delivery as any;
          const date = delivery?.date || new Date().toLocaleDateString('en-
CA');
          const materialName = delivery?.material?.name || '';

          return (
            <TransactionCard
              key={r.id}
              title={vehicle ? `${vehicle.vehicle_type}
(${vehicle.vehicle_number})` : 'Transport'}
              subtitle={
                [r.source_location,
materialName].filter(Boolean).join(' ! ' ) || 'Transport'}
              amount={r.amount}
              date={date}
              module="transport"
              icon="🚚"
              onDelete={async () => {
                'use server';
                await deleteRecord('transport_records', r.id);
              }}
            </TransactionCard>
          );
        })}
      </div>
    </main>
  );
}
```

```

    />
  });
  })}

  {records.length === 0 && (
    <div className="text-center text-text-muted py-8">
      No transport records found.
    </div>
  )}
</div>

{ /* Floating Add Button */}
<Link
  href="/transport/add"
  className="fixed bottom-20 right-4 w-14 h-14 bg-accent text-white
rounded-full flex items-center justify-center shadow-lg hover:brightness-110
transition-all z-50"
>
  <svg xmlns="http://www.w3.org/2000/svg" className="h-8 w-8"
fill="none" viewBox="0 0 24 24" stroke="currentColor">
    <path strokeLinecap="round" strokeLinejoin="round" strokeWidth={2}
d="M12 4v16m8-8H4" />
  </svg>
</Link>
</main>
);
}
...

## `src\components\BottomNav.tsx`

```tsx
'use client';

import Link from 'next/link';
import { usePathname } from 'next/navigation';

const NAV_ITEMS = [
 { href: '/', label: 'Home', icon: HomeIcon },
 { href: '/expenses', label: 'Expenses', icon: ListIcon },
 { href: '/money', label: 'Money', icon: WalletIcon },
 { href: '/reports', label: 'Reports', icon: ChartIcon },
 { href: '/more', label: 'More', icon: MoreIcon },
];

export function BottomNav() {
 const pathname = usePathname();

 return (
 <nav className="fixed bottom-0 left-0 right-0 z-50 glass-elevated border-
t border-border safe-area-bottom">
 <div className="max-w-lg mx-auto flex items-center justify-around h-16
px-2">
 {NAV_ITEMS.map(({ href, label, icon: Icon }) => {
 const isActive = href === '/' ? pathname === '/' :
pathname.startsWith(href);
 return (
 <Link
 key={href}
 href={href}

```

```

 className={`flex flex-col items-center justify-center gap-0.5
w-16 h-14 rounded-xl transition-all duration-200 ${
 isActive
 ? 'text-accent scale-105'
 : 'text-text-muted hover:text-text-secondary
active:scale-95'
 }}`
 >
 <Icon filled={isActive} />
 {label}

 </Link>
);
 })}
</div>
</nav>
);
}

```

```

// =====
// Inline SVG Icons (no external dependency)
// =====

```

```

function HomeIcon({ filled }: { filled: boolean }) {
 return (
 <svg width="22" height="22" viewBox="0 0 24 24" fill="none"
stroke="currentColor" strokeWidth={filled ? 2.5 : 2} strokeLinecap="round"
strokeLinejoin="round">
 <path d="M3 9l9-7 9 7v11a2 2 0 01-2 2H5a2 2 0 01-2 2z" fill={filled ?
'currentColor' : 'none'} opacity={filled ? 0.2 : 1} />
 <polyline points="9 22 9 12 15 12 15 22" />
 </svg>
);
}

```

```

function ListIcon({ filled }: { filled: boolean }) {
 return (
 <svg width="22" height="22" viewBox="0 0 24 24" fill="none"
stroke="currentColor" strokeWidth={filled ? 2.5 : 2} strokeLinecap="round"
strokeLinejoin="round">
 <rect x="3" y="3" width="18" height="18" rx="3" fill={filled ?
'currentColor' : 'none'} opacity={filled ? 0.2 : 1} />
 <line x1="8" y1="8" x2="16" y2="8" />
 <line x1="8" y1="12" x2="16" y2="12" />
 <line x1="8" y1="16" x2="13" y2="16" />
 </svg>
);
}

```

```

function WalletIcon({ filled }: { filled: boolean }) {
 return (
 <svg width="22" height="22" viewBox="0 0 24 24" fill="none"
stroke="currentColor" strokeWidth={filled ? 2.5 : 2} strokeLinecap="round"
strokeLinejoin="round">
 <rect x="2" y="5" width="20" height="15" rx="3" fill={filled ?
'currentColor' : 'none'} opacity={filled ? 0.2 : 1} />
 <path d="M16 12a1 1 0 100-2 1 1 0 000 2z" fill="currentColor" />
 <path d="M2 10h20" />
 </svg>
);
}

```

```

function ChartIcon({ filled }: { filled: boolean }) {
 return (
 <svg width="22" height="22" viewBox="0 0 24 24" fill="none"
stroke="currentColor" strokeWidth={filled ? 2.5 : 2} strokeLinecap="round"
strokeLinejoin="round">
 <rect x="3" y="3" width="18" height="18" rx="3" fill={filled ?
'currentColor' : 'none'} opacity={filled ? 0.2 : 1} />
 <line x1="8" y1="16" x2="8" y2="11" />
 <line x1="12" y1="16" x2="12" y2="8" />
 <line x1="16" y1="16" x2="16" y2="13" />
 </svg>
);
}

function MoreIcon({ filled }: { filled: boolean }) {
 return (
 <svg width="22" height="22" viewBox="0 0 24 24" fill="none"
stroke="currentColor" strokeWidth={filled ? 2.5 : 2} strokeLinecap="round"
strokeLinejoin="round">
 <circle cx="12" cy="6" r="1.5" fill="currentColor" />
 <circle cx="12" cy="12" r="1.5" fill="currentColor" />
 <circle cx="12" cy="18" r="1.5" fill="currentColor" />
 </svg>
);
}

...

`src\components\DeletePartyButton.tsx`

```tsx
'use client';

import { useRouter } from 'next/navigation';
import { deleteParty } from '@actions/parties';
import { useState } from 'react';

export function DeletePartyButton({ partyId, partyName }: { partyId: string,
partyName: string }) {
  const router = useRouter();
  const [isDeleting, setIsDeleting] = useState(false);

  const handleDelete = async () => {
    if (window.confirm(`Are you sure you want to delete ${partyName}? Their
historical records will be preserved, but they will be hidden from all
lists.`)) {
      setIsDeleting(true);
      try {
        await deleteParty(partyId);
        router.push('/more/people');
      } catch (err) {
        console.error(err);
        alert('Failed to delete person');
        setIsDeleting(false);
      }
    }
  };

  return (
    <button

```

```

        onClick={handleDelete}
        disabled={isDeleting}
        className="text-red-500 bg-red-500/10 hover:bg-red-500/20 px-4 py-2
rounded-xl text-sm font-semibold transition-colors disabled:opacity-50 flex
items-center gap-2"
      >
        <svg xmlns="http://www.w3.org/2000/svg" className="h-4 w-4" fill="none"
viewBox="0 0 24 24" stroke="currentColor">
          <path strokeLinecap="round" strokeLinejoin="round" strokeWidth={2}
d="M19 7l-.867 12.142A2 2 0 0116.138 21H7.862a2 2 0 01-1.995-1.858L5 7m5
4v6m4-6v6m1-10V4a1 1 0 00-1-1h-4a1 1 0 00-1-1v3M4 7h16" />
        </svg>
        {isDeleting ? 'Deleting...' : 'Delete'}
      </button>
    );
  }
}
...

```

```

## `src\components\Modal.tsx`

```

```

```.tsx
'use client';

import { useEffect, useRef } from 'react';

interface ModalProps {
 isOpen: boolean;
 onClose: () => void;
 title: string;
 children: React.ReactNode;
}

export function Modal({ isOpen, onClose, title, children }: ModalProps) {
 const overlayRef = useRef<HTMLDivElement>(null);

 useEffect(() => {
 const handleEscape = (e: KeyboardEvent) => {
 if (e.key === 'Escape') onClose();
 };

 if (isOpen) {
 document.addEventListener('keydown', handleEscape);
 document.body.style.overflow = 'hidden'; // Prevent scrolling
 }

 return () => {
 document.removeEventListener('keydown', handleEscape);
 document.body.style.overflow = 'unset';
 };
 }, [isOpen, onClose]);

 if (!isOpen) return null;

 return (
 <div
 className="fixed inset-0 z-50 flex items-end sm:items-center justify-
center p-4 bg-black/60 backdrop-blur-sm animate-fade-in"
 onClick={(e) => {
 if (e.target === overlayRef.current) onClose();
 }}
 >

```



```

 ref={overlayRef}
 >
 <div className="w-full max-w-lg bg-bg-card rounded-t-3xl sm:rounded-3xl
shadow-2xl overflow-hidden animate-slide-up sm:animate-fade-in flex flex-col
max-h-[90vh]">

 <div className="flex items-center justify-between p-5 border-b border-
border">
 <h2 className="text-xl font-bold">{title}</h2>
 <button
 onClick={onClose}
 className="w-8 h-8 flex items-center justify-center bg-bg-
elevated text-text-secondary rounded-full hover:bg-bg-input hover:text-text
transition-colors"
 >
 ,
 </button>
 </div>

 <div className="p-5 overflow-y-auto">
 {children}
 </div>

 </div>
 </div>
);
 }
}
...

```

```

`src\components\SearchableSelect.tsx`

```

```

````tsx
'use client';

import { useState, useRef, useEffect, useMemo } from 'react';

interface Option {
  id: string;
  label: string;
  subLabel?: string;
}

interface SearchableSelectProps {
  options: Option[];
  value: string;
  onChange: (value: string) => void;
  placeholder?: string;
  onAddNew?: (searchVal?: string) => void;
  addNewLabel?: string;
  label?: string;
  storageKey?: string; // Key to persist recent selections
}

export function SearchableSelect({
  options,
  value,
  onChange,
  placeholder = 'Search...',
  onAddNew,
  addNewLabel = '+ Add New',

```

```

    label,
    storageKey
  }: SearchableSelectProps) {
    const [isOpen, setIsOpen] = useState(false);
    const [search, setSearch] = useState('');
    const [recents, setRecents] = useState<string[]>([]);
    const wrapperRef = useRef<HTMLDivElement>(null);

    // Load recents on mount
    useEffect(() => {
      if (storageKey) {
        try {
          const stored = localStorage.getItem(`recent_${storageKey}`);
          if (stored) {
            setRecents(JSON.parse(stored));
          }
        } catch (e) {
          // ignore storage errors
        }
      }
    }, [storageKey]);

    // Close dropdown when clicking outside
    useEffect(() => {
      function handleClickOutside(event: MouseEvent) {
        if (wrapperRef.current && !wrapperRef.current.contains(event.target as
Node)) {
          setIsOpen(false);
        }
      }
      document.addEventListener('mousedown', handleClickOutside);
      return () => document.removeEventListener('mousedown',
handleClickOutside);
    }, []);

    const handleSelect = (id: string) => {
      onChange(id);
      setIsOpen(false);

      // Save to recents
      if (storageKey) {
        try {
          const updated = [id, ...recents.filter(r => r !== id)].slice(0, 3);
          setRecents(updated);
          localStorage.setItem(`recent_${storageKey}`, JSON.stringify(updated));
        } catch (e) {}
      }
    };

    const selectedOption = options.find(opt => opt.id === value);
    const displayValue = selectedOption ? selectedOption.label : '';

    const { recentOptions, normalOptions } = useMemo(() => {
      let filtered = options;
      if (search.trim()) {
        const lowerSearch = search.toLowerCase();
        filtered = options.filter(opt =>
          opt.label.toLowerCase().includes(lowerSearch) ||
          opt.subLabel?.toLowerCase().includes(lowerSearch)
        );
      }
      // If searching, don't separate recents

```

```

    return { recentOptions: [], normalOptions: filtered };
  }

  if (recents.length > 0) {
    const r = recents.map(id => options.find(o => o.id ===
id)).filter(Boolean) as Option[];
    const n = options.filter(o => !recents.includes(o.id));
    return { recentOptions: r, normalOptions: n };
  }

  return { recentOptions: [], normalOptions: options };
}, [options, search, recents]);

const displayedNormalOptions = useMemo(() => normalOptions.slice(0, 50),
[normalOptions]);

const renderOption = (opt: Option) => (
  <div
    key={opt.id}
    onClick={() => handleSelect(opt.id)}
    className={`px-3 py-3 rounded-lg cursor-pointer transition-colors flex
justify-between items-center ${
      value === opt.id
        ? 'bg-accent/10 text-accent font-semibold'
        : 'hover:bg-bg-elevated text-text-secondary hover:text-text'
      }`
    >
    <div className="flex flex-col">
      <span>{opt.label}</span>
      {opt.subLabel && <span className="text-xs text-text-muted
mt-0.5">{opt.subLabel}</span>}
    </div>
    {value === opt.id && <span>' </span>}}
  </div>
);

return (
  <div className="relative w-full" ref={wrapperRef}>
    {label && (
      <label className="block text-xs font-semibold text-text-muted
uppercase tracking-wider mb-2">
        {label}
      </label>
    )}

    {/* Selector Button */}
    <div
      onClick={() => {
        setIsOpen(true);
        setSearch(''); // Reset search when opening
      }}
      className={`w-full flex items-center justify-between bg-bg border
rounded-xl px-4 py-3 cursor-pointer transition-colors ${
        isOpen ? 'border-accent' : 'border-border hover:border-accent/50'
      }`
    >
      <span className={`truncate ${!selectedOption ? 'text-text-muted' :
'text-text font-medium'}`}>
        {selectedOption ? displayValue : placeholder}
      </span>
      <span className="text-text-muted text-xs ml-2">%4</span>
    </div>
  </div>
);

```

```

</div>

{/* Dropdown */}
{isOpen && (
  <div className="absolute z-40 top-full left-0 right-0 mt-2 bg-bg-card
border border-border rounded-xl shadow-2xl overflow-hidden animate-fade-in
flex flex-col max-h-[60vh]">

    {/* Search Input */}
    <div className="p-2 border-b border-border bg-bg-elevated sticky
top-0 z-10">
      <div className="relative">
        <span className="absolute left-3 top-1/2 -translate-y-1/2 text-
text-muted">0=Ý </span>
        <input
          type="text"
          placeholder="Search..."
          value={search}
          onChange={(e) => setSearch(e.target.value)}
          className="w-full bg-bg border border-border rounded-lg pl-9
pr-4 py-2.5 text-sm focus:outline-none focus:border-accent"
        />
      </div>
    </div>

    {/* Options List */}
    <div className="overflow-y-auto p-2 space-y-1">
      {recentOptions.length === 0 && normalOptions.length === 0 ? (
        <div className="text-center py-4 text-text-muted text-sm">
          No results found
        </div>
      ) : (
        <>
          {recentOptions.length > 0 && (
            <div className="mb-2">
              <div className="text-[10px] uppercase font-bold text-text-
muted px-2 mb-1 tracking-wider">Recent</div>
              {recentOptions.map(renderOption)}
              <div className="h-px bg-border my-2 mx-2" />
            </div>
          )}
          {displayedNormalOptions.map(renderOption)}
          {normalOptions.length > 50 && (
            <div className="text-center py-2 text-xs text-text-muted
italic">
              Showing top 50 results. Keep typing to filter more.
            </div>
          )}
        </>
      )}
    </div>

    {/* Add New Button */}
    {onAddNew && (
      <div className="p-2 border-t border-border bg-bg-elevated sticky
bottom-0 z-10">
        <button
          type="button"
          onClick={() => {
            setIsOpen(false);
            onAddNew(search);

```

```

        }}
        className="w-full py-2.5 bg-accent/10 text-accent hover:bg-
accent/20 rounded-lg text-sm font-semibold transition-colors flex items-
center justify-center gap-2"
      >
        <span>+</span> {search.trim() ? `Add "${search.trim()}"` :
addNewLabel}
      </button>
    </div>
  )}
</div>
)}
</div>
);
}
...

## `src\components\Skeletons.tsx`

```tsx
'use client';

/**
 * Reusable skeleton primitives for loading states.
 * Used by all loading.tsx files across the app.
 */

export function SkeletonLine({ width = 'w-full', height = 'h-4' }: { width?:
string; height?: string }) {
 return <div className={` ${width} ${height} bg-bg-elevated rounded-lg
animate-pulse-subtle`} />;
}

export function SkeletonCard({ height = 'h-20' }: { height?: string }) {
 return (
 <div className={`bg-bg-card border border-border rounded-2xl p-4
${height} animate-pulse-subtle`} />
);
}

export function SkeletonFormPage() {
 return (
 <div className="px-4 pt-6 pb-24 max-w-lg mx-auto animate-fade-in">
 { /* Back button + title */ }
 <div className="flex items-center mb-6 gap-3">
 <div className="w-8 h-8 bg-bg-elevated rounded-lg animate-pulse-
subtle" />
 <SkeletonLine width="w-48" height="h-7" />
 </div>

 { /* Form card */ }
 <div className="bg-bg-card border border-border rounded-2xl p-5 space-
y-4 mb-4">
 <SkeletonLine width="w-24" height="h-3" />
 <SkeletonCard height="h-12" />
 <SkeletonLine width="w-20" height="h-3" />
 <SkeletonCard height="h-12" />
 <SkeletonLine width="w-28" height="h-3" />
 <SkeletonCard height="h-12" />
 </div>
 </div>
);
}

```

```

 { /* Second card */
 <div className="bg-bg-card border border-border rounded-2xl p-5 space-
y-4 mb-4">
 <SkeletonLine width="w-32" height="h-3" />
 <SkeletonCard height="h-12" />
 <SkeletonLine width="w-24" height="h-3" />
 <SkeletonCard height="h-12" />
 </div>

 { /* Submit button */
 <div className="h-14 bg-bg-elevated rounded-2xl animate-pulse-subtle
mt-4" />
 </div>
);
}

export function SkeletonListPage() {
 return (
 <main className="p-4 max-w-lg mx-auto pb-24 animate-fade-in">
 <div className="flex justify-between items-center mb-6">
 <SkeletonLine width="w-40" height="h-7" />
 </div>
 <div className="space-y-3">
 {[...Array(6)].map((_, i) => (
 <SkeletonCard key={i} height="h-[72px]" />
))}
 </div>
 </main>
);
}

export function SkeletonHomePage() {
 return (
 <div className="px-4 pt-6 max-w-lg mx-auto pb-24 animate-fade-in">
 { /* Greeting */
 <div className="mb-6">
 <SkeletonLine width="w-40" height="h-7" />
 <div className="mt-2">
 <SkeletonLine width="w-24" height="h-4" />
 </div>
 </div>

 { /* Balance cards */
 <div className="grid grid-cols-2 gap-3 mb-4">
 <SkeletonCard height="h-20" />
 <SkeletonCard height="h-20" />
 </div>

 { /* Month spent */
 <div className="h-20" />

 { /* Quick entry */
 <div className="mt-6 mb-4">
 <SkeletonLine width="w-24" height="h-3" />
 <div className="grid grid-cols-2 gap-3 mt-3">
 {[...Array(6)].map((_, i) => (
 <SkeletonCard key={i} height="h-24" />
))}
 </div>
 </div>
 </div>
);
}

```

```

 </div>
);
}
...

`src\components\Toast.tsx`

```tsx
'use client';

import { createContext, useContext, useState, ReactNode } from 'react';

type ToastType = 'success' | 'error' | 'info';

interface Toast {
  id: number;
  message: string;
  type: ToastType;
}

interface ToastContextType {
  success: (message: string) => void;
  error: (message: string) => void;
  info: (message: string) => void;
}

const ToastContext = createContext<ToastContextType | undefined>(undefined);

export function ToastProvider({ children }: { children: ReactNode }) {
  const [toasts, setToasts] = useState<Toast[]>([]);

  const addToast = (message: string, type: ToastType) => {
    const id = Date.now();
    setToasts((prev) => [...prev, { id, message, type }]);
    setTimeout(() => {
      setToasts((prev) => prev.filter((t) => t.id !== id));
    }, 3000);
  };

  const success = (message: string) => addToast(message, 'success');
  const error = (message: string) => addToast(message, 'error');
  const info = (message: string) => addToast(message, 'info');

  return (
    <ToastContext.Provider value={{ success, error, info }}>
      {children}
      <div className="fixed bottom-24 left-1/2 -translate-x-1/2 z-50 flex
flex-col gap-2 items-center pointer-events-none w-full max-w-sm px-4">
        {toasts.map((toast) => (
          <div
            key={toast.id}
            className={`animate-fade-in px-4 py-3 rounded-2xl shadow-lg
border text-sm font-semibold flex items-center gap-2 w-full max-w-xs ${
              toast.type === 'success'
                ? 'bg-green/10 border-green/20 text-green'
                : toast.type === 'error'
                ? 'bg-red-light/10 border-red-light/20 text-red-light'
                : 'bg-bg-elevated border-border text-text-primary'
            }`}
          >

```

```

        <span>
            {toast.type === 'success' ? '' : toast.type === 'error' ? '& p ' :
'!9p '}
        </span>
        {toast.message}
    </div>
    )}}
</div>
</ToastContext.Provider>
);
}

export function useToast() {
    const context = useContext(ToastContext);
    if (!context) {
        throw new Error('useToast must be used within a ToastProvider');
    }
    return context;
}

...

## `src\components\TransactionCard.tsx`

```tsx
'use client';

import { formatCurrency, formatDateShort } from '@lib/utils';

interface TransactionCardProps {
 icon?: string;
 title: string;
 subtitle: string;
 amount: number | null;
 date: string;
 module?: string;
 isIncome?: boolean;
 onDelete?: () => void;
}

const MODULE_COLORS: Record<string, string> = {
 labour: 'border-l-amber',
 food: 'border-l-green',
 machinery: 'border-l-accent',
 salary: 'border-l-red',
 transport: 'border-l-accent-light',
 materials: 'border-l-amber',
 money_in: 'border-l-green',
 transfer: 'border-l-accent-light',
};

export function TransactionCard({
 icon = 'Ø=ÜÄ',
 title,
 subtitle,
 amount,
 date,
 module = 'default',
 isIncome,
 onDelete
}: TransactionCardProps) {

```



```

const borderColor = MODULE_COLORS[module] || 'border-l-border';

return (
 <div className={`bg-bg-card border border-border ${borderColor} border-
l-4 rounded-2xl p-4 flex items-center gap-3 shadow-sm group`}>
 <div className="w-10 h-10 rounded-xl bg-bg-elevated flex items-center
justify-center text-lg shrink-0">
 {icon}
 </div>
 <div className="flex-1 min-w-0">
 <h3 className="text-sm font-semibold truncate">{title}</h3>
 <p className="text-xs text-text-muted truncate mt-0.5">{subtitle}</p>
 </div>
 <div className="text-right shrink-0 flex items-center gap-2">
 <div className="flex flex-col items-end">
 {amount != null && amount > 0 && (
 <p className={`text-sm font-bold ${isIncome ? 'text-green' :
'text-text-primary'}`}>
 {isIncome ? '+' : ''}{formatCurrency(amount)}
 </p>
)}
 <p className="text-[10px] text-text-muted
mt-0.5">{formatDateShort(date)}</p>
 </div>

 {onDelete && (
 <button
 onClick={(e) => {
 e.preventDefault();
 e.stopPropagation();
 if (window.confirm('Are you sure you want to delete this
record?')) {
 onDelete();
 }
 }}
 className="w-8 h-8 flex items-center justify-center text-red-500
bg-red-500/10 rounded-lg hover:bg-red-500/20 opacity-0 group-
hover:opacity-100 transition-opacity"
 title="Delete"
 >
 <svg xmlns="http://www.w3.org/2000/svg" className="h-4 w-4"
fill="none" viewBox="0 0 24 24" stroke="currentColor">
 <path strokeLinecap="round" strokeLinejoin="round"
strokeWidth={2} d="M19 7l-.867 12.142A2 2 0 0116.138 21H7.862a2 2 0
01-1.995-1.858L5 7m5 4v6m4-6v6m1-10V4a1 1 0 00-1-1h-4a1 1 0 00-1 1v3M4 7h16" /
>
 </svg>
 </button>
)}
 </div>
 </div>
);
}

...

`src\lib\constants.ts`

```ts
// Default project ID used throughout the app (single-project MVP)
export const DEFAULT_PROJECT_ID = '10000000-0000-0000-0000-000000000000';

```

```

// Default account IDs
export const DEFAULT_CASH_ACCOUNT_ID = '00000000-0000-0000-0000-000000000000';

// Payment method labels
export const PAYMENT_METHOD_LABELS: Record<string, string> = {
  cash: 'Cash',
  bank_transfer: 'Bank Transfer',
  upi: 'UPI',
  other: 'Other',
};

// Party class labels
export const PARTY_CLASS_LABELS: Record<string, string> = {
  person: 'Person',
  shop: 'Shop',
  supplier: 'Supplier',
};

// Branding & Localization
export const APP_NAME = process.env.NEXT_PUBLIC_APP_NAME || 'Shambala';
export const CURRENCY = '₹';
...

## `src\lib\excel-parser.ts`

```ts
import * as XLSX from 'xlsx';

// =====
// TYPES
// =====

export interface ParsedRow {
 sheetName: string;
 rowNumber: number;
 rawData: Record<string, unknown>;
 parsedData: {
 date?: string;
 amount?: number;
 description?: string;
 category?: string;
 categoryId?: string;
 party?: string;
 type?: string;
 paymentMethod?: string;
 quantity?: number;
 unit?: string;
 rate?: number;
 };
 status: 'auto' | 'review' | 'rejected';
 confidence: number;
 reason: string;
}

export interface ParseResult {
 total: number;
 auto: number;
 review: number;
 rejected: number;
}

```

```

 rows: ParsedRow[];
}

// =====
// SHEET TYPE DETECTION
// =====

type SheetType = 'expenditure' | 'cash_input' | 'salary' | 'payment' |
'material' | 'unknown';

function detectSheetType(sheetName: string, headers: string[]): SheetType {
 const name = sheetName.toUpperCase();
 const headerStr = headers.join(' ').toUpperCase();

 if (name.includes('EX') || name.includes('EXPENDITURE') ||
headerStr.includes('EXPENDITURE')) {
 return 'expenditure';
 }
 if (name.includes('IN PUT') || name.includes('INPUT') ||
headerStr.includes('CASH AMOUNT')) {
 return 'cash_input';
 }
 if (name.includes('SALARY') || name.includes('SALAR')) {
 return 'salary';
 }
 if (name.includes('PAY MENT') || name.includes('PAYMENT')) {
 return 'payment';
 }
 // Material sheets: cement, sand, gravel, steel, bricks, RMC, stone, robo
 if (name.includes('CEMENT') || name.includes('SAND') ||
name.includes('GRAVEL') ||
 name.includes('STEEL') || name.includes('BRICK') ||
name.includes('RMC') ||
 name.includes('STONE') || name.includes('ROBO') ||
name.includes('INFRA') ||
 name.includes('READY MIX') || name.includes('VEDA') ||
headerStr.includes('NET WIGHT') || headerStr.includes('QUANTIY') ||
headerStr.includes('CUBICK')) {
 return 'material';
 }
 return 'unknown';
}

// =====
// DATE PARSING
// =====

function parseExcelDate(value: unknown): string | null {
 if (typeof value === 'number' && value > 40000 && value < 55000) {
 // Excel serial date
 const date = XLSX.SSF.parse_date_code(value);
 if (date) {
 return `${date.y}-${String(date.m).padStart(2, '0')}-${
String(date.d).padStart(2, '0')}`;
 }
 }
 if (typeof value === 'string') {
 // Try common text date formats
 const match = value.match(/(\d{1,2})[/-](\d{1,2})[/-](\d{2,4})/);
 if (match) {
 const day = parseInt(match[1]);

```

```

 const month = parseInt(match[2]);
 let year = parseInt(match[3]);
 if (year < 100) year += 2000;
 if (day > 0 && day <= 31 && month > 0 && month <= 12) {
 return `${year}-${String(month).padStart(2, '0')}-${
String(day).padStart(2, '0')}`;
 }
 }
}
return null;
}

// =====
// DESCRIPTION CLASSIFICATION
// =====

const CATEGORY_KEYWORDS: { pattern: RegExp; category: string; categoryId:
string }[] = [
 // Income patterns
 { pattern: /VARUN\s*SIR\s*CASH/i, category: 'Income', categoryId: '' },
 { pattern: /K\.?RAJESH\s*SIR\s*CASH/i, category: 'Income', categoryId: '' },
 { pattern: /CASH\s*(RECEIVED|INPUT)/i, category: 'Income', categoryId: '' },

 // Carry forward / balance
 { pattern: /CARRY\s*FORWARD/i, category: '_carry_forward', categoryId: '' },

 // Labour
 { pattern: /PAIRS?\s**?\d/i, category: 'Labour', categoryId:
'10000000-0000-0000-0000-000000000001' },
 { pattern: /MASON/i, category: 'Labour', categoryId:
'10000000-0000-0000-0000-000000000001' },
 { pattern: /DRILING|DRILLING/i, category: 'Labour', categoryId:
'10000000-0000-0000-0000-000000000001' },
 { pattern: /CARPENTER/i, category: 'Labour', categoryId:
'10000000-0000-0000-0000-000000000001' },
 { pattern: /RETAINING?\s*WALL/i, category: 'Labour', categoryId:
'10000000-0000-0000-0000-000000000001' },
 { pattern: /FLINTH\s*BEEM/i, category: 'Labour', categoryId:
'10000000-0000-0000-0000-000000000001' },
 { pattern: /SOIL\s*WORK/i, category: 'Labour', categoryId:
'10000000-0000-0000-0000-000000000001' },
 { pattern: /GUNEES\s*WORK/i, category: 'Labour', categoryId:
'10000000-0000-0000-0000-000000000001' },
 { pattern: /BLOSTING/i, category: 'Labour', categoryId:
'10000000-0000-0000-0000-000000000001' },
 { pattern: /SUPER\s*VISER|SUPERVISOR/i, category: 'Labour', categoryId:
'10000000-0000-0000-0000-000000000001' },
 { pattern: /WATCH\s*MAN/i, category: 'Labour', categoryId:
'10000000-0000-0000-0000-000000000001' },
 { pattern: /POP\s*/i, category: 'Labour', categoryId:
'10000000-0000-0000-0000-000000000001' },

 // Food & Groceries
 { pattern: /KIRANAM|KIRANA/i, category: 'Food & Groceries', categoryId:
'10000000-0000-0000-0000-000000000003' },
 { pattern: /VEG\b|VEGETABLE/i, category: 'Food & Groceries', categoryId:
'10000000-0000-0000-0000-000000000003' },
 { pattern: /MILK|MILLK/i, category: 'Food & Groceries', categoryId:
'10000000-0000-0000-0000-000000000003' },
 { pattern: /FOOD/i, category: 'Food & Groceries', categoryId:
'10000000-0000-0000-0000-000000000003' },

```

```

 { pattern: /WATER\s*BOTTLE/i, category: 'Food & Groceries', categoryId:
'10000000-0000-0000-0000-000000000003' },
 { pattern: /D\s*MART/i, category: 'Food & Groceries', categoryId:
'10000000-0000-0000-0000-000000000003' },
 { pattern: /SUPER\s*MARKET/i, category: 'Food & Groceries', categoryId:
'10000000-0000-0000-0000-000000000003' },
 { pattern: /OIL\s*\(?s*GO/i, category: 'Food & Groceries', categoryId:
'10000000-0000-0000-0000-000000000003' },
 { pattern: /EGGS?\b/i, category: 'Food & Groceries', categoryId:
'10000000-0000-0000-0000-000000000003' },
 { pattern: /TEA|SNACK/i, category: 'Food & Groceries', categoryId:
'10000000-0000-0000-0000-000000000003' },

 // Electrical
 { pattern: /ELECTRIC/i, category: 'Electrical', categoryId:
'10000000-0000-0000-0000-000000000004' },
 { pattern: /RAGHAVENDRA\s*ELECTRICAL/i, category: 'Electrical', categoryId:
'10000000-0000-0000-0000-000000000004' },

 // Transport / Equipment
 { pattern: /JCB/i, category: 'Equipment', categoryId:
'10000000-0000-0000-0000-000000000008' },
 { pattern: /DOZER/i, category: 'Equipment', categoryId:
'10000000-0000-0000-0000-000000000008' },
 { pattern: /TRACTOR/i, category: 'Equipment', categoryId:
'10000000-0000-0000-0000-000000000008' },
 { pattern: /AUTO\b/i, category: 'Transport', categoryId:
'10000000-0000-0000-0000-000000000005' },

 // Fuel
 { pattern: /DIESEL/i, category: 'Fuel & Diesel', categoryId:
'10000000-0000-0000-0000-000000000006' },
 { pattern: /FILING\s*STATION|FILLING\s*STATION/i, category: 'Fuel &
Diesel', categoryId: '10000000-0000-0000-0000-000000000006' },
 { pattern: /PETROL/i, category: 'Fuel & Diesel', categoryId:
'10000000-0000-0000-0000-000000000006' },

 // Salary
 { pattern: /SALARY/i, category: 'Salaries', categoryId:
'10000000-0000-0000-0000-000000000007' },
 { pattern: /BATHA/i, category: 'Salaries', categoryId:
'10000000-0000-0000-0000-000000000007' },

 // Materials
 { pattern: /CEMENT/i, category: 'Materials', categoryId:
'10000000-0000-0000-0000-000000000002' },
 { pattern: /BINDING\s*WIRE/i, category: 'Materials', categoryId:
'10000000-0000-0000-0000-000000000002' },
 { pattern: /COVERING\s*BOX|CAVARING\s*BOX/i, category: 'Materials',
categoryId: '10000000-0000-0000-0000-000000000002' },
 { pattern: /SANITARY/i, category: 'Materials', categoryId:
'10000000-0000-0000-0000-000000000002' },
 { pattern: /PLOSTICK|PLASTIC/i, category: 'Materials', categoryId:
'10000000-0000-0000-0000-000000000002' },
 { pattern: /GLASS\s*D00R/i, category: 'Materials', categoryId:
'10000000-0000-0000-0000-000000000002' },
 { pattern: /PIPE\s*LINE/i, category: 'Plumbing', categoryId:
'10000000-0000-0000-0000-000000000009' },

 // Site expenses
 { pattern: /JIO|SIM/i, category: 'Site Expenses', categoryId:
'10000000-0000-0000-0000-000000000010' },

```

```

 { pattern: /REGISTER|REGESTER/i, category: 'Site Expenses', categoryId:
'10000000-0000-0000-0000-000000000010' },
 { pattern: /TOKEN/i, category: 'Site Expenses', categoryId:
'10000000-0000-0000-0000-000000000010' },
 { pattern: /PEN|STATIONERY/i, category: 'Site Expenses', categoryId:
'10000000-0000-0000-0000-000000000010' },
 { pattern: /NEWS\s*PAPER/i, category: 'Site Expenses', categoryId:
'10000000-0000-0000-0000-000000000010' },
];

```

```

function extractPartyFromDescription(desc: string): string | null {
 // Common patterns: "NAME description" or "NAME quantity description"
 // Try to extract the first capitalized name-like segment
 const cleaned = desc.trim();

```

```

 // Known person patterns from the workbook

```

```

const personPatterns = [
 /^(B\.?BALAKRISHNA)/i,
 /^(P\.?JITENDER)/i,
 /^(S\.?KALYAN)/i,
 /^(SANDEEP\s+KIRANAM)/i,
 /^(SANDEEP\s+SUPER\s+MARKET)/i,
 /^(SK\s+KIRANAM)/i,
 /^(SAMBHAIAH)/i,
 /^(SRI\s+RAGHAVENDRA\s+ELECTRICAL)/i,
 /^(THIRUMALA\s+\w+)/i,
 /^(CHINTHALA\s+\w+)/i,
 /^(MALAHAL\s+RAO)/i,
 /^(A\.?UMESH)/i,
 /^(BABUL)/i,
 /^(K\.?SHARATH)/i,
 /^(R\.?MOGLI)/i,
 /^(VARUN\s+SIR)/i,
 /^(K\.?RAJESH\s+SIR)/i,
 /^(RANJITH\s+KIRANAM)/i,
 /^(ORAJABABU)/i,
 /^(ANADACHRY)/i,
 /^(ROHITH\s+\w+)/i,
 /^(J\.?GANESH)/i,
 /^(N\.?SATYANARAYANA)/i,
 /^(P\.?BALAKRISHAN)/i,
 /^(G\.?SUBBARAO)/i,
 /^(S\.?K\.?s*ABBAS)/i,
 /^(SUDHAKER)/i,
 /^(CH\.?SUNIL)/i,
 /^(P\.?KIRAN)/i,
 /^(DOZER\s+S\.?PRASAD)/i,
 /^(GANEH?SH?\s+JCB)/i,
 /^(SRI\s*MATHA)/i,
 /^(EKASHILA)/i,
 /^(SAIBABA)/i,
 /^(S\s*MART)/i,
 /^(GM\s+STONE)/i,
 /^(SUBBARAO)/i,
];

```

```

];

```

```

for (const pattern of personPatterns) {
 const match = cleaned.match(pattern);
 if (match) return match[1].trim();
}

```

```

 return null;
}

function classifyDescription(desc: string): { category: string; categoryId:
string; type: string } {
 for (const kw of CATEGORY_KEYWORDS) {
 if (kw.pattern.test(desc)) {
 if (kw.category === 'Income') {
 return { category: 'Income', categoryId: '', type: 'income' };
 }
 if (kw.category === '_carry_forward') {
 return { category: '_carry_forward', categoryId: '', type:
'opening_balance' };
 }
 return { category: kw.category, categoryId: kw.categoryId, type:
'expense' };
 }
 }
 return { category: 'Other', categoryId:
'100000000-0000-0000-0000-0000000000012', type: 'expense' };
}

// =====
// EXPENDITURE SHEET PARSER
// =====

function parseExpenditureSheet(sheetName: string, data: unknown[][]):
ParsedRow[] {
 const rows: ParsedRow[] = [];
 let currentDate: string | null = null;
 let pendingDescription = '';
 let pendingRow: ParsedRow | null = null;

 for (let i = 0; i < data.length; i++) {
 const row = data[i];
 if (!row || row.every(c => c === '' || c === null || c === undefined))
 continue;

 const col0 = row[0];
 const col1 = row[1];
 const col2 = row[2];
 const col3 = row[3];

 // Try to detect date row (Excel serial in col0 or standalone date)
 const dateFromCol0 = parseExcelDate(col0);
 if (dateFromCol0 && (!col1 || col1 === '')) {
 currentDate = dateFromCol0;
 continue;
 }

 // Detect header rows
 const headerStr = String(col0 || '') + String(col1 || '');
 if (headerStr.toUpperCase().includes('SNO') ||
headerStr.toUpperCase().includes('EXPENDITURE')) {
 continue;
 }

 // Detect total/summary rows
 if (typeof col0 === 'string' && col0.toUpperCase().includes('TOTAL'))
 continue;
 }
}

```

```

const desc = String(col1 || '').trim();
const credit = typeof col2 === 'number' ? col2 : null;
const debit = typeof col3 === 'number' ? col3 : null;

// Skip empty description rows with no amounts
if (!desc && !credit && !debit) continue;

// If there's a serial number in col0 (a number), this is a primary row
const hasSNO = typeof col0 === 'number' && col0 > 0 && col0 < 1000;

// If no SNO but has description, it's a continuation line
if (!hasSNO && desc && !credit && !debit) {
 if (pendingRow) {
 pendingRow.parsedData.description =
 (pendingRow.parsedData.description || '') + ' ' + desc;
 }
 continue;
}

// Flush pending row
if (pendingRow) {
 finalizeParsedRow(pendingRow);
 rows.push(pendingRow);
}

if (!desc && !hasSNO) {
 pendingRow = null;
 continue;
}

// Determine amount and type
const amount = debit || credit || null;
if (!amount) {
 pendingRow = null;
 continue;
}

const classification = classifyDescription(desc);

// Skip carry-forward rows
if (classification.category === '_carry_forward') {
 pendingRow = null;
 continue;
}

// If it's a credit with income classification, it's money in
// If it's a credit without income pattern, check if it's a carry-forward
or income
let txnType = classification.type;
if (credit && !debit) {
 txnType = 'income';
} else if (debit && !credit) {
 txnType = classification.type === 'income' ? 'income' : 'expense';
}

const party = extractPartyFromDescription(desc);

pendingRow = {
 sheetName,
 rowNumber: i + 1,
 rawData: { col0, col1: desc, col2: credit, col3: debit },

```



```

 parsedData: {
 date: currentDate || undefined,
 amount,
 description: desc,
 category: classification.category,
 categoryId: classification.categoryId || undefined,
 party: party || undefined,
 type: txnType,
 paymentMethod: 'cash',
 },
 status: 'auto',
 confidence: 0,
 reason: '',
 };
}

// Flush last pending row
if (pendingRow) {
 finalizeParsedRow(pendingRow);
 rows.push(pendingRow);
}

return rows;
}

function finalizeParsedRow(row: ParsedRow) {
 const d = row.parsedData;

 // Calculate confidence
 let confidence = 0.5;
 if (d.date) confidence += 0.15;
 if (d.amount && d.amount > 0) confidence += 0.15;
 if (d.category && d.category !== 'Other') confidence += 0.1;
 if (d.party) confidence += 0.1;

 row.confidence = Math.min(confidence, 1.0);
 row.status = confidence >= 0.8 ? 'auto' : confidence >= 0.5 ? 'review' :
'rejected';
 row.reason = buildReason(row);
}

function buildReason(row: ParsedRow): string {
 const parts: string[] = [];
 const d = row.parsedData;
 if (!d.date) parts.push('Missing date');
 if (!d.amount || d.amount <= 0) parts.push('Invalid amount');
 if (d.category === 'Other') parts.push('Could not classify category');
 if (!d.party) parts.push('Could not identify party');
 if (parts.length === 0) return 'Auto-classified';
 return parts.join('; ');
}

// =====
// CASH INPUT SHEET PARSER
// =====

function parseCashInputSheet(sheetName: string, data: unknown[][]):
ParsedRow[] {
 const rows: ParsedRow[] = [];

 // The "IN PUT" sheet has multiple side-by-side columns for different cash
sources

```

```

// Pattern: SNO, DATE, CASH AMOUNT (repeated for each source)
// Source names are in the rows above the headers

let sources: { col: number; name: string }[] = [];

for (let i = 0; i < data.length; i++) {
 const row = data[i];
 if (!row) continue;

 // Detect source names (rows with names like "VARUN SIR CASH")
 for (let c = 0; c < row.length; c++) {
 const val = String(row[c] || '').trim();
 if (val.includes('CASH') && !val.includes('SNO') && !
val.includes('AMOUNT') && val.length > 5) {
 // Find if there's a header row nearby
 sources.push({ col: c, name: val.replace(/\s*CASH\s*/i, '').trim() });
 }
 }

 // Detect data rows (numeric SNO + date + amount)
 for (const source of sources) {
 const snoCol = source.col;
 // Look for SNO column (might be source.col or source.col - 1)
 const lookCols = [snoCol, snoCol - 1, snoCol - 2].filter(c => c >= 0);

 for (const startCol of lookCols) {
 const sno = row[startCol];
 const dateVal = row[startCol + 1];
 const amountVal = row[startCol + 2];

 if (typeof sno === 'number' && sno > 0 && sno < 500) {
 const date = parseExcelDate(dateVal);
 const amount = typeof amountVal === 'number' && amountVal >= 100 ?
amountVal : null;

 if (date && amount) {
 rows.push({
 sheetName,
 rowNum: i + 1,
 rawData: { sno, date: dateVal, amount: amountVal, source:
source.name },
 parsedData: {
 date,
 amount,
 description: `Cash from ${source.name}`,
 category: 'Income',
 party: source.name,
 type: 'income',
 paymentMethod: 'cash',
 },
 status: 'auto',
 confidence: 0.9,
 reason: 'Cash input record',
 });
 break; // Don't double-count
 }
 }
 }
 }
}
}
}
}

```

```

// Deduplicate by rowNumber
const seen = new Set<string>();
return rows.filter(r => {
 const key = `${r.rowNumber}-${r.parsedData.amount}-${r.parsedData.party}`;
 if (seen.has(key)) return false;
 seen.add(key);
 return true;
});
}

// =====
// SALARY SHEET PARSER
// =====

function parseSalarySheet(sheetName: string, data: unknown[][]): ParsedRow[] {
 const rows: ParsedRow[] = [];

 for (let i = 0; i < data.length; i++) {
 const row = data[i];
 if (!row) continue;

 // Look for rows with: SNO, NAME, DESIGNATION, PAID SALARY, BANK, CASH
 const sno = row[0] || row[7]; // Two sections side by side
 const name = row[1] || row[8];
 const designation = row[2] || row[9];
 const salary = row[3] || row[10];

 if (typeof sno === 'number' && sno > 0 && sno < 100 && typeof name ===
'string' && name.length > 2) {
 const amount = typeof salary === 'number' ? salary : null;
 if (!amount || amount <= 0) continue;

 const bankCol = row[4] || row[11];
 const paymentMethod = String(bankCol ||
'').toUpperCase().includes('IDBI') ||
String(bankCol || '').toUpperCase().includes('NEFT') ?
'bank_transfer' : 'cash';

 rows.push({
 sheetName,
 rowNumber: i + 1,
 rawData: { sno, name, designation, salary, bank: bankCol },
 parsedData: {
 amount,
 description: `Salary - ${String(designation || '')}`,
 category: 'Salaries',
 categoryId: '10000000-0000-0000-0000-000000000007',
 party: String(name).trim(),
 type: 'salary',
 paymentMethod,
 },
 status: 'auto',
 confidence: 0.85,
 reason: 'Salary record',
 });
 }
 }

 return rows;
}

```

```

// =====
// PAYMENT SHEET PARSER
// =====

function parsePaymentSheet(sheetName: string, data: unknown[][]): ParsedRow[]
{
 const rows: ParsedRow[] = [];
 let currentPerson = '';

 for (let i = 0; i < data.length; i++) {
 const row = data[i];
 if (!row) continue;

 // Detect person name rows (text in col0 without numbers)
 const col0Str = String(row[0] || '').trim();
 if (col0Str.length > 3 && !col0Str.match(/^(SNO|TOTAL)/i) && typeof
row[0] === 'string') {
 // Check if this looks like a name (has letters, maybe has "PG NO")
 const nameMatch = col0Str.match(/^[A-Z][A-Z.\s]+/);
 if (nameMatch) {
 currentPerson = nameMatch[1].trim();
 continue;
 }
 }

 // Data rows: SNO, DATE, BYCASH, BANK
 const sno = row[0];
 const dateVal = row[1];
 const cashAmount = typeof row[2] === 'number' ? row[2] : null;
 const bankAmount = typeof row[3] === 'number' ? row[3] : null;

 if (typeof sno === 'number' && sno > 0 && sno < 500) {
 const date = parseExcelDate(dateVal);
 const amount = cashAmount || bankAmount;
 if (!date || !amount) continue;

 rows.push({
 sheetName,
 rowNumber: i + 1,
 rawData: { sno, date: dateVal, cash: cashAmount, bank: bankAmount,
person: currentPerson },
 parsedData: {
 date,
 amount,
 description: `Payment to ${currentPerson}`,
 party: currentPerson || undefined,
 type: 'supplier_payment',
 paymentMethod: bankAmount ? 'bank_transfer' : 'cash',
 },
 status: currentPerson ? 'auto' : 'review',
 confidence: currentPerson ? 0.85 : 0.6,
 reason: currentPerson ? 'Payment record' : 'Payment - person unclear',
 });
 }
 }

 return rows;
}

// =====
// MATERIAL SHEET PARSER

```

```
// =====

function parseMaterialSheet(sheetName: string, data: unknown[][]):
ParsedRow[] {
 const rows: ParsedRow[] = [];
 let supplier = '';
 let materialType = '';

 // Detect supplier and material from early rows
 for (let i = 0; i < Math.min(10, data.length); i++) {
 const row = data[i];
 if (!row) continue;
 for (const cell of row) {
 const val = String(cell || '').trim().toUpperCase();
 if (val.length > 3 && !val.match(/^(SNO|DATE|NO|SIZE|NET)/)) {
 if (!supplier && val.match(/[A-Z]{3,}/)) {
 // First significant text is likely supplier or material
 if (val.includes('SAND') || val.includes('CEMENT') ||
val.includes('GRAVEL') ||
val.includes('STEEL') || val.includes('BRICK') ||
val.includes('RMC') ||
val.includes('M-SAND') || val.includes('20MM') ||
val.includes('40MM') ||
val.includes('C-SAND')) {
 materialType = val;
 } else {
 supplier = val;
 }
 }
 }
 }
 }

 // Parse data rows
 for (let i = 0; i < data.length; i++) {
 const row = data[i];
 if (!row) continue;

 // Look for: SNO/NO, DATE, quantity/weight, ...
 for (let c = 0; c < row.length - 2; c++) {
 const sno = row[c];
 const dateVal = row[c + 1];
 const quantityVal = row[c + 2];

 if (typeof sno === 'number' && sno > 0 && sno < 500) {
 const date = parseExcelDate(dateVal);
 if (!date) continue;

 // Try to extract quantity
 let quantity: number | undefined;
 let unit = 'units';

 if (typeof quantityVal === 'number') {
 quantity = quantityVal;
 } else if (typeof quantityVal === 'string') {
 const numMatch = quantityVal.match(/(\d+[\d,]*)/);
 if (numMatch) quantity = parseInt(numMatch[1].replace(/,/g, ''));
 if (quantityVal.toUpperCase().includes('KG')) unit = 'kg';
 if (quantityVal.toUpperCase().includes('BAG')) unit = 'bags';
 if (quantityVal.toUpperCase().includes('M 3') ||
quantityVal.toUpperCase().includes('CUBICK')) unit = 'cubic m';

```

```

 }
 rows.push({
 sheetName,
 rowNum: i + 1,
 rawData: { sno, date: dateVal, quantity: quantityVal, supplier,
material: materialType },
 parsedData: {
 date,
 description: `${materialType || sheetName} delivery${supplier ? ' '
from ' + supplier : ''}`,
 category: 'Materials',
 categoryId: '10000000-0000-0000-0000-000000000002',
 party: supplier || undefined,
 type: 'supplier_purchase',
 quantity,
 unit,
 },
 status: 'review', // Material records often lack amounts, mark for
review
 confidence: 0.6,
 reason: 'Material delivery record (no amount in source)',
 });
 break; // Move to next row
 }
}
}
}

```

```

return rows;
}

```

```

// =====
// MAIN PARSE FUNCTION
// =====

```

```

export function parseWorkbook(buffer: ArrayBuffer): ParseResult {
 const wb = XLSX.read(buffer, { type: 'array' });
 const allRows: ParsedRow[] = [];

 for (const sheetName of wb.SheetNames) {
 const ws = wb.Sheets[sheetName];
 if (!ws || !ws['!ref']) continue;

 const data = XLSX.utils.sheet_to_json<unknown[]>(ws, { header: 1, defval:
'' });
 if (data.length < 3) continue;

 // Flatten first few rows for header detection
 const headerCells: string[] = [];
 for (let i = 0; i < Math.min(8, data.length); i++) {
 for (const cell of (data[i] as unknown[]) || []) {
 headerCells.push(String(cell || '').toUpperCase());
 }
 }

 const sheetType = detectSheetType(sheetName, headerCells);

 let parsed: ParsedRow[] = [];
 switch (sheetType) {
 case 'expenditure':

```

```

 parsed = parseExpenditureSheet(sheetName, data as unknown[][]);
 break;
 case 'cash_input':
 parsed = parseCashInputSheet(sheetName, data as unknown[][]);
 break;
 case 'salary':
 parsed = parseSalarySheet(sheetName, data as unknown[][]);
 break;
 case 'payment':
 parsed = parsePaymentSheet(sheetName, data as unknown[][]);
 break;
 case 'material':
 parsed = parseMaterialSheet(sheetName, data as unknown[][]);
 break;
 default:
 // Skip unknown sheets
 break;
 }

 allRows.push(...parsed);
}

// Deduplicate: flag potential duplicates across sheets
for (let i = 0; i < allRows.length; i++) {
 for (let j = i + 1; j < allRows.length; j++) {
 const a = allRows[i].parsedData;
 const b = allRows[j].parsedData;
 if (
 a.date === b.date &&
 a.amount === b.amount &&
 a.amount !== undefined &&
 a.party === b.party &&
 a.party !== undefined
) {
 // Mark the later one as duplicate
 allRows[j].status = 'review';
 allRows[j].confidence = Math.min(allRows[j].confidence, 0.4);
 allRows[j].reason = `Possible duplicate of row
${allRows[i].rowNumber} in ${allRows[i].sheetName}`;
 }
 }
}

const auto = allRows.filter(r => r.status === 'auto').length;
const review = allRows.filter(r => r.status === 'review').length;
const rejected = allRows.filter(r => r.status === 'rejected').length;

return {
 total: allRows.length,
 auto,
 review,
 rejected,
 rows: allRows,
};
}

...

`src\lib\supabase\client.ts`

```ts

```

```

import { createBrowserClient } from '@supabase/ssr';

export function createClient() {
  return createBrowserClient(
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!
  );
}

...

## `src\lib\supabase\middleware.ts`

```ts
import { createServerClient } from '@supabase/ssr';
import { NextResponse, type NextRequest } from 'next/server';

export async function updateSession(request: NextRequest) {
 let supabaseResponse = NextResponse.next({
 request,
 });

 const supabase = createServerClient(
 process.env.NEXT_PUBLIC_SUPABASE_URL!,
 process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!,
 {
 cookies: {
 getAll() {
 return request.cookies.getAll();
 },
 setAll(cookiesToSet) {
 cookiesToSet.forEach(({ name, value, options }) =>
 request.cookies.set(name, value));
 supabaseResponse = NextResponse.next({
 request,
 });
 cookiesToSet.forEach(({ name, value, options }) =>
 supabaseResponse.cookies.set(name, value, options)
);
 },
 },
 }
);

 const {
 data: { user },
 } = await supabase.auth.getUser();

 // Protect all routes except /login and /auth (for callbacks)
 if (
 !user &&
 !request.nextUrl.pathname.startsWith('/login') &&
 !request.nextUrl.pathname.startsWith('/auth') &&
 !request.nextUrl.pathname.match(/\.(ico|png|jpg|jpeg|svg)$/))
) {
 const url = request.nextUrl.clone();
 url.pathname = '/login';
 return NextResponse.redirect(url);
 }

 // If user is logged in and tries to go to login page, redirect to home

```



```

 if (user && request.nextUrl.pathname.startsWith('/login')) {
 const url = request.nextUrl.clone();
 url.pathname = '/';
 return NextResponse.redirect(url);
 }

 return supabaseResponse;
 }
 ...

`src\lib\supabase\server.ts`

```ts
import { createServerClient } from '@supabase/ssr';
import { cookies } from 'next/headers';

export async function createClient() {
  const cookieStore = await cookies();

  return createServerClient(
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!,
    {
      cookies: {
        getAll() {
          return cookieStore.getAll();
        },
        setAll(cookiesToSet) {
          try {
            cookiesToSet.forEach(({ name, value, options }) =>
              cookieStore.set(name, value, options)
            );
          } catch {
            // The `setAll` method was called from a Server Component.
            // This can be ignored if you have middleware refreshing user
            sessions.
          }
        }
      }
    }
  );
}
...

## `src\lib\types.ts`

```ts
// Shambala - App Types (ERP Model)

export type PaymentMethod = 'cash' | 'bank_transfer' | 'upi' | 'other';
export type PartyClass = 'person' | 'shop' | 'supplier';

export interface Project {
 id: string;
 name: string;
 description?: string;
}

export interface Account {

```

```

 id: string;
 name: string;
 type: 'cash' | 'bank' | 'upi' | 'other';
 initial_balance: number;
 is_default: boolean;
}

export interface AccountBalance {
 account_id: string;
 account_name: string;
 account_type: 'cash' | 'bank' | 'upi' | 'other';
 balance: number;
}

export interface Building {
 id: string;
 direction: string;
 number: string;
 name?: string;
 display_name: string;
}

export interface WorkerType {
 id: string;
 name: string;
}

export interface WorkType {
 id: string;
 name: string;
}

export interface Party {
 id: string;
 class: PartyClass;
 role: string;
 name: string;
 worker_type_id?: string;
 phone?: string;
 notes?: string;
 is_active: boolean;
 worker_type?: WorkerType; // Joined relation
}

export interface Machine {
 id: string;
 machine_id: string;
 machine_type: string;
 registration_number?: string;
 operator_id?: string;
 fuel_type?: string;
 meter_type?: string;
 is_active: boolean;
 operator?: Party; // Joined relation
}

export interface TransportVehicle {
 id: string;
 vehicle_number: string;
 vehicle_type: string;
 owner_id?: string;
}

```

```

 driver_id?: string;
 capacity?: string;
 notes?: string;
 driver?: Party; // Joined relation
}

export interface Material {
 id: string;
 class: 'bulk' | 'hardware' | 'electrical' | 'plumbing' | 'finishing' |
 'other';
 name: string;
 default_unit?: string;
}

export interface MaterialVariant {
 id: string;
 material_id: string;
 name: string;
}

export interface FoodCategory {
 id: string;
 name: string;
 parent_category?: string;
}

export interface Weighbridge {
 id: string;
 name: string;
 location?: string;
 default_fee?: number;
}

//

// Operational Records
//

export interface LabourRecord {
 id: string;
 worker_id: string;
 worker_type_id?: string;
 work_type_id?: string;
 building_id?: string;
 date: string;
 time?: string;
 amount?: number;
 cash_provider_id?: string;
 payment_method: PaymentMethod;
 comments?: string;

 worker?: Party;
 worker_type?: WorkerType;
 work_type?: WorkType;
 building?: Building;
 cash_provider?: Party;
}

export interface MaterialDelivery {
 id: string;

```

```

 material_id: string;
 variant_id?: string;
 supplier_id?: string;
 building_id?: string;
 date: string;
 time?: string;
 quantity?: number;
 unit?: string;
 material_cost?: number;
 comments?: string;

 material?: Material;
 variant?: MaterialVariant;
 supplier?: Party;
 building?: Building;
 transport?: TransportRecord;
 weighbridge?: WeighbridgeRecord;
}

export interface TransportRecord {
 id: string;
 delivery_id: string;
 vehicle_id?: string;
 transport_type?: string;
 source_location?: string;
 amount?: number;

 vehicle?: TransportVehicle;
}

export interface WeighbridgeRecord {
 id: string;
 delivery_id: string;
 weighbridge_id?: string;
 gross_weight?: number;
 tare_weight?: number;
 net_weight?: number;
 fee?: number;

 weighbridge?: Weighbridge;
}

export interface FoodRecord {
 id: string;
 worker_id?: string;
 worker_type_id?: string;
 food_category_id: string;
 start_date: string;
 end_date: string;
 quantity?: number;
 unit?: string;
 amount?: number;
 shop_id?: string;
 comments?: string;

 worker?: Party;
 food_category?: FoodCategory;
 shop?: Party;
}

export interface SalaryRecord {

```

```

 id: string;
 employee_id: string;
 worker_type_id?: string;
 start_date: string;
 end_date: string;
 amount: number;
 payment_date: string;
 payment_method: PaymentMethod;
 cash_provider_id?: string;
 comments?: string;

 employee?: Party;
 worker_type?: WorkerType;
 cash_provider?: Party;
}

export interface MachineryRecord {
 id: string;
 machine_id: string;
 operator_id?: string;
 building_id?: string;
 date: string;
 start_time?: string;
 end_time?: string;
 hours?: number;
 amount?: number;
 charging_type?: string;
 comments?: string;

 machine?: Machine;
 operator?: Party;
 building?: Building;
}

export interface FuelRecord {
 id: string;
 machine_id: string;
 building_id?: string;
 fuel_type: string;
 quantity: number;
 unit?: string;
 rate?: number;
 amount?: number;
 previous_meter?: number;
 current_meter?: number;
 date: string;
 provider_id?: string;
 comments?: string;

 machine?: Machine;
 building?: Building;
 provider?: Party;
}

...

`src\lib\utils.ts`

```ts
import { format, isToday, isYesterday, parseISO } from 'date-fns';

```

```

/** Format amount with 1 symbol and Indian numbering */
export function formatCurrency(amount: number): string {
  return '1' + amount.toLocaleString('en-IN', {
    minimumFractionDigits: 0,
    maximumFractionDigits: 0,
  });
}

/** Format date for display */
export function formatDate(dateStr: string): string {
  const date = parseISO(dateStr);
  if (isToday(date)) return 'Today';
  if (isYesterday(date)) return 'Yesterday';
  return format(date, 'dd-MM-yyyy');
}

/** Format date as short label (for cards) */
export function formatDateShort(dateStr: string): string {
  const date = parseISO(dateStr);
  if (isToday(date)) return 'Today';
  if (isYesterday(date)) return 'Yesterday';
  return format(date, 'd MMM');
}

/** Get today's date as YYYY-MM-DD */
export function todayISO(): string {
  return format(new Date(), 'yyyy-MM-dd');
}

/** Get current month/year label */
export function currentMonthLabel(): string {
  return format(new Date(), 'MMMM yyyy');
}

/** Get greeting based on time of day */
export function getGreeting(): string {
  const hour = new Date().getHours();
  if (hour < 12) return 'Good Morning';
  if (hour < 17) return 'Good Afternoon';
  return 'Good Evening';
}

...

## `src\middleware.ts`

```ts
import { type NextRequest } from 'next/server';
import { updateSession } from '@lib/supabase/middleware';

export async function middleware(request: NextRequest) {
 return await updateSession(request);
}

export const config = {
 matcher: [
 /*
 * Match all request paths except for the ones starting with:
 * - _next/static (static files)
 * - _next/image (image optimization files)
 * - favicon.ico (favicon file)
 */
]
}

```

```

 * Feel free to modify this pattern to include more paths.
 */
 '/((?!_next/static|_next/image|favicon.ico|.*\\.(?:svg|png|jpg|jpeg|gif|
webp)$).*)',
],
};
...

`supabase\migrations\001_schema.sql`

```sql
-- Shambala: Construction Site ERP Schema
-- Supabase PostgreSQL

CREATE EXTENSION IF NOT EXISTS "uuid-osspl";

-- =====
-- 1. CORE & FINANCIAL MASTERS
-- =====

CREATE TABLE projects (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  name TEXT NOT NULL,
  description TEXT,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE TYPE account_type AS ENUM ('cash', 'bank', 'upi', 'other');

CREATE TABLE accounts (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  project_id UUID NOT NULL REFERENCES projects(id) ON DELETE CASCADE,
  name TEXT NOT NULL,
  type account_type NOT NULL DEFAULT 'cash',
  initial_balance NUMERIC(15,2) NOT NULL DEFAULT 0,
  is_default BOOLEAN NOT NULL DEFAULT FALSE,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- =====
-- 2. OPERATIONAL MASTERS
-- =====

-- Buildings / Locations
CREATE TABLE buildings (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  project_id UUID NOT NULL REFERENCES projects(id) ON DELETE CASCADE,
  direction TEXT,
  number TEXT,
  name TEXT, -- Optional, e.g. "Villa 05"
  display_name TEXT GENERATED ALWAYS AS (COALESCE(direction || ' ' || number,
name)) STORED,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- Worker Types (e.g. Labour, Mason, Watchman)
CREATE TABLE worker_types (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  project_id UUID NOT NULL REFERENCES projects(id) ON DELETE CASCADE,
  name TEXT NOT NULL,

```

```

    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- Work Types (e.g. Soil Dressing, Cleaning)
CREATE TABLE work_types (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    project_id UUID NOT NULL REFERENCES projects(id) ON DELETE CASCADE,
    name TEXT NOT NULL,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- Parties (People, Shops, Suppliers)
CREATE TYPE party_class AS ENUM ('person', 'shop', 'supplier');

CREATE TABLE parties (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    project_id UUID NOT NULL REFERENCES projects(id) ON DELETE CASCADE,
    class party_class NOT NULL,
    role TEXT DEFAULT 'general',
    name TEXT NOT NULL,
    worker_type_id UUID REFERENCES worker_types(id) ON DELETE SET NULL, -- for
people
    phone TEXT,
    notes TEXT,
    is_active BOOLEAN NOT NULL DEFAULT TRUE,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- Machinery Master
CREATE TABLE machinery (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    project_id UUID NOT NULL REFERENCES projects(id) ON DELETE CASCADE,
    machine_id TEXT NOT NULL, -- e.g. "JCB-01"
    machine_type TEXT NOT NULL,
    registration_number TEXT,
    operator_id UUID REFERENCES parties(id) ON DELETE SET NULL,
    fuel_type TEXT,
    meter_type TEXT,
    is_active BOOLEAN NOT NULL DEFAULT TRUE,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- Transport Vehicles Master
CREATE TABLE transport_vehicles (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    project_id UUID NOT NULL REFERENCES projects(id) ON DELETE CASCADE,
    vehicle_number TEXT NOT NULL,
    vehicle_type TEXT NOT NULL,
    owner_id UUID REFERENCES parties(id) ON DELETE SET NULL,
    driver_id UUID REFERENCES parties(id) ON DELETE SET NULL,
    capacity TEXT,
    notes TEXT,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- Materials Master (Bulk & Small)
CREATE TYPE material_class AS ENUM ('bulk', 'hardware', 'electrical',
'plumbing', 'finishing', 'other');

CREATE TABLE materials (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),

```



```

    project_id UUID NOT NULL REFERENCES projects(id) ON DELETE CASCADE,
    class material_class NOT NULL DEFAULT 'bulk',
    name TEXT NOT NULL,
    default_unit TEXT,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- Material Variants (Sizes)
CREATE TABLE material_variants (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    material_id UUID NOT NULL REFERENCES materials(id) ON DELETE CASCADE,
    name TEXT NOT NULL, -- e.g. "8 mm", "9x4x3"
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- Food Categories
CREATE TABLE food_categories (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    project_id UUID NOT NULL REFERENCES projects(id) ON DELETE CASCADE,
    name TEXT NOT NULL,
    parent_category TEXT,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- General Store Categories
CREATE TABLE general_store_categories (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    project_id UUID NOT NULL REFERENCES projects(id) ON DELETE CASCADE,
    name TEXT NOT NULL,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- General Store Items
CREATE TABLE general_store_items (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    category_id UUID NOT NULL REFERENCES general_store_categories(id) ON DELETE
CASCADE,
    name TEXT NOT NULL,
    default_unit TEXT,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- Weighbridges Master
CREATE TABLE weighbridges (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    project_id UUID NOT NULL REFERENCES projects(id) ON DELETE CASCADE,
    name TEXT NOT NULL,
    location TEXT,
    default_fee NUMERIC(15,2),
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- =====
-- 3. CENTRAL FINANCIAL LEDGER
-- =====

CREATE TYPE transaction_type AS ENUM (
    'money_in',
    'transfer',
    'operational_expense',
    'general_expense',

```

```

        'supplier_payment',
        'opening_balance'
    );

CREATE TYPE payment_method AS ENUM ('cash', 'bank_transfer', 'upi', 'other');

CREATE TABLE transactions (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    project_id UUID NOT NULL REFERENCES projects(id) ON DELETE CASCADE,
    type transaction_type NOT NULL,
    date DATE NOT NULL DEFAULT CURRENT_DATE,
    amount NUMERIC(15,2) NOT NULL,
    account_id UUID REFERENCES accounts(id) ON DELETE SET NULL,
    to_account_id UUID REFERENCES accounts(id) ON DELETE SET NULL, -- transfers
    party_id UUID REFERENCES parties(id) ON DELETE SET NULL, -- payer/payee
    description TEXT,
    payment_method payment_method NOT NULL DEFAULT 'cash',
    reference_table TEXT, -- 'labour_records', 'food_records', etc.
    reference_id UUID, -- linking back to operational record
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- =====
-- 4. OPERATIONAL RECORDS
-- =====

-- Labour Records
CREATE TABLE labour_records (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    project_id UUID NOT NULL REFERENCES projects(id) ON DELETE CASCADE,
    worker_id UUID NOT NULL REFERENCES parties(id),
    worker_type_id UUID REFERENCES worker_types(id),
    work_type_id UUID REFERENCES work_types(id),
    building_id UUID REFERENCES buildings(id),
    date DATE NOT NULL,
    time TIME,
    amount NUMERIC(15,2),
    cash_provider_id UUID REFERENCES parties(id),
    payment_method payment_method DEFAULT 'cash',
    comments TEXT,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- Food Records
CREATE TABLE food_records (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    project_id UUID NOT NULL REFERENCES projects(id) ON DELETE CASCADE,
    worker_id UUID REFERENCES parties(id), -- Specific person or group
    worker_type_id UUID REFERENCES worker_types(id),
    food_category_id UUID NOT NULL REFERENCES food_categories(id),
    start_date DATE NOT NULL,
    end_date DATE NOT NULL,
    quantity NUMERIC(12,2),
    unit TEXT,
    amount NUMERIC(15,2),
    shop_id UUID REFERENCES parties(id),
    comments TEXT,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- Salary Records

```

```

CREATE TABLE salary_records (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  project_id UUID NOT NULL REFERENCES projects(id) ON DELETE CASCADE,
  employee_id UUID NOT NULL REFERENCES parties(id),
  worker_type_id UUID REFERENCES worker_types(id),
  start_date DATE NOT NULL,
  end_date DATE NOT NULL,
  amount NUMERIC(15,2) NOT NULL,
  payment_date DATE NOT NULL,
  payment_method payment_method NOT NULL,
  cash_provider_id UUID REFERENCES parties(id),
  comments TEXT,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

```

-- Machinery Records

```

CREATE TABLE machinery_records (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  project_id UUID NOT NULL REFERENCES projects(id) ON DELETE CASCADE,
  machine_id UUID NOT NULL REFERENCES machinery(id),
  operator_id UUID REFERENCES parties(id),
  building_id UUID REFERENCES buildings(id),
  date DATE NOT NULL,
  start_time TIME,
  end_time TIME,
  hours NUMERIC(10,2),
  amount NUMERIC(15,2),
  charging_type TEXT, -- 'Hourly', 'Daily', 'Fixed', etc.
  comments TEXT,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

```

-- Fuel Records

```

CREATE TABLE fuel_records (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  project_id UUID NOT NULL REFERENCES projects(id) ON DELETE CASCADE,
  machine_id UUID NOT NULL REFERENCES machinery(id),
  building_id UUID REFERENCES buildings(id),
  fuel_type TEXT NOT NULL,
  quantity NUMERIC(10,2) NOT NULL,
  unit TEXT DEFAULT 'Litres',
  rate NUMERIC(10,2),
  amount NUMERIC(15,2),
  previous_meter NUMERIC(15,2),
  current_meter NUMERIC(15,2),
  date DATE NOT NULL,
  provider_id UUID REFERENCES parties(id),
  comments TEXT,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

```

-- Material Deliveries

```

CREATE TABLE material_deliveries (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  project_id UUID NOT NULL REFERENCES projects(id) ON DELETE CASCADE,
  material_id UUID NOT NULL REFERENCES materials(id),
  variant_id UUID REFERENCES material_variants(id),
  supplier_id UUID REFERENCES parties(id),
  building_id UUID REFERENCES buildings(id),
  date DATE NOT NULL,
  time TIME,

```

```

    quantity NUMERIC(12,2),
    unit TEXT,
    material_cost NUMERIC(15,2), -- Without transport/weighbridge
    comments TEXT,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- Transport Records (Linked to Material Deliveries)
CREATE TABLE transport_records (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    delivery_id UUID NOT NULL REFERENCES material_deliveries(id) ON DELETE
    CASCADE,
    vehicle_id UUID REFERENCES transport_vehicles(id),
    transport_type TEXT,
    source_location TEXT,
    amount NUMERIC(15,2),
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- Weighbridge Records (Linked to Material Deliveries)
CREATE TABLE weighbridge_records (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    delivery_id UUID NOT NULL REFERENCES material_deliveries(id) ON DELETE
    CASCADE,
    weighbridge_id UUID REFERENCES weighbridges(id),
    gross_weight NUMERIC(12,3),
    tare_weight NUMERIC(12,3),
    net_weight NUMERIC(12,3) GENERATED ALWAYS AS (gross_weight - tare_weight)
    STORED,
    fee NUMERIC(15,2),
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- General Store Records
CREATE TABLE general_store_records (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    project_id UUID NOT NULL REFERENCES projects(id) ON DELETE CASCADE,
    item_id UUID NOT NULL REFERENCES general_store_items(id),
    shop_id UUID REFERENCES parties(id),
    quantity NUMERIC(10,2),
    unit TEXT,
    amount NUMERIC(15,2),
    date DATE NOT NULL,
    comments TEXT,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- General Site Expenses (Electricity, Internet, etc.)
CREATE TABLE general_site_expenses (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    project_id UUID NOT NULL REFERENCES projects(id) ON DELETE CASCADE,
    expense_category TEXT NOT NULL,
    amount NUMERIC(15,2) NOT NULL,
    date DATE NOT NULL,
    paid_to_id UUID REFERENCES parties(id),
    payment_method payment_method DEFAULT 'cash',
    comments TEXT,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- =====

```

```

-- INDEXES
-- =====
CREATE INDEX idx_labour_project ON labour_records(project_id);
CREATE INDEX idx_materials_project ON material_deliveries(project_id);
CREATE INDEX idx_food_project ON food_records(project_id);
CREATE INDEX idx_transactions_project ON transactions(project_id);

...

## `supabase\migrations\002_seed.sql`

```sql
-- Shambala: Initial Master Data Seed (ERP Model)

-- Ensure we insert into the default project with a specific UUID so it
matches our constants
INSERT INTO projects (id, name, description)
VALUES ('10000000-0000-0000-0000-000000000000', 'Narth Villas', 'Primary
construction project')
ON CONFLICT (id) DO NOTHING;

-- Initial Accounts
INSERT INTO accounts (id, project_id, name, type, initial_balance, is_default)
VALUES
 ('00000000-0000-0000-0000-000000000000',
 '10000000-0000-0000-0000-000000000000', 'Cash on Hand', 'cash', 0, TRUE),
 ('00000000-0000-0000-0000-000000000011',
 '10000000-0000-0000-0000-000000000000', 'IDBI Bank', 'bank', 0, FALSE)
ON CONFLICT (id) DO NOTHING;

-- Buildings / Locations
INSERT INTO buildings (project_id, direction, number, name)
VALUES
 ('10000000-0000-0000-0000-000000000000', 'North', '01', NULL),
 ('10000000-0000-0000-0000-000000000000', 'North', '02', NULL),
 ('10000000-0000-0000-0000-000000000000', 'North', '03', NULL),
 ('10000000-0000-0000-0000-000000000000', 'North', '05', NULL),
 ('10000000-0000-0000-0000-000000000000', 'West', '01', NULL),
 ('10000000-0000-0000-0000-000000000000', 'West', '02', NULL),
 ('10000000-0000-0000-0000-000000000000', 'East', '01', NULL),
 ('10000000-0000-0000-0000-000000000000', 'East', '02', NULL),
 ('10000000-0000-0000-0000-000000000000', 'East', '03', NULL),
 ('10000000-0000-0000-0000-000000000000', 'Common', 'Site', 'Site Store');

-- Worker Types
INSERT INTO worker_types (id, project_id, name)
VALUES
 ('20000000-0000-0000-0000-000000000001',
 '10000000-0000-0000-0000-000000000000', 'Labour'),
 ('20000000-0000-0000-0000-000000000002',
 '10000000-0000-0000-0000-000000000000', 'Mason'),
 ('20000000-0000-0000-0000-000000000003',
 '10000000-0000-0000-0000-000000000000', 'Helper'),
 ('20000000-0000-0000-0000-000000000004',
 '10000000-0000-0000-0000-000000000000', 'Electrician'),
 ('20000000-0000-0000-0000-000000000005',
 '10000000-0000-0000-0000-000000000000', 'Plumber'),
 ('20000000-0000-0000-0000-000000000006',
 '10000000-0000-0000-0000-000000000000', 'Carpenter'),
 ('20000000-0000-0000-0000-000000000007',
 '10000000-0000-0000-0000-000000000000', 'Watchman'),

```

```

('20000000-0000-0000-0000-00000000000008',
'10000000-0000-0000-0000-00000000000000', 'Security'),
('20000000-0000-0000-0000-00000000000009',
'10000000-0000-0000-0000-00000000000000', 'Office Boy'),
('20000000-0000-0000-0000-00000000000010',
'10000000-0000-0000-0000-00000000000000', 'House Help'),
('20000000-0000-0000-0000-00000000000011',
'10000000-0000-0000-0000-00000000000000', 'Driver'),
('20000000-0000-0000-0000-00000000000012',
'10000000-0000-0000-0000-00000000000000', 'Operator'),
('20000000-0000-0000-0000-00000000000013',
'10000000-0000-0000-0000-00000000000000', 'Contractor')
ON CONFLICT DO NOTHING;

```

-- Work Types

```

INSERT INTO work_types (project_id, name)
VALUES
('10000000-0000-0000-0000-00000000000000', 'Soil Dressing'),
('10000000-0000-0000-0000-00000000000000', 'Soil Leveling'),
('10000000-0000-0000-0000-00000000000000', 'Shifting'),
('10000000-0000-0000-0000-00000000000000', 'Cleaning'),
('10000000-0000-0000-0000-00000000000000', 'Steel Unloading'),
('10000000-0000-0000-0000-00000000000000', 'Brick Work'),
('10000000-0000-0000-0000-00000000000000', 'Masonry'),
('10000000-0000-0000-0000-00000000000000', 'Plastering'),
('10000000-0000-0000-0000-00000000000000', 'Painting'),
('10000000-0000-0000-0000-00000000000000', 'Electrical Work'),
('10000000-0000-0000-0000-00000000000000', 'Plumbing Work'),
('10000000-0000-0000-0000-00000000000000', 'Excavation'),
('10000000-0000-0000-0000-00000000000000', 'Loading'),
('10000000-0000-0000-0000-00000000000000', 'Unloading'),
('10000000-0000-0000-0000-00000000000000', 'Transportation');

```

-- Food Categories

```

INSERT INTO food_categories (id, project_id, name, parent_category)
VALUES
('30000000-0000-0000-0000-00000000000001',
'10000000-0000-0000-0000-00000000000000', 'Milk', 'Milk & dairy'),
('30000000-0000-0000-0000-00000000000002',
'10000000-0000-0000-0000-00000000000000', 'Curd', 'Milk & dairy'),
('30000000-0000-0000-0000-00000000000003',
'10000000-0000-0000-0000-00000000000000', 'Rice', 'Staples'),
('30000000-0000-0000-0000-00000000000004',
'10000000-0000-0000-0000-00000000000000', 'Dal', 'Staples'),
('30000000-0000-0000-0000-00000000000005',
'10000000-0000-0000-0000-00000000000000', 'Vegetables', 'Vegetables'),
('30000000-0000-0000-0000-00000000000006',
'10000000-0000-0000-0000-00000000000000', 'Chicken', 'Non-vegetarian'),
('30000000-0000-0000-0000-00000000000007',
'10000000-0000-0000-0000-00000000000000', 'Eggs', 'Non-vegetarian'),
('30000000-0000-0000-0000-00000000000008',
'10000000-0000-0000-0000-00000000000000', 'Meals', 'Food'),
('30000000-0000-0000-0000-00000000000009',
'10000000-0000-0000-0000-00000000000000', 'Snacks', 'Food'),
('30000000-0000-0000-0000-00000000000010',
'10000000-0000-0000-0000-00000000000000', 'Drinking water', 'Food'),
('30000000-0000-0000-0000-00000000000011',
'10000000-0000-0000-0000-00000000000000', 'Groceries', 'Other');

```

-- Materials Master (Bulk & Small)

-- Bulk Materials

```

INSERT INTO materials (id, project_id, class, name, default_unit)
VALUES
 ('40000000-0000-0000-0000-0000000000001',
 '10000000-0000-0000-0000-000000000000', 'bulk', 'Sand', 'Tonnes'),
 ('40000000-0000-0000-0000-0000000000002',
 '10000000-0000-0000-0000-000000000000', 'bulk', 'M-Sand', 'Tonnes'),
 ('40000000-0000-0000-0000-0000000000003',
 '10000000-0000-0000-0000-000000000000', 'bulk', 'Gravel', 'Tonnes'),
 ('40000000-0000-0000-0000-0000000000004',
 '10000000-0000-0000-0000-000000000000', 'bulk', 'Steel', 'Tonnes'),
 ('40000000-0000-0000-0000-0000000000005',
 '10000000-0000-0000-0000-000000000000', 'bulk', 'Bricks', 'Pieces'),
 ('40000000-0000-0000-0000-0000000000006',
 '10000000-0000-0000-0000-000000000000', 'bulk', 'Cement', 'Bags'),
 ('40000000-0000-0000-0000-0000000000007',
 '10000000-0000-0000-0000-000000000000', 'bulk', 'RMC', 'Cubic M'),
 ('40000000-0000-0000-0000-0000000000008',
 '10000000-0000-0000-0000-000000000000', 'plumbing', 'PVC Pipe', 'Pieces'),
 ('40000000-0000-0000-0000-0000000000009',
 '10000000-0000-0000-0000-000000000000', 'electrical', 'Wire', 'Rolls'),
 ('40000000-0000-0000-0000-0000000000010',
 '10000000-0000-0000-0000-000000000000', 'hardware', 'Screws', 'Boxes');

```

-- Material Variants

-- Steel

```

INSERT INTO material_variants (material_id, name) VALUES
 ('40000000-0000-0000-0000-0000000000004', '6 mm'),
 ('40000000-0000-0000-0000-0000000000004', '8 mm'),
 ('40000000-0000-0000-0000-0000000000004', '10 mm'),
 ('40000000-0000-0000-0000-0000000000004', '12 mm'),
 ('40000000-0000-0000-0000-0000000000004', '16 mm'),
 ('40000000-0000-0000-0000-0000000000004', '20 mm'),
 ('40000000-0000-0000-0000-0000000000004', '25 mm');

```

-- Bricks

```

INSERT INTO material_variants (material_id, name) VALUES
 ('40000000-0000-0000-0000-0000000000005', '9 x 4 x 3'),
 ('40000000-0000-0000-0000-0000000000005', '9 x 4 x 4');

```

-- PVC Pipe

```

INSERT INTO material_variants (material_id, name) VALUES
 ('40000000-0000-0000-0000-0000000000008', '1 inch'),
 ('40000000-0000-0000-0000-0000000000008', '2 inch'),
 ('40000000-0000-0000-0000-0000000000008', '3 inch'),
 ('40000000-0000-0000-0000-0000000000008', '4 inch');

```

-- General Store Categories

```

INSERT INTO general_store_categories (id, project_id, name)
VALUES
 ('50000000-0000-0000-0000-0000000000001',
 '10000000-0000-0000-0000-000000000000', 'Cleaning'),
 ('50000000-0000-0000-0000-0000000000002',
 '10000000-0000-0000-0000-000000000000', 'Adhesives'),
 ('50000000-0000-0000-0000-0000000000003',
 '10000000-0000-0000-0000-000000000000', 'Office'),
 ('50000000-0000-0000-0000-0000000000004',
 '10000000-0000-0000-0000-000000000000', 'Safety');

```

-- General Store Items

```

INSERT INTO general_store_items (category_id, name, default_unit)
VALUES

```

```

('50000000-0000-0000-0000-000000000001', 'Sponge', 'Pieces'),
('50000000-0000-0000-0000-000000000001', 'Broom', 'Pieces'),
('50000000-0000-0000-0000-000000000001', 'Floor cleaner', 'Litres'),
('50000000-0000-0000-0000-000000000002', 'Chippo glue', 'Kg'),
('50000000-0000-0000-0000-000000000002', 'Fevicol', 'Kg'),
('50000000-0000-0000-0000-000000000003', 'Register', 'Pieces'),
('50000000-0000-0000-0000-000000000003', 'Pens', 'Pieces'),
('50000000-0000-0000-0000-000000000004', 'Gloves', 'Pairs');

-- Machinery
INSERT INTO machinery (id, project_id, machine_id, machine_type, fuel_type,
meter_type)
VALUES
('60000000-0000-0000-0000-000000000001',
'10000000-0000-0000-0000-000000000000', 'JCB-01', 'JCB', 'Diesel', 'Hours'),
('60000000-0000-0000-0000-000000000002',
'10000000-0000-0000-0000-000000000000', 'Crane-01', 'Crane', 'Diesel',
'Hours'),
('60000000-0000-0000-0000-000000000003',
'10000000-0000-0000-0000-000000000000', 'Tractor-01', 'Tractor', 'Diesel',
'Hours');

...

`supabase\migrations\003_party_roles.sql`

```sql
-- Add role column to parties to separate labour, salaried, cash_in
(investor), etc.
ALTER TABLE parties ADD COLUMN role TEXT DEFAULT 'general';

-- Update existing worker types to have a role (optional, as role will
default to general)
UPDATE parties SET role = 'labour' WHERE worker_type_id IS NOT NULL;

...

## `tsconfig.json`

```json
{
 "compilerOptions": {
 "target": "ES2017",
 "lib": ["dom", "dom.iterable", "esnext"],
 "allowJs": true,
 "skipLibCheck": true,
 "strict": true,
 "noEmit": true,
 "esModuleInterop": true,
 "module": "esnext",
 "moduleResolution": "bundler",
 "resolveJsonModule": true,
 "isolatedModules": true,
 "jsx": "react-jsx",
 "incremental": true,
 "plugins": [
 {
 "name": "next"
 }
],
 "paths": {

```



```
 "@/*": ["/src/*"]
 }
},
"include": [
 "next-env.d.ts",
 "**/*.ts",
 "**/*.tsx",
 ".next/types/**/*.ts",
 ".next/dev/types/**/*.ts",
 "**/*.mts"
],
"exclude": ["node_modules"]
}
```
```